

Spark Ecosystem

Advanced / new topics of the Apache ecosystem

Spark Ecosystem

- › This section builds on previous classes
- › Some Hadoop/Spark Ecosystem knowledge is required/expected (e.g., Big Data by Adaltas)
- › In this section we will go beyond those to learn more about:
 - Advanced spark concepts and spark under the hood
 - Unit test in spark
 - Iceberg / delta tables
 - Lambda/Kappa Architectures
 - Kafka & Spark streaming

1

Apache Spark

Spark Streaming is a component of Apache Spark that enables scalable, high-throughput, and fault-tolerant stream processing of live data streams.

2

Unit testing in data engineering

Some practical tips how to run unit tests in spark

3

Data Lakehouse

Data Lakehouses are the future! Apache Iceberg is an open table format for large analytic datasets that supports schema evolution, hidden partitioning, and ACID transactions.

4

Lambda / Kappa

Learn about Lambda and Kappa architectures to support near real-time and analytical workloads in one unified architecture

5

Kafka & Spark Streaming

Learn about Kafka & Spark Streaming is a component of Apache Spark that enables scalable, high-throughput, and fault-tolerant stream processing of live data streams.

Apache Spark

Harness the power of big data processing

Apache Spark

Apache Spark is an open-source, **distributed computing system** designed for fast and efficient big data processing. It provides a unified analytics engine capable of handling large-scale data processing tasks, including batch processing, streaming, machine learning, and graph computation. Built on the concept of **resilient distributed datasets** (RDDs), Spark ensures fault tolerance and high reliability by distributing data and computations across clusters. Its **in-memory processing** capabilities significantly enhance speed compared to traditional disk-based systems, making it a popular choice for real-time analytics. Spark integrates seamlessly with big data tools like Hadoop and various storage systems, including HDFS, S3, and Kafka, making it versatile for diverse data engineering and analytics workflows. With APIs available in languages like Python, Java, Scala, and R, Spark caters to a wide range of developers and data professionals.

1

Hadoop Zoo & friends and its survivors

The Hadoop Zoo refers to the ecosystem of related tools and frameworks built around Hadoop, including components like HDFS, MapReduce, HBase, Hive, and others.

2

Spark Computational Model

The Spark computational model uses resilient distributed datasets (RDDs) to enable distributed, in-memory computations, optimizing performance and fault tolerance for big data processing across clusters.

3

AWS Glue

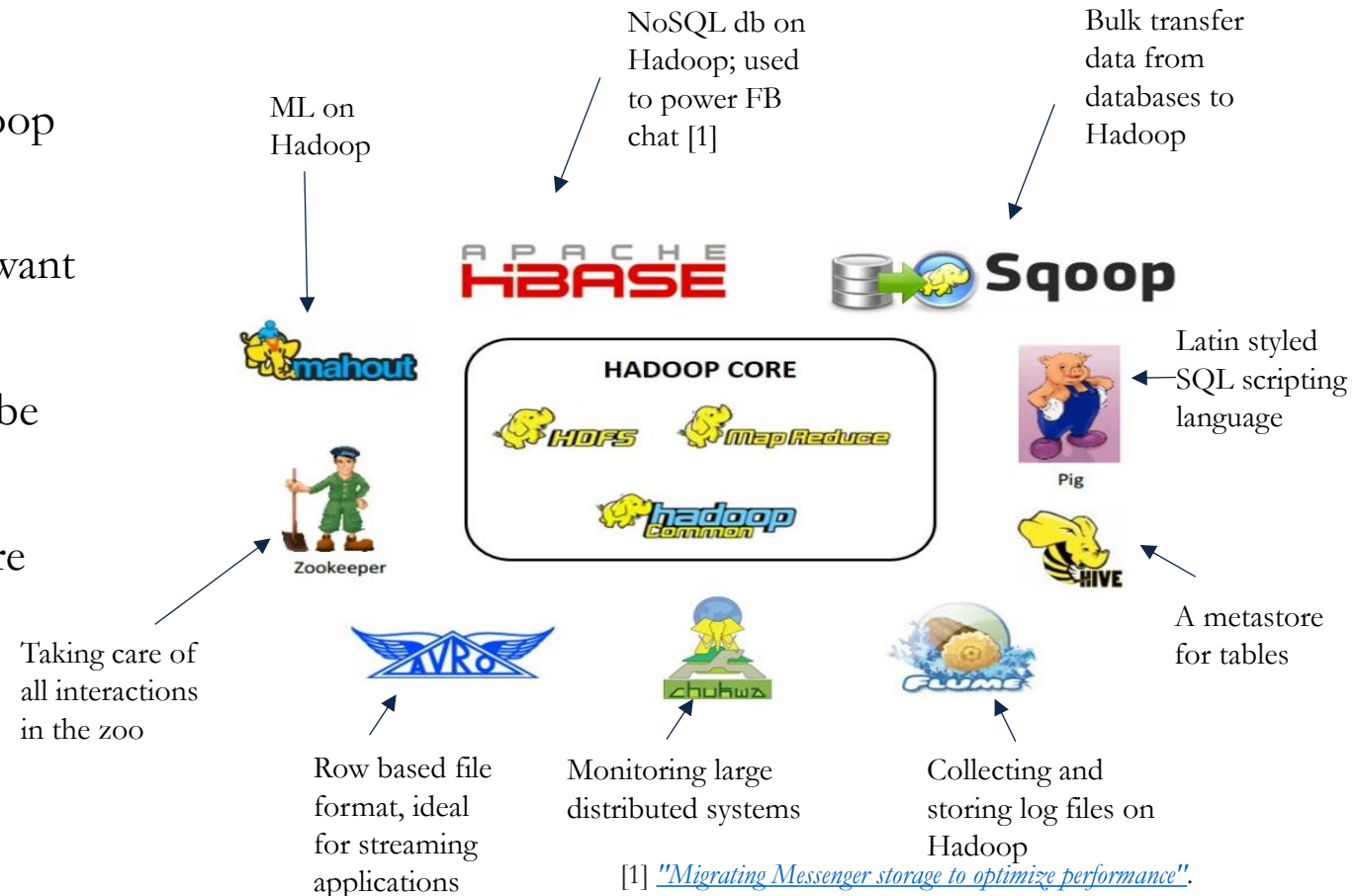
AWS Glue is a fully managed ETL (Extract, Transform, Load) service built on Apache Spark, making it easy to prepare and transform data for analytics, machine learning, and application development.

The good old days: Hadoop Ecosystem (Hadoop "zoo")

MapReduce & HDFS

- › Java MapReduce jobs read and write data to disk
- › Requires some scalable distributed file system (→ **H**adoop **D**istributed **F**ile **S**ystem)
- › Java is difficult to write, files are annoying to track, we want tables (→ Hive)
- › Writing files in every MapReduce step is slow, it would be more efficient to compute in memory (→ Spark)
- › Most other "animals" from the zoo "went extinct" or are about too
- › Survivor: Hive

Hadoop Zoo



[1] *"Migrating Messenger storage to optimize performance"*. www.facebook.com. 26 June 2018. Retrieved 5 July 2018.

Apache Parquet is a columnar storage file format optimized for efficient data processing and analytics, commonly used in big data environments.

Parquet File Format

- › Parquet uses columnar storage, enabling better compression and efficient scanning, which reduces the amount of data read from disk
- › Parquet supports schema evolution, allowing users to add new columns to existing data without affecting backward compatibility, which is essential for growing datasets.
- › Parquet's design aligns well with Spark's in-memory processing model, allowing for faster query execution.

Compression Algorithms

- › Run-Length Encoding (RLE): This simple algorithm compresses data by replacing sequences of repeated characters or data values with a single value and a count.
- › Huffman Coding: A lossless compression algorithm that assigns shorter codes to more frequent symbols and longer codes to less frequent ones.
- › Delta Encoding: This algorithm compresses data by storing the differences (deltas) between sequential data points

Demo: Run length compression

$$\begin{bmatrix} a \\ a \\ a \\ a \\ a \\ c \\ c \\ c \\ c \end{bmatrix} \rightarrow \begin{bmatrix} a[5] \\ c[4] \end{bmatrix}$$

How should you sort the data to save space on S3 etc?

Cheap to store integers, expansive to store long strings. Sort by country to make use of run-length-encoding!

Country	Year
GERMANY	2000
GERMANY	2001
GERMANY	2002
GERMANY	2003
FRANCE	2000
FRANCE	2001
FRANCE	2002
FRANCE	2003
FRANCE	2004

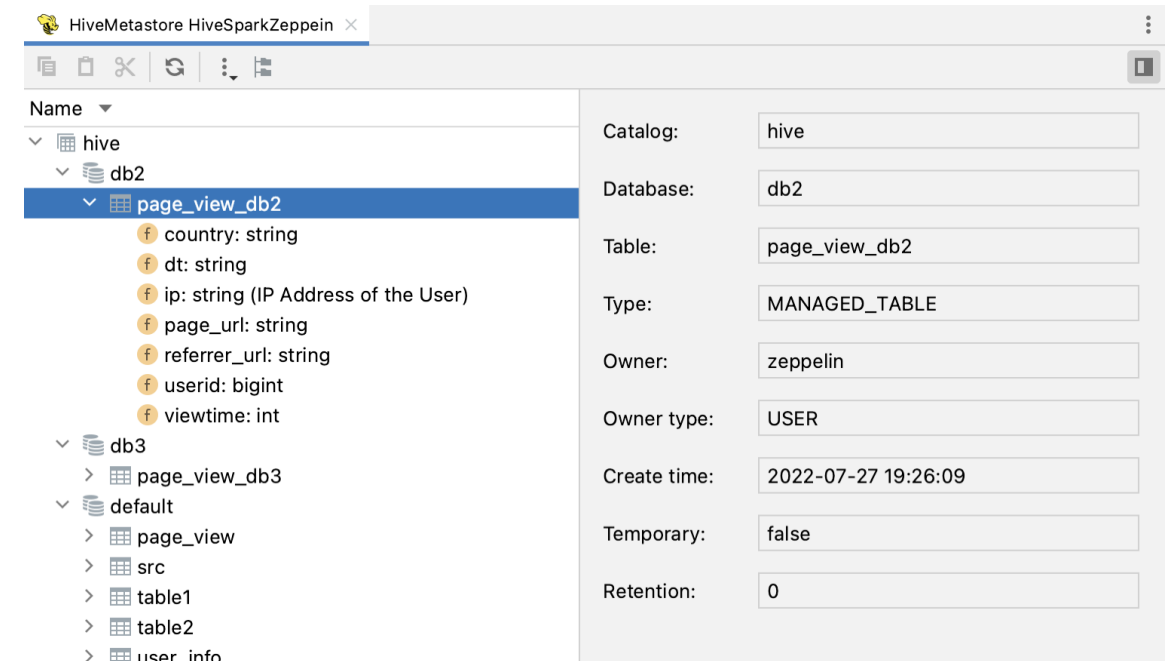
Country	Year
GERMANY	2000
FRANCE	2000
GERMANY	2001
FRANCE	2001
GERMANY	2002
FRANCE	2002
GERMANY	2003
FRANCE	2003
FRANCE	2004

Apache Hive is a data warehousing and SQL-like query language tool built on top of Hadoop for managing and querying large datasets

Hive Meta Store (HMS)

- › The **H**ive **M**eta **S**tore (HMS) is a crucial component of Apache Hive that acts as a system catalog or metadata repository. It stores metadata about the structure of Hive tables, databases, partitions, columns, and other objects. This metadata includes information like table schemas, column types, data locations, and storage formats.
- › The MetaStore is typically backed by a relational database such as MySQL, PostgreSQL, Oracle, or Derby.
- › The MetaStore is vital for query planning and execution, as Hive and other engines (Spark, ...) relies on it to retrieve metadata for constructing and optimizing execution plans.

Example



Apache Ranger is a Centralized Security Solution for Managing Data Access Across the Hadoop Ecosystem



It helps organizations manage access control policies

The primary goal of Apache Ranger is to provide a centralized platform for managing security policies consistently across various Hadoop services, including HDFS, Hive, HBase, Kafka, and more. It simplifies the process of setting up and enforcing data access controls:

- › Centralized Security Management: Allows administrators to create and manage access policies from a single interface.
- › Fine-Grained Access Control: Supports granular permissions, e.g., row/column level access.
- › Auditing and Monitoring: Provides detailed logs of access attempts and policy changes
- › Scalable and Flexible: Designed to scale with the growing data needs of modern enterprises, and adaptable to various security requirements.

User request

- A user queries data in impala, hive, ...
- Via JDBC connection or HUE

Ranger plugin

- Ranger plugin in each component forwards request to ranger server

Ranger server

- Ranger server checks for permissions in the underlying database
- Returns response to client which data a user gets to see

Understanding the Architecture of Apache Ranger: How Centralized Policy Management Enables Distributed Enforcement

It helps organizations manage access control policies

Ranger **Admin**: Web application for defining security policies.

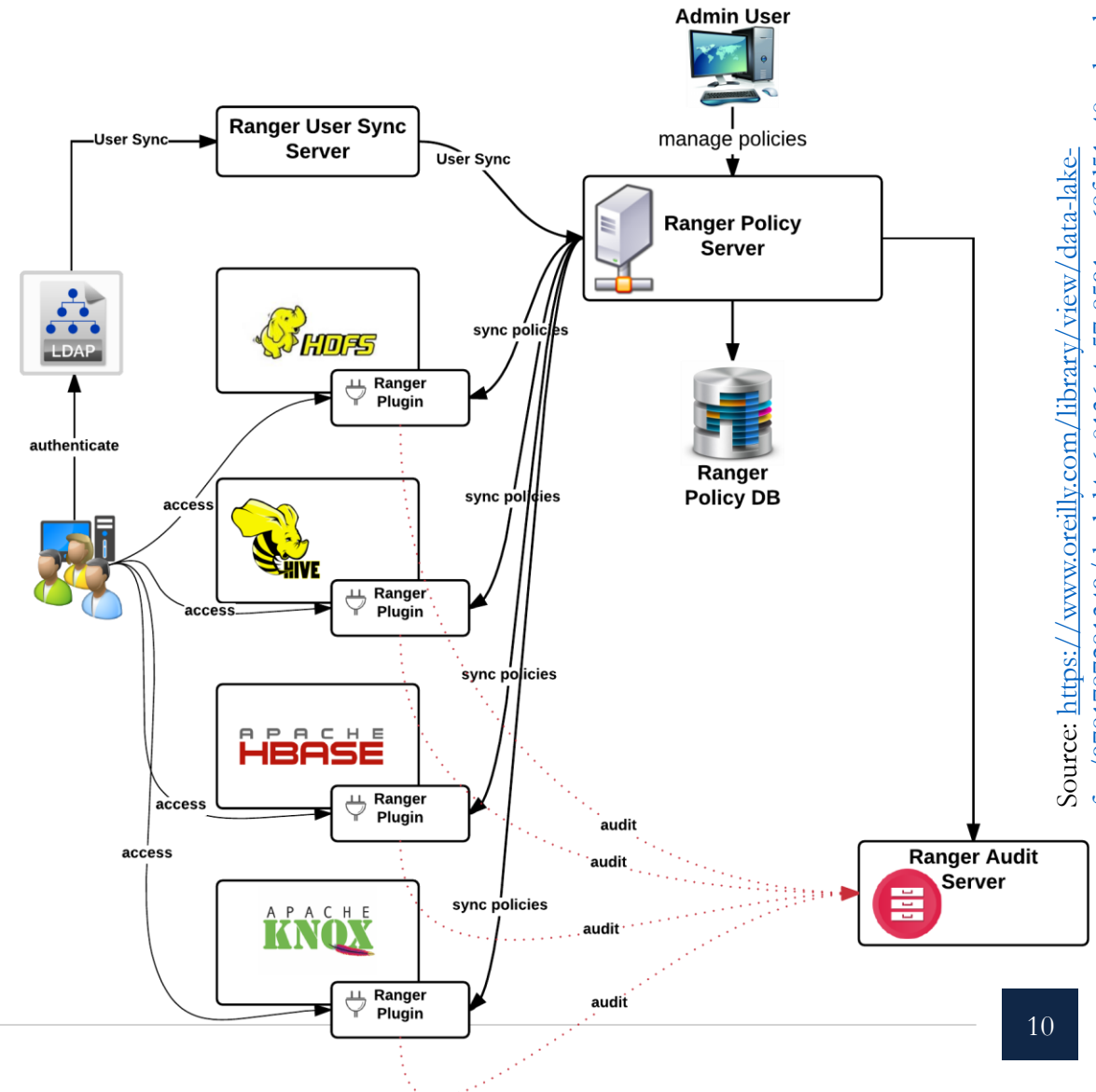
Ranger **UserSync**: Syncs users and groups from AD/LDAP.

Ranger **Plugins**: Embedded in Hadoop ecosystem components (HDFS, Hive, HBase, etc.).

Workflow:

- › Admin defines policies in Ranger Admin.
- › Policies are stored in a centralized database.
- › Plugins pull policies and enforce them locally.

Integration: Supports HDFS, Hive, HBase, Kafka, and more.



Effective Policy Management in Apache Ranger: How to Define, Apply, and Enforce Access Controls

Structuring and Implementing Policies for Optimal Data Security

Access policies: Define who can access which resources.

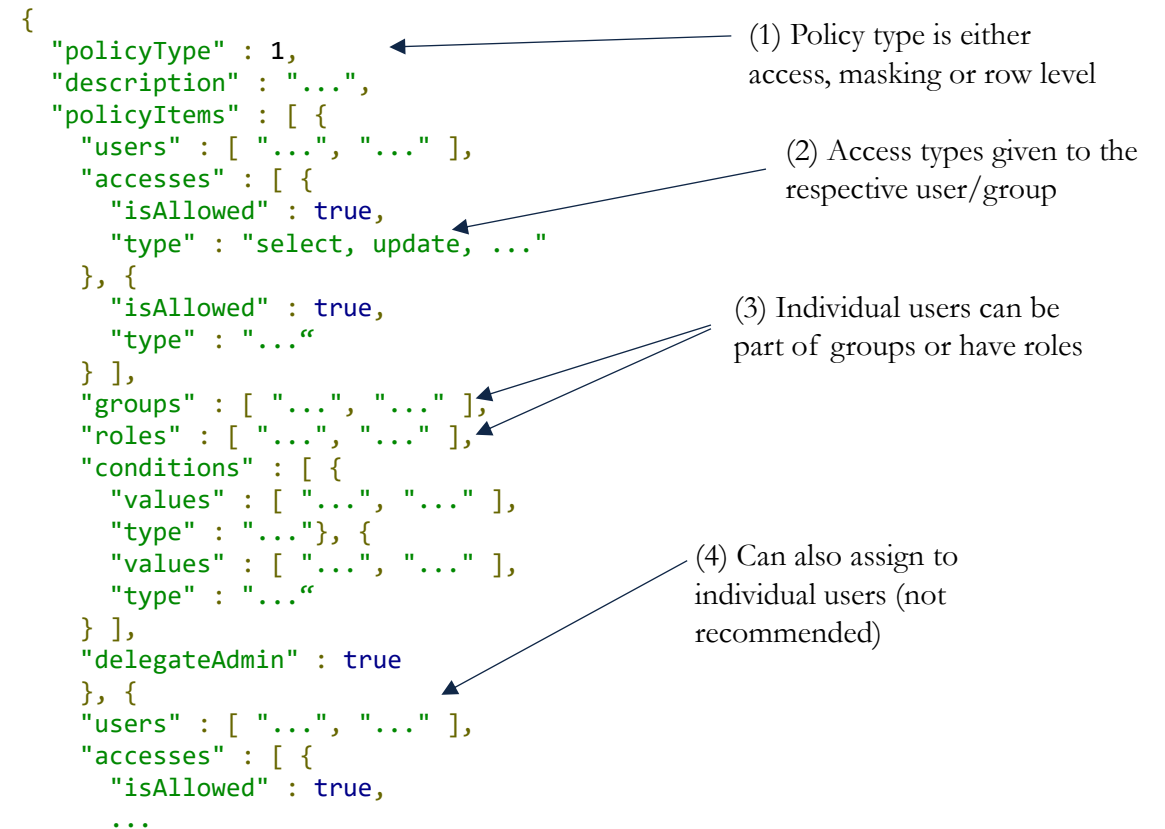
Masking policies: Define how sensitive data is masked.

Row-level filtering policies: Define filters on data rows.

Policy Components:

- › Resource: Specific dataset or component (e.g., Hive table). A resource is a triple (database/table/column)
- › Permissions: Actions that can be performed (e.g., read, write).
- › Users/Groups: Who the policy applies to.

Policy Structure



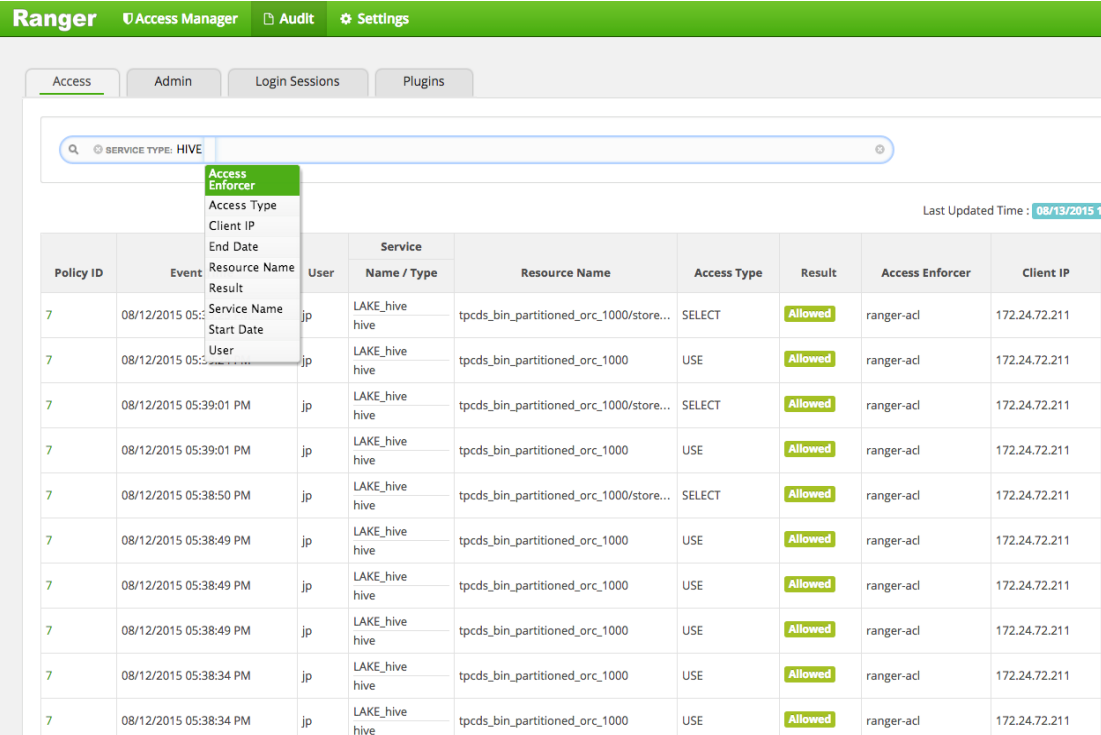
Example based on: https://ranger.apache.org/apidocs/json_RangerPolicy.html

Comprehensive Auditing and Monitoring in Apache Ranger: How to Track, Alert, and Report Access Activities for Enhanced Security and Compliance

Ensuring Visibility and Control Over Data Access

- › Detailed Audit Logs: Capture all access events centrally, providing a complete record of data access activities.
- › Real-Time Monitoring: Implement real-time monitoring to get immediate alerts for policy violations and unauthorized access attempts.
- › Comprehensive Reporting: Generate detailed reports on access patterns, user activities, and compliance with regulatory requirements. This includes periodic and on-demand reports that can be customized for specific needs.
- › Integration with Third-Party Tools: Seamlessly integrate audit logs with third-party monitoring tools like the ELK stack (Elasticsearch, Logstash, Kibana) for enhanced data visualization, search capabilities, and deeper insights.

Example Audit Log



Ranger Access Manager Audit Settings									
Access Admin Login Sessions Plugins									
SERVICE TYPE: HIVE									
Last Updated Time : 08/13/2015 12:00:00									
Policy ID	Event	User	Service Name / Type	Resource Name	Access Type	Result	Access Enforcer	Client IP	
7	08/12/2015 05:38:34 PM	jp	LAKE_hive hive	tpcds_bin_partitioned_orc_1000/store...	SELECT	Allowed	ranger-acl	172.24.72.211	
7	08/12/2015 05:38:34 PM	jp	LAKE_hive hive	tpcds_bin_partitioned_orc_1000	USE	Allowed	ranger-acl	172.24.72.211	
7	08/12/2015 05:39:01 PM	jp	LAKE_hive hive	tpcds_bin_partitioned_orc_1000/store...	SELECT	Allowed	ranger-acl	172.24.72.211	
7	08/12/2015 05:39:01 PM	jp	LAKE_hive hive	tpcds_bin_partitioned_orc_1000	USE	Allowed	ranger-acl	172.24.72.211	
7	08/12/2015 05:38:50 PM	jp	LAKE_hive hive	tpcds_bin_partitioned_orc_1000/store...	SELECT	Allowed	ranger-acl	172.24.72.211	
7	08/12/2015 05:38:49 PM	jp	LAKE_hive hive	tpcds_bin_partitioned_orc_1000	USE	Allowed	ranger-acl	172.24.72.211	
7	08/12/2015 05:38:49 PM	jp	LAKE_hive hive	tpcds_bin_partitioned_orc_1000	USE	Allowed	ranger-acl	172.24.72.211	
7	08/12/2015 05:38:49 PM	jp	LAKE_hive hive	tpcds_bin_partitioned_orc_1000	USE	Allowed	ranger-acl	172.24.72.211	
7	08/12/2015 05:38:34 PM	jp	LAKE_hive hive	tpcds_bin_partitioned_orc_1000	USE	Allowed	ranger-acl	172.24.72.211	
7	08/12/2015 05:38:34 PM	jp	LAKE_hive hive	tpcds_bin_partitioned_orc_1000	USE	Allowed	ranger-acl	172.24.72.211	

Source: <https://cwiki.apache.org/confluence/display/RANGER/Install+and+Configure+Solr+for+Ranger+Audits+-+Apache+Ranger+0.5>

Spark Computational Model

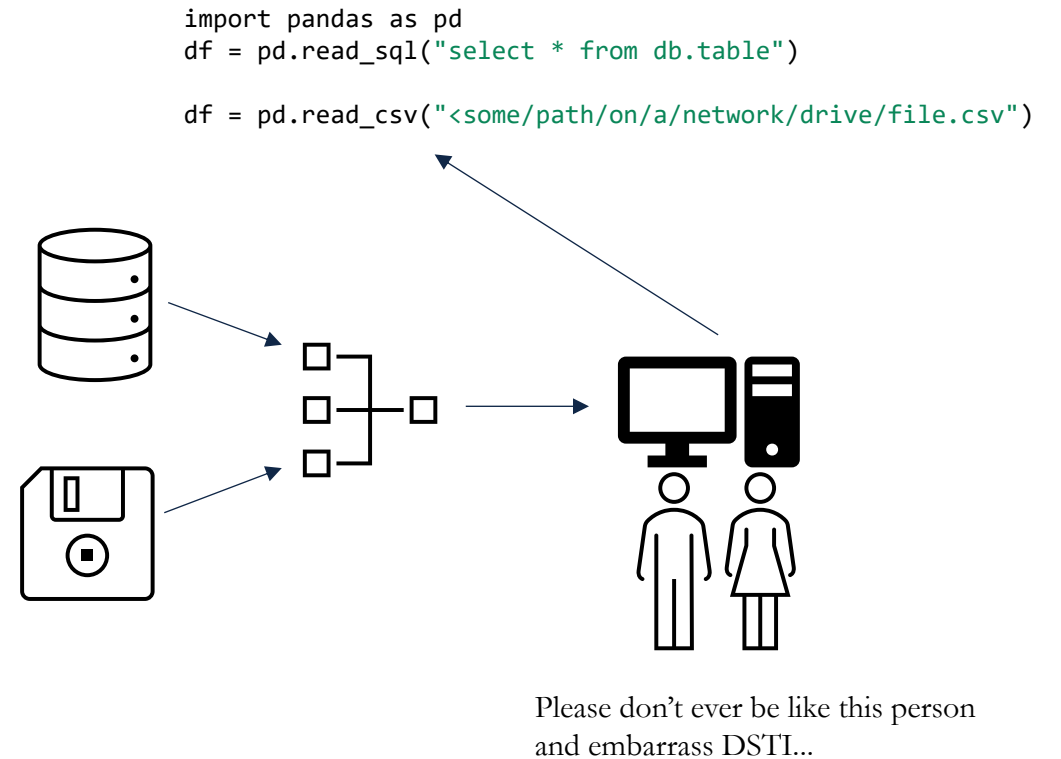
Harness the power of big data processing

Big Data Processing (if you have no idea what you are doing)

Brute force approach

- › Data is centrally located either in a database (best-case)
- › Or in file(s) on some network drive (worst-case)
- › Beginners would read in all the data to their computer (or ask for a high-powered machine)
- › Then use their R/Python/matlab/Stata/... to do data analysis
- › This setup does not scale. Fine for super small explorative data analysis
- › No go for any kind of production setup.
- › Any uses your data product downstream? Don't

Schematic

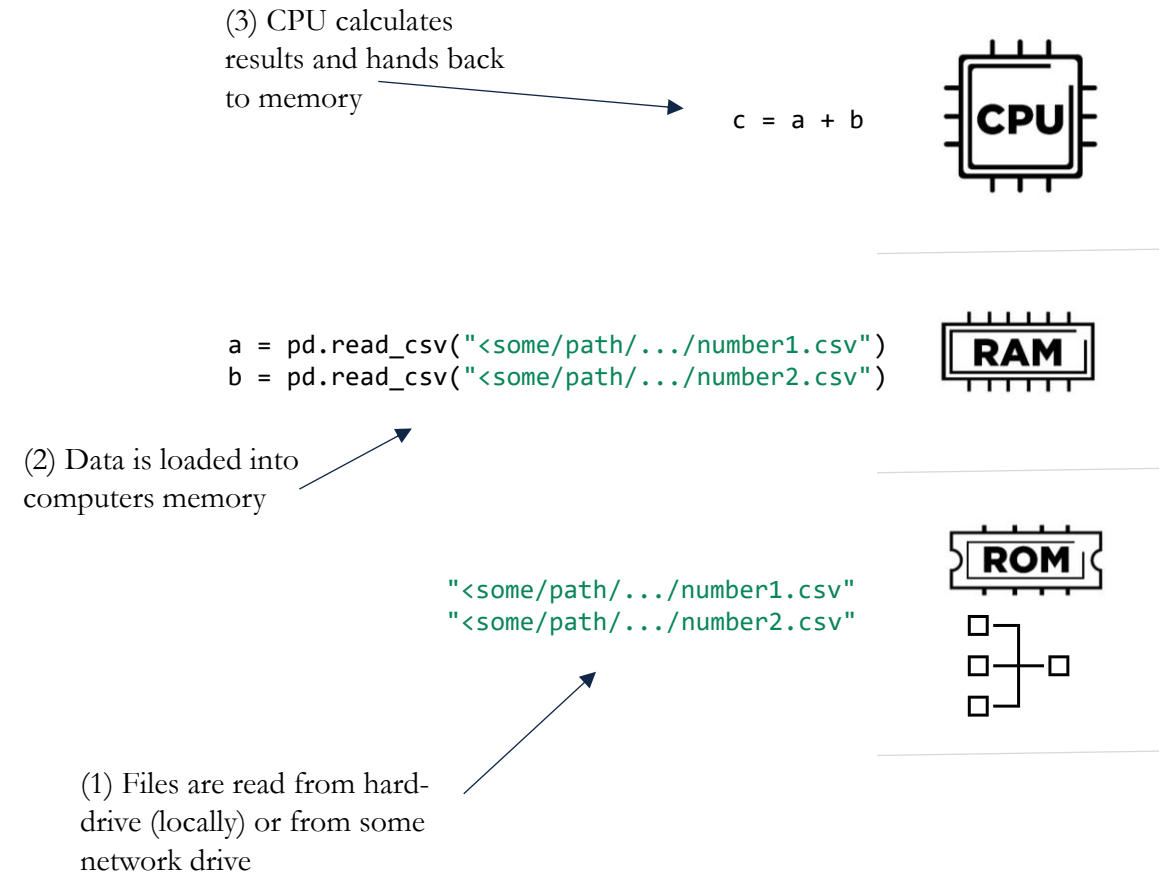


A PC works by reading data from harddrive/network into memory from where the CPU can take data to make fast operations

How does a computer compute?

- › Data is persisted somewhere (e.g., on **Read-Only Memory** (ROM), i.e., hard-drive or somewhere on a network)
 - › This can be (very, very) slow to read from
- › Data is loaded into **Random Access Memory** (RAM)
 - › Depending on how much memory you have, this can become a bottleneck if you data files > memory
- › **Central Processing Unit** (CPU) is used to perform calculations
 - › This can fast, depending on the calculation
 - › If you can parallelize it well, more cores can significantly speed up the problem
 - › Side note: for some problems (matrix algebra, aka ML/ AI) GPU performs better

Schematic



Three problems when analyzing data “old school”: poor network performance, time complexity and space complexity

(poor) network performance

When dealing with large files stored on a network drive, the speed at which data can be transferred to your local machine is heavily dependent on the network's performance. If the network is congested or the connection is slow, it can create a significant bottleneck in the data pipeline, causing delays in data retrieval. This lag can be especially frustrating when you need to frequently access or stream large datasets, as each request might take much longer than anticipated. The result is a workflow hampered by extended wait times, reducing overall productivity and potentially leading to inconsistent or incomplete data being loaded for analysis.

Time complexity

The number of cores in your computer determines its ability to perform parallel processing, which is essential for handling computationally intensive tasks, especially with large datasets. When you have a limited number of cores, your machine can only perform a certain amount of calculations simultaneously. This limitation becomes evident as the dataset grows larger, requiring more computational power to process in a reasonable time.. This bottleneck can be particularly problematic when dealing with complex algorithms or when trying to scale your analysis to meet the demands of big data.

Space complexity

Large datasets often exceed the available memory (RAM) on a typical computer, leading to critical performance issues. When your machine attempts to load a dataset that is too large for the available memory, it resorts to using disk space as an extension of RAM, a process known as "paging" or "swapping." This operation is considerably slower than using RAM alone, causing the system to become sluggish. In extreme cases, the machine may freeze or crash. Even when the system remains operational, the time required to load and manipulate the data increases dramatically.

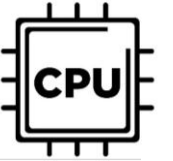
Recap: Time complexity is about CPU bound problems; Space complexity is about memory bound problems

Fibonacci



- › Consider the Fibonacci sequence:
 - › $F(n) = F(n - 1) + F(n - 2) \rightarrow 0, 1, 1, 2, 3, 5, 8, 13, 21, ?$
- › Space complexity?
 - › Step 1: 0, 1
 - › Step 2: 1, 1
 - › Step 3: 1, 2
 - › Step 4: 2, 3
 - › Need only ever two numbers in memory $O(1)$
- › Time complexity?
 - › Need n calculations to get the n -th number
 - › Linear time complexity: I cannot split up the work
 - › $O(n)$ time complexity

Matrix Algebra



- › $100,000 \times 100,000$ matrix of zeros
- › Add 1 to each cell. What is the result?
- › $100,000 \times 100,000$ matrix of ones
- › Let python do it:

```
import numpy as np
N=100_000
zeros = np.zeros((N, N))
```

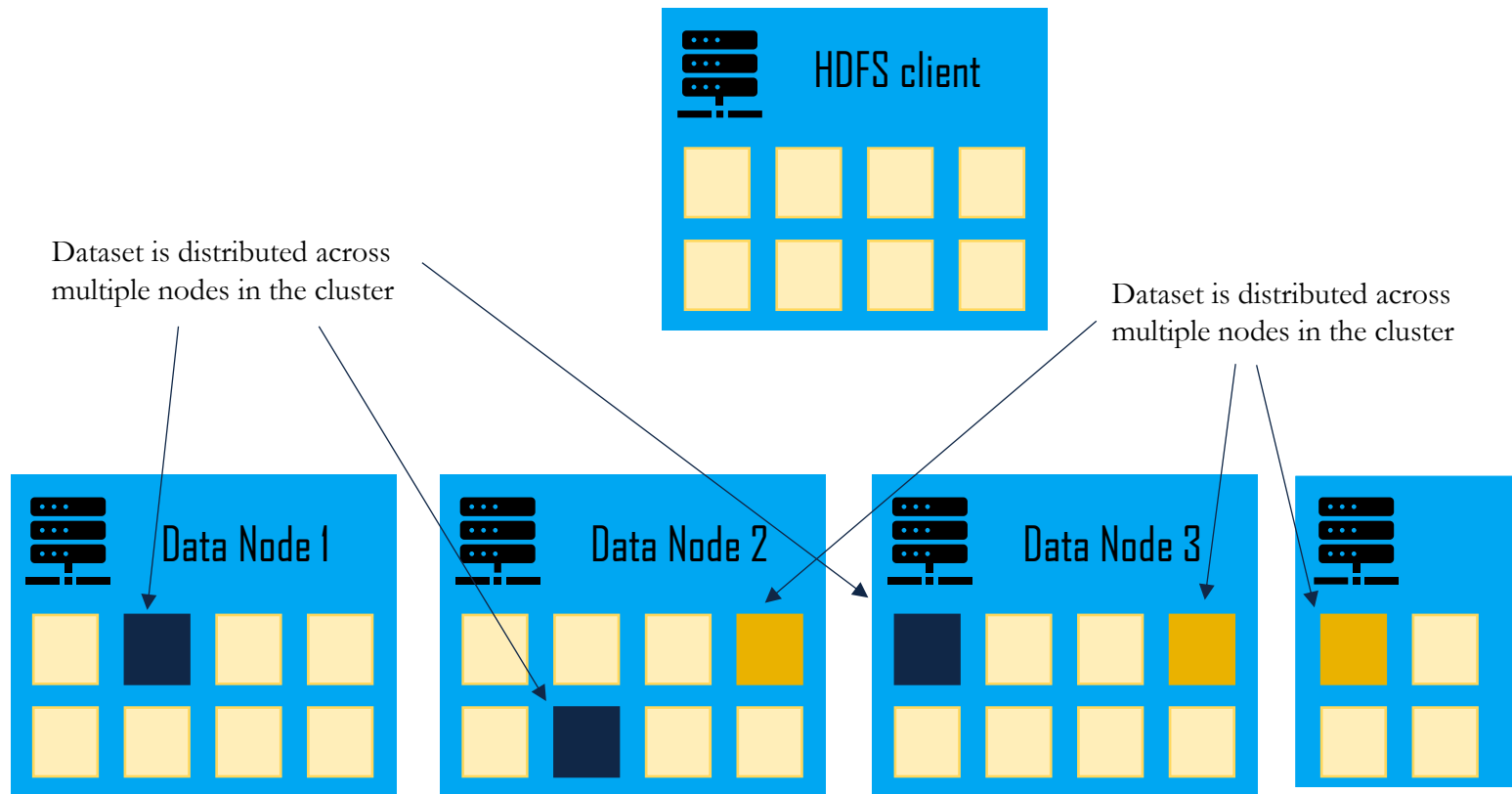
>> **MemoryError:** Unable to allocate 74.% GiB for an array with shape (100000, 100000) and data type float
- › We would need 75GB of available memory to populate the matrix
 - › Note that we have not even computed anything (no the “+1” part yet), just initializing the matrix

In a Hadoop cluster, data is stored distributed across multiple machines for parallel processing and replicated multiple times for redundancy

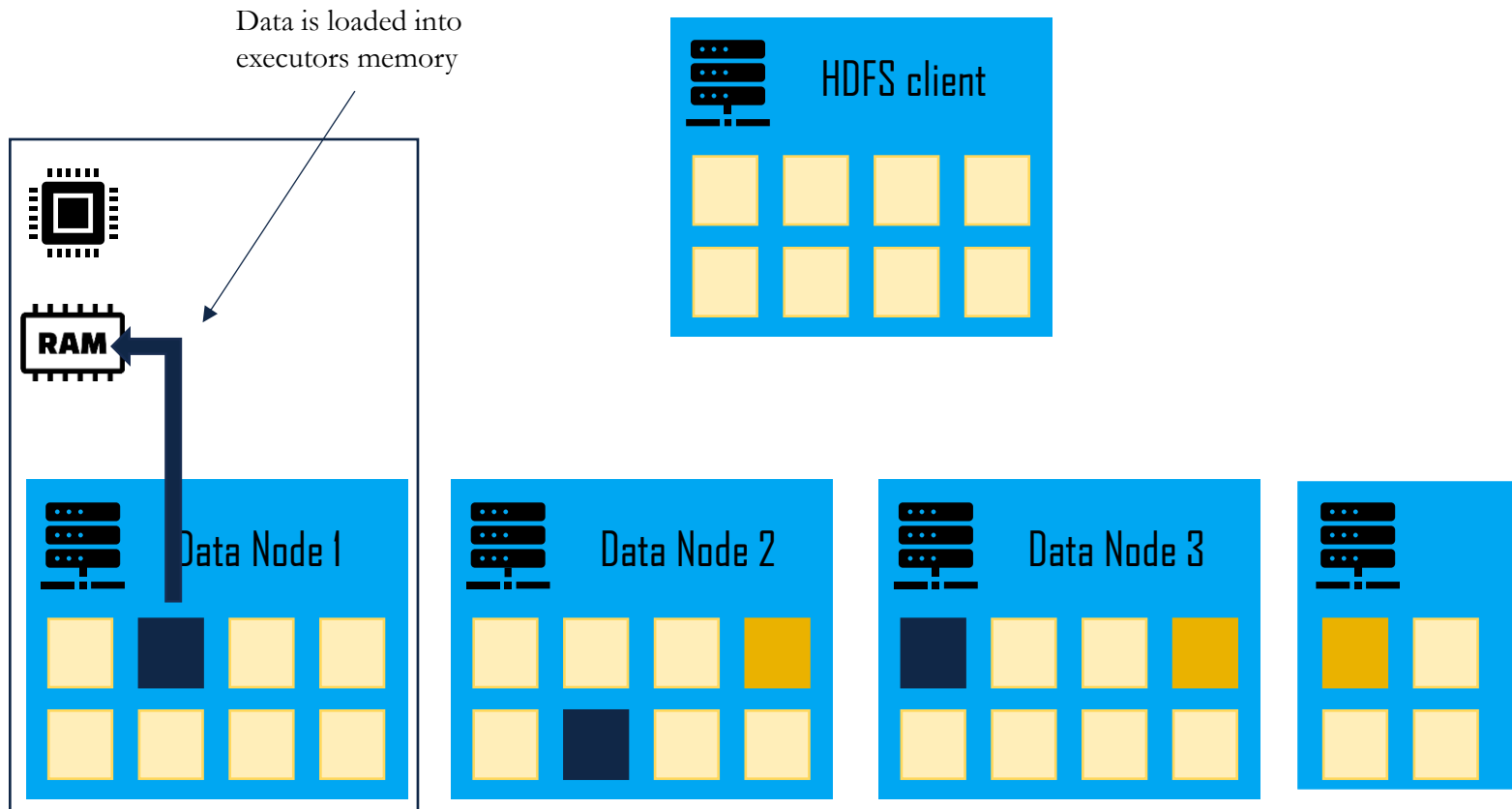


Summing up many numbers

Consider we have 75GB dataset of numbers which we want to calculate the sum of. The data is scattered across the cluster, and typically replicated, such that if any single data nodes goes down, the data is never lost. The HDFS client manages the meta data (which files is where).



Local processing of data saves network travel

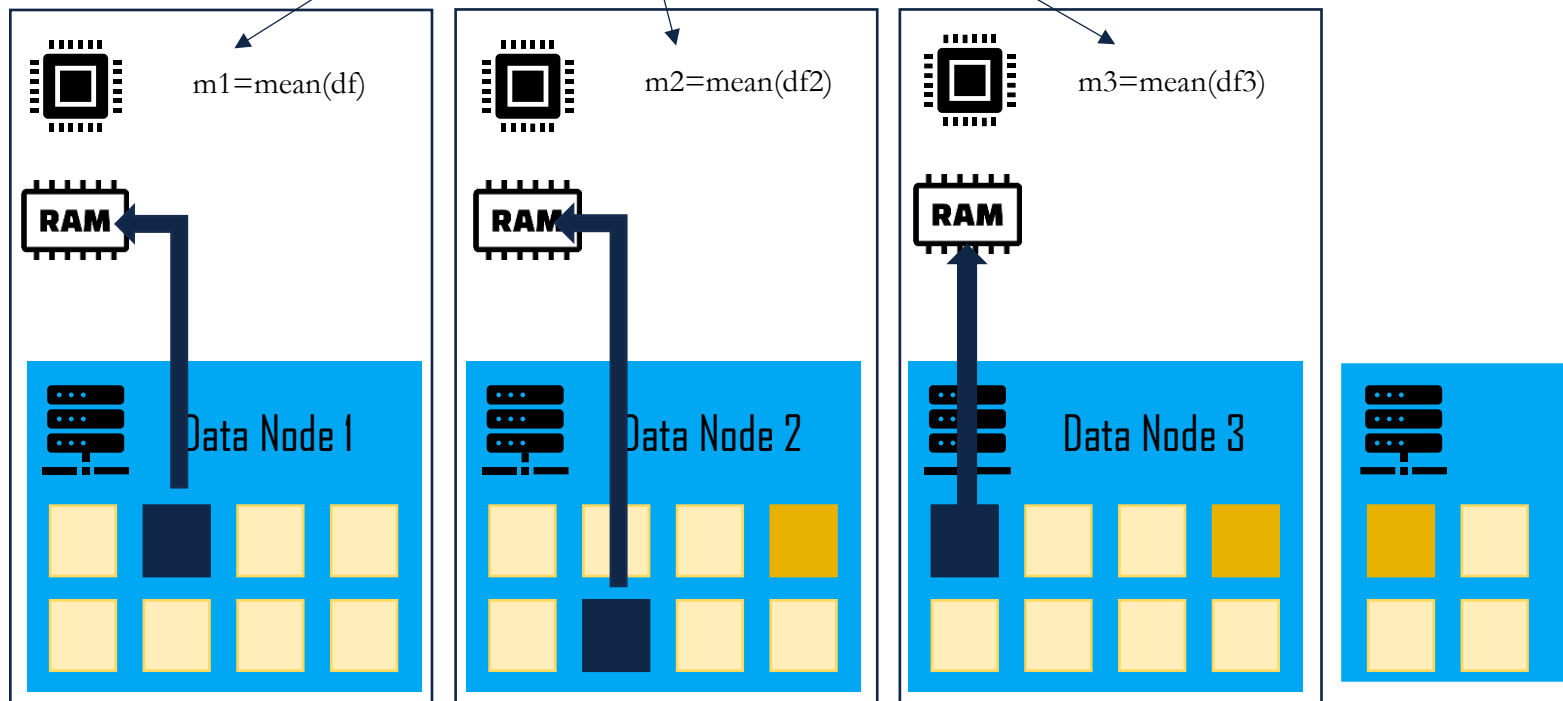


Saving network travel

In a Hadoop cluster, data can be loaded directly into the executors memory to performance computations without having to move data through the network (at least in the initial non-exchanged) data phase.

Each executor calculates the individual sum

Each executor performs part of the calculations, then we can sum up:
 $m = (m1 + m2 + m3) / 3$



Parallel calculation

Each nodes can calculate the sum of the partial data. Finally, the partial sums can be summed up.

Problems solved:

- › **I/O:** Take slittle time to read since is on the machine's hard drive
- › **Space complexity:** Small chunks fit into executors memory
- › **Time complexity:** Can be cut in three (or however many nodes you have)

Dean and Ghemawat introduced a scalable programming model for processing large datasets, leveraging a simple abstraction for parallelism across distributed systems.

MapReduce: Simplified Data Processing on Large Clusters

Jeffrey Dean and Sanjay Ghemawat

jeff@google.com, sanjay@google.com

Google, Inc.

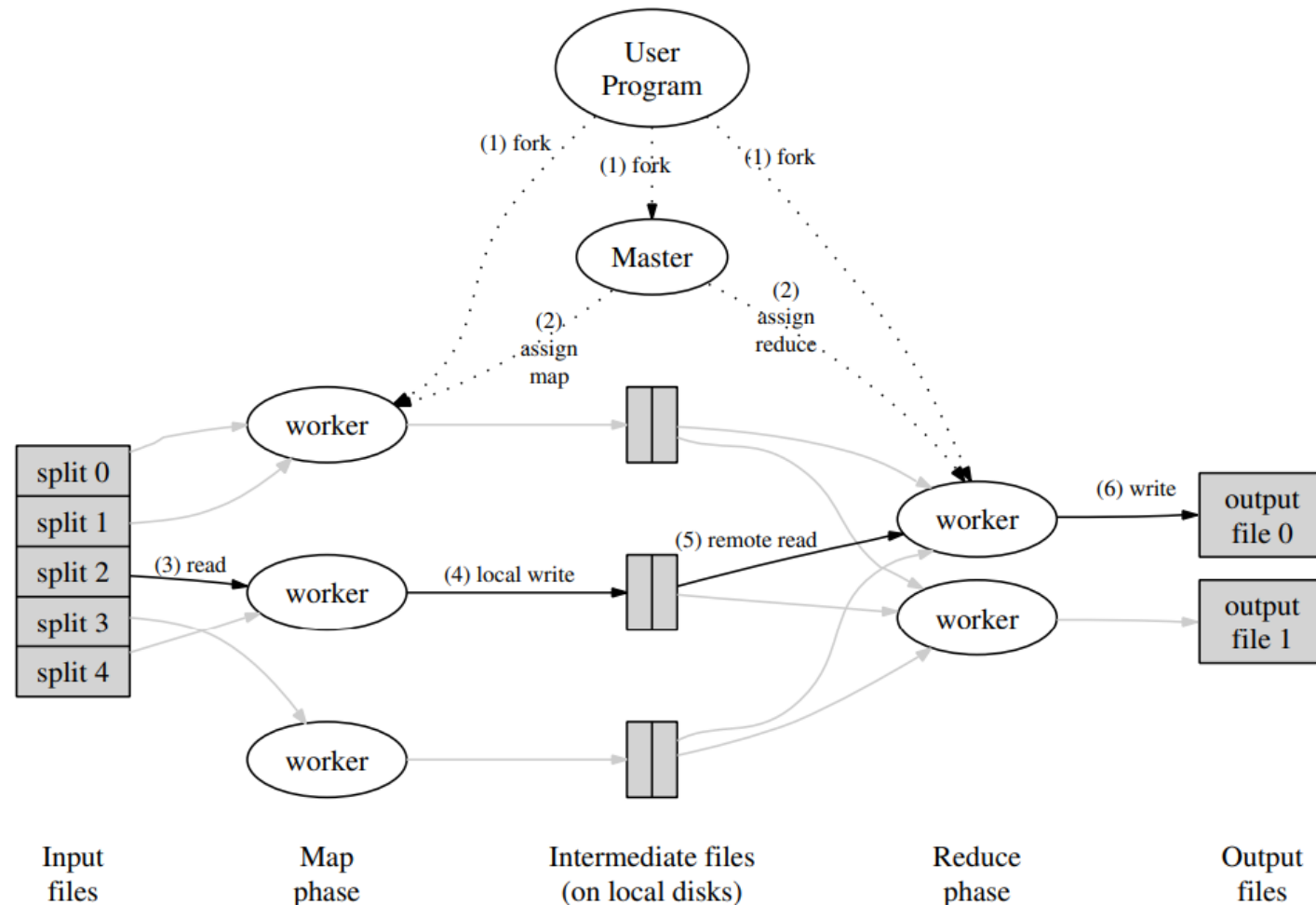
Abstract

MapReduce is a programming model and an associated implementation for processing and generating large data sets. Users specify a *map* function that processes a key/value pair to generate a set of intermediate key/value pairs, and a *reduce* function that merges all intermediate values associated with the same intermediate key. Many real world tasks are expressible in this model, as shown in the paper.

given day, etc. Most such computations are conceptually straightforward. However, the input data is usually large and the computations have to be distributed across hundreds or thousands of machines in order to finish in a reasonable amount of time. The issues of how to parallelize the computation, distribute the data, and handle failures conspire to obscure the original simple computation with large amounts of complex code to deal with these issues.

As a reaction to this complexity, we designed a new

MapReduce executes distributed data processing tasks by dividing them into independent map and reduce phases, where data is transformed and aggregated across a cluster.



Building on top of MapReduce, Zaharia et.al. (2012) introduce Spark to leverage in-memory computing to speed up data handling

Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing

Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma,
Murphy McCauley, Michael J. Franklin, Scott Shenker, Ion Stoica
University of California, Berkeley

Abstract

We present Resilient Distributed Datasets (RDDs), a distributed memory abstraction that lets programmers perform in-memory computations on large clusters in a fault-tolerant manner. RDDs are motivated by two types of applications that current computing frameworks handle inefficiently: iterative algorithms and interactive data mining tools. In both cases, keeping data in memory can improve performance by an order of magnitude.

tion, which can dominate application execution times.

Recognizing this problem, researchers have developed specialized frameworks for some applications that require data reuse. For example, Pregel [22] is a system for iterative graph computations that keeps intermediate data in memory, while HaLoop [7] offers an iterative MapReduce interface. However, these frameworks only support specific computation patterns (*e.g.*, looping a series of MapReduce steps), and perform data sharing implicitly

Apache Spark is a unified analytics engine for large-scale data processing which builds on the MapReduce Framework but performs calculations in memory



Spark solves MapReduce Short Comings

Hadoop MapReduce: Scalable + fault-tolerant, but complex, high latency, poor support for iterative and interactive queries.

Spark: In-memory computing and optimized execution, high-level APIs and a unified framework. Supports iterative processing and interactive queries:

- › Core Abstractions: RDD, DataFrame, Dataset
- › Key Libraries: Spark SQL, MLlib, GraphX, Spark Streaming
- › Performance: Catalyst Optimizer, Tungsten Execution Engine
- › Language Support: Scala, Java, Python, R

Spark History

Origins and Development (2009-2014): Apache Spark was initiated in 2009 at UC Berkeley's AMPLab as a research project to improve Hadoop's performance by enabling in-memory data processing. It was open-sourced in 2010 and became an Apache Top-Level Project in 2014.

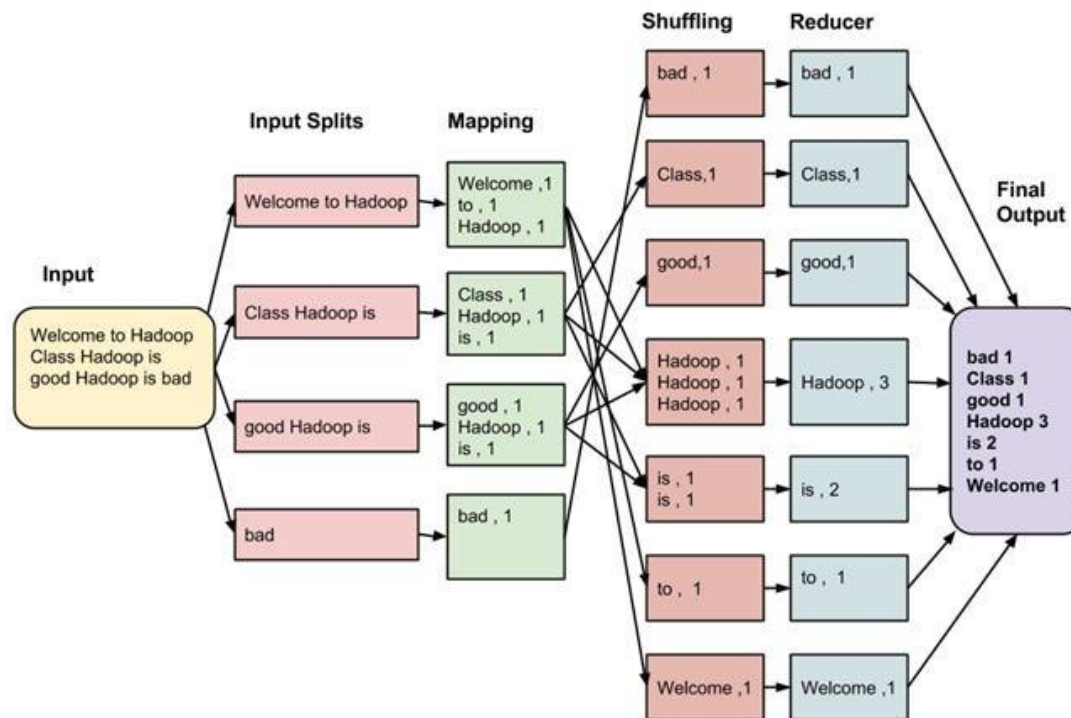
Key Milestones: Spark introduced the Resilient Distributed Dataset (RDD) abstraction, enabling fault-tolerant and distributed computation. It quickly gained popularity for its ability to handle iterative algorithms, stream processing, and complex analytics.

Adoption and Ecosystem Growth: Spark has grown into a comprehensive ecosystem with modules like Spark SQL, MLlib, and GraphX, and is widely adopted in industries for big data processing, machine learning, and streaming analytics.

Apache Spark is a unified analytics engine for large-scale data processing which builds on the MapReduce Framework but performs calculations in memory



Map Reduce Algorithm is implemented in memory in Spark



Hadoop MapReduce → Spark

Hadoop MapReduce: Scalable +fault-tolerant, but complex, high latency, poor support for iterative and interactive queries.

Spark: In-memory computing and optimized execution, high-level APIs and a unified framework. Supports iterative processing and interactive queries:

- Core Abstractions: RDD, DataFrame, Dataset
- Key Libraries: Spark SQL, MLlib, GraphX, Spark Streaming
- Performance: Catalyst Optimizer, Tungsten Execution Engine
- Language Support: Scala, Java, Python, R

Don't blindly use spark. Use it for large scale batch processing. Otherwise, use something else!



Advantages / use cases

- › Main use case is batch processing of 100GB+ datasets
- › Much faster than earlier iterations of distributed big data handling (hive, Java MR) since it uses memory instead of intermediate writes
- › Spark is storage agnostic (you can store on S3, Azure Blob, your own HDFS,...). This is great to prevent vendor lock-in!
- › Huge developer community/online resources + chatGPT is a great pair programming partner

Disadvantages / anti patterns

- › Slow for small data (if it fits in one python process, use that instead, much, much simpler!)
- › Don't use it for live dashboards etc, it is meant for (large) batch processing
- › "Mindset lock-in": If no one else in your company knows/uses spark, don't push it.
- › "Vendor lock-in": If your company uses BigQuery etc., don't try to migrate to spark unless there is a good reason

RDDs, DataFrames and Datasets are the core data structures available in Spark

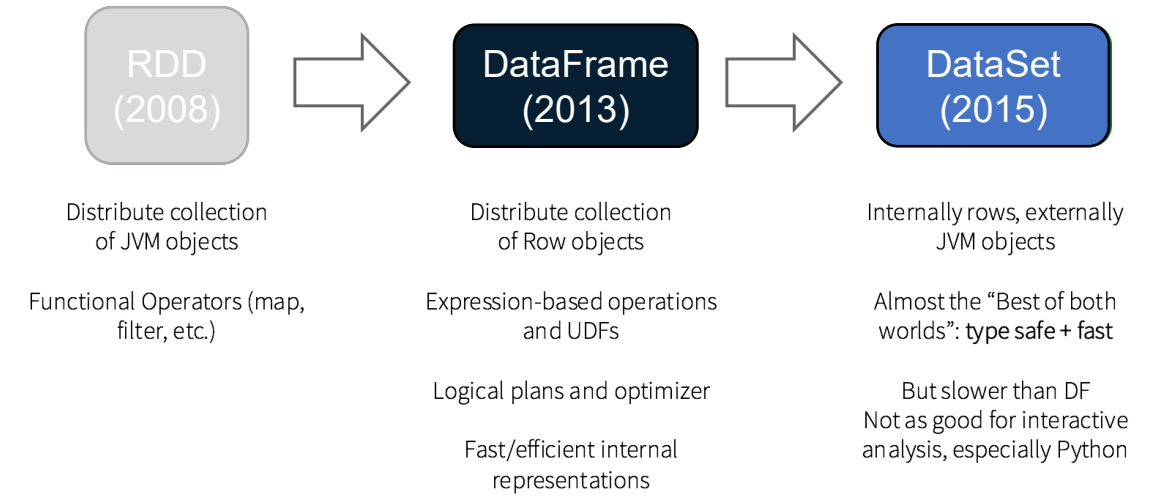
Main DS include RDDs, DataFrames and Datasets

Resilient Distributed Dataset (RDD) is the fundamental data structure of Spark. It is an immutable distributed collection of objects that can be processed in parallel. Key operations: Transformations (e.g., map, filter, reduceByKey) and Actions (e.g., collect, count).

DataFrame is a distributed collection of data organized into named columns, conceptually equivalent to a table in a relational database. They provide a higher-level abstraction than RDDs, offering optimizations and ease of use.

Dataset is a strongly-typed (compile type safe), distributed collection of data. It is an extension of DataFrame API, providing the benefits of RDDs (strong typing, lambda functions) with the benefits of Spark SQL's optimized execution engine. Not available in Python.

History of spark APIs



```
case class Person(name: String, age: Int)
val ds: Dataset[Person] = Seq(
  Person("Alice", 25),
  Person("Bob", 30),
  Person("Cathy", 23)
).toDS()
val adults: Dataset[Person] = ds.filter(person => person.age >= 18)
```

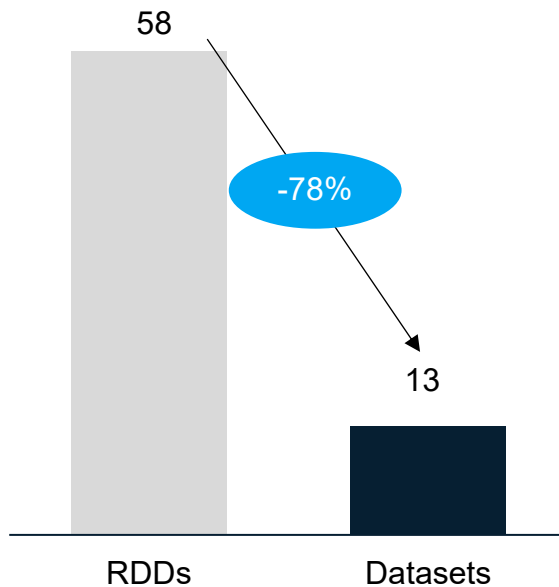
Mix of class and table allows for typesafe "table" operations

Spark SQL and its DataFrames and Datasets interfaces are the future of Spark performance

As they provide more efficient storage options, advanced optimizer, and direct operations on serialized data

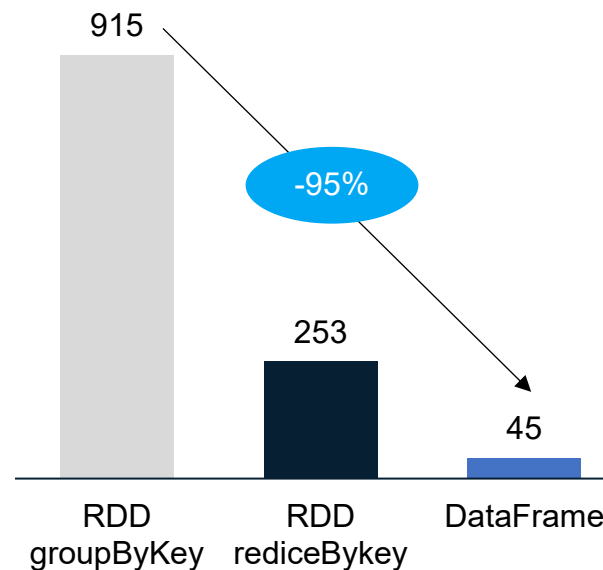
Datasets & DataFrames outperform in terms of memory usage...

Memory Usage when caching
Data Size (GB)



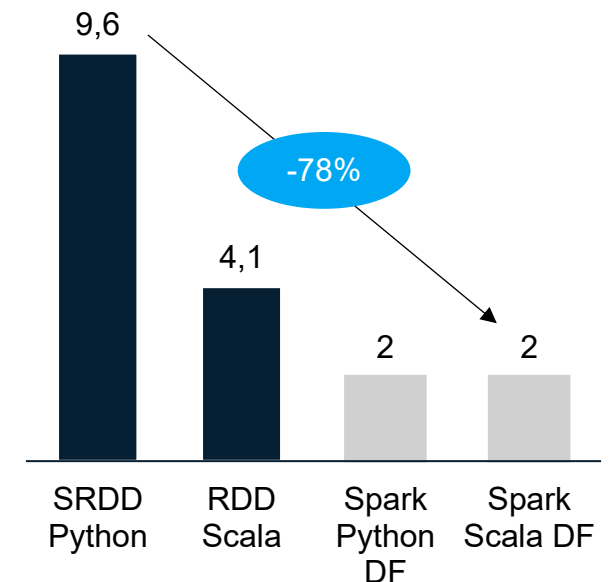
... as well as in terms of execution time ...

Execution Time
seconds



... irrespective of whether the code is executed in python or scala

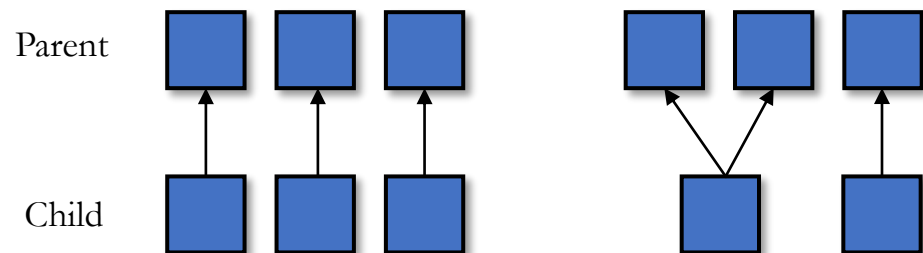
Execution time
seconds



Wide transformations trigger a data shuffle triggering a new stage in the job, while narrow transformations do not

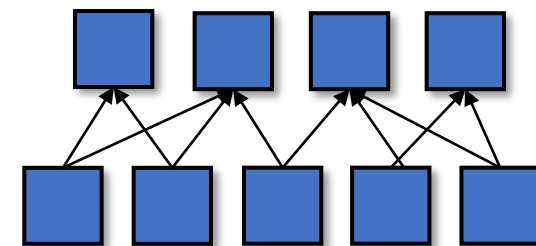
Narrow transformations

Narrow transformations are those in which each partition of the parent RDD is used by at most one partition of the child RDD. This means that the data required to compute the records in a single partition of the child RDD exists in a single partition of the parent RDD. Examples include `map()`, `filter()`, `flatMap()`, `MapPartitions()`



Wide transformations

Wide transformations involve shuffling data across multiple nodes and partitions. In these transformations, the data required to compute the records in a single partition of the child RDD may be located in many partitions of the parent RDD. Examples include: `GroupByKey()`, `ReduceByKey()`, `Join()`, `Distinct()`



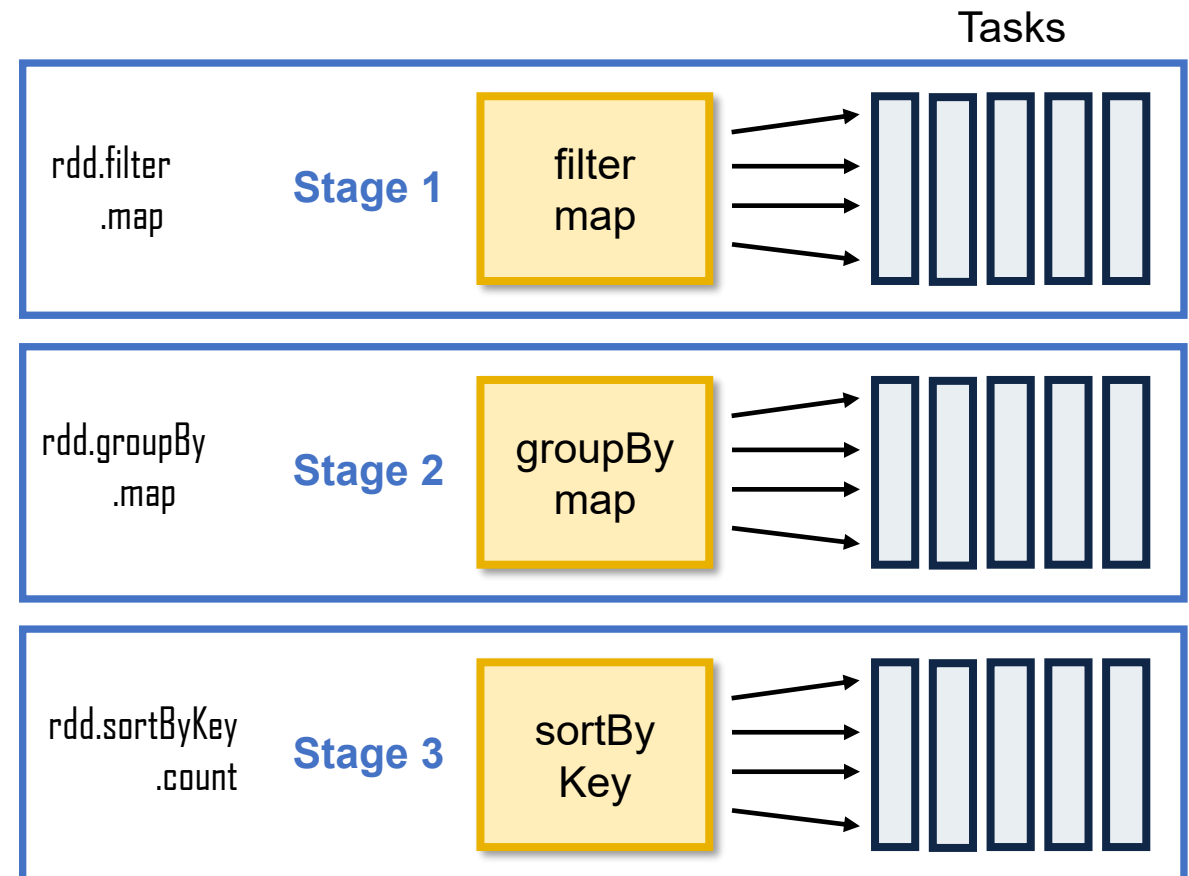
Spark jobs aim to parallelize computations as much as possible. Operations that require a data shuffle trigger a new stage in the job

Jobs are organized into stages which consists of tasks

This Spark program takes an RDD of Double values and performs several transformations. In the first stage, it filters values less than 1000 and maps each value to a tuple containing the value and 1. In the second stage, it groups the tuples by their first element, sums the grouped values, and pairs the sum with the original value. In the final stage, it sorts the results by the sum (key) and counts the number of resulting elements, returning this count as a Long.

```
def simpleSparkProgram(rdd : RDD[Double]): Long = {  
  //stage1  
  rdd.filter(_ < 1000.0)  
    .map(x => (x, 1))  
  //stage2  
    .groupByKey()  
    .map{ case (value, groups) => (groups.sum, value)}  
  //stage 3  
    .sortByKey()  
    .count()  
}
```

Stage diagram: Anatomy of a spark job



Lazy evaluation in Apache Spark optimizes data transformations by deferring execution until an action is triggered, reducing redundant computations and improving performance.

What is Lazy Evaluation?

- › Spark does not execute transformations (e.g., `map()`, `filter()`) immediately but records them as a logical Directed Acyclic Graph (DAG).
- › Actual computation begins only when an action like `count()`, `collect()`, or `save()` is invoked, signaling Spark to process the DAG.
- › This approach avoids unnecessary computations for intermediate results that might not be needed in the final output.

Benefits of Lazy Evaluation

- › **Optimized Execution:** Spark evaluates the entire DAG and optimizes the execution plan by combining stages and minimizing data shuffling, resulting in faster processing.
- › **Fault Tolerance:** If a failure occurs, Spark can recompute lost partitions using the logical DAG without restarting the entire job, ensuring data consistency and resilience.
- › **Resource Efficiency:** By deferring execution, Spark avoids using memory and CPU on intermediate transformations, allowing for more efficient use of cluster resources.

Slide on the most important settings

Element	Setting	Explanation
Driver	<code>spark.driver.memory</code>	Memory of the driver. Unless you need to collect data to the driver or run rest calls from the driver no need to tweak this.
	<code>spark.driver.memoryOverheadFactor</code>	Memory required for the overhead planning. Unless you have a very complex job, the default 10% will be fine
Executors	<code>spark.executor.memory</code>	Memory of executors used for data processing
	<code>spark.executor.cores</code>	How many tasks can happen on each machine in parallel.
	<code>spark.executor.memoryOverheadFactor</code>	Again, usually 10% is fine unless you have many, many UDFs.

Partitioning (when processing data) is the key bottleneck in most spark apps

Partitions must be small enough to fit memory

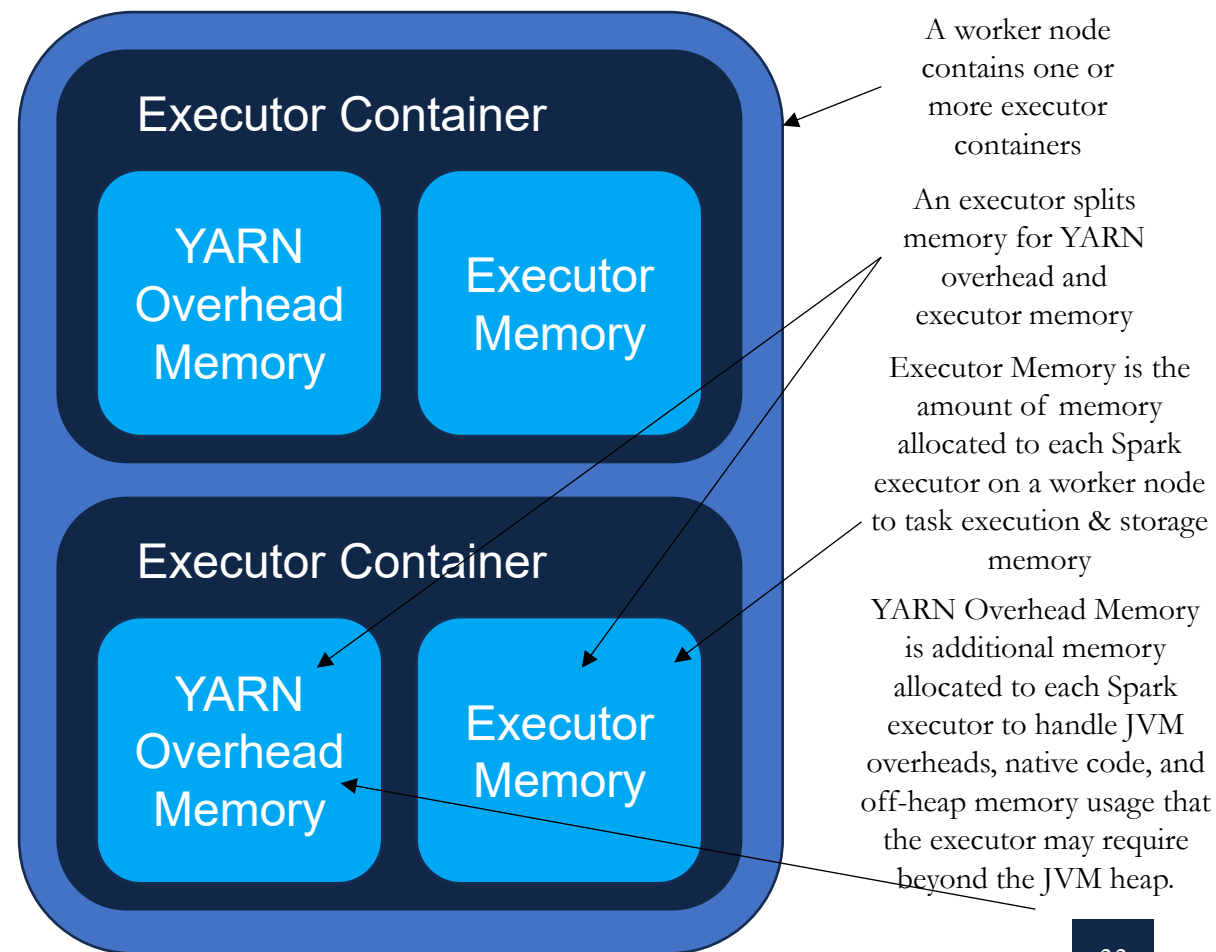
Assume: Dataset size (500 GB), desired partition size (128 MB) and 200 available cores in the cluster:

$$\text{\#Partitions} = \frac{500 \times 1,024 \text{ MB}}{128 \text{ MB}} = 4,000 \text{ partitions}$$

Imagine a Spark job that processes a dataset with a very large partition (e.g., 5 GB) on an executor with 4 GB of memory:

- › When the task loads the partition, it tries to allocate memory for the entire 5 GB partition, which exceeds the executor's 4 GB limit.
- › The JVM attempts to run garbage collection to free up memory, but it's insufficient because the partition itself is too large.
- › The task fails with an `OutOfMemoryError`, causing the executor to retry the task.

Worker Nodes hosts executors which execute tasks



Partitioning (when persisting data) properly can speed up user queries

Partitioning data properly is key

- › Partitioning a table is easy in Hive:

```
CREATE TABLE db.table1_hive (  
  order_id BIGINT,  
  customer_id BIGINT,  
  order_amount DECIMAL(10, 2)  
)  
  
PARTITIONED BY (order_date BIGINT)  
  
STORED AS PARQUET;
```

(1) Hive creates a partitioned table (you must have or derive this column)

```
INSERT INTO db.table1_hive PARTITION (order_date)  
  SELECT 123, 456, 36.17, 20210117 as order_date;
```

```
INSERT INTO db.table1_hive PARTITION (order_date)  
  SELECT 987, 321, 40.00, 20210118 as order_date;
```

```
INSERT INTO db.table1_hive PARTITION (order_date)  
  SELECT 9487, 3261, 41.00, 20210119 as order_date;
```

```
EXPLAIN EXTENDED SELECT * FROM db.table1_hive  
  WHERE order_date = 20210118 ;
```

(2) Hive will create one file in each of the three order date "sub folders"

(3) Hive knows what to read only the folder contents where the date matches

(4) This saves reading 66% (in this case) of all files!

Using explain to learn about the query plan

- › Hive (and other engines) will "push down the filter" to the partition level and only read the required files

4	STAGE DEPENDENCIES:
5	Stage-0 is a root stage
6	
7	STAGE PLANS:
8	Stage: Stage-0
9	Fetch Operator
10	limit: -1
11	Partition Description:
12	Partition
13	input format: org.apache.hadoop.hive.ql.io.parquet.MapredParquetInputFormat
14	output format: org.apache.hadoop.hive.ql.io.parquet.MapredParquetOutputFormat
15	partition values:
16	order_date 20210118

Joining data properly make or break any data pipeline

Apache Spark offers multiple join strategies, such as Broadcast Hash Join, Shuffle Hash Join, Shuffle Sort Merge Join, Broadcast Nested Loop Join, and Cartesian Product Join, to handle varying data sizes and structures efficiently. The choice of join strategy is influenced by factors like dataset size, join key distribution, and whether join keys are specified. To optimize joins, understanding data characteristics, enabling auto-broadcast for small datasets, and partitioning data effectively can significantly improve performance.

- ▶ **Broadcast Hash Join:** A highly efficient join where a small dataset is broadcast to all executors, allowing local hash table lookups to join with a larger dataset.
- ▶ **Shuffle Hash Join:** A join strategy that involves shuffling data from both sides to create hash tables, enabling joining of large datasets with distributed processing.
- ▶ **Shuffle Sort Merge Join:** A join type that sorts and partitions datasets on join keys, merging them efficiently, best suited for large, evenly distributed datasets.
- ▶ **Broadcast Nested Loop Join:** A join strategy that broadcasts a small dataset to all executors and iterates over all combinations to find matches, used when no join keys are specified.
- ▶ **Cartesian Product Join** (Shuffle-and-Replicate Nested Loop Join): A join that generates all possible row combinations between datasets, typically used for cross joins without specific conditions.
- ▶ **Bucket Join:** In a bucket join, data is “presorted” in a bucket to avoid shuffling data around the cluster by prefixing the stored data on s3 or other storage technologies

Consider the following example tables

We do a simple inner join and match rows

Id1	...
10	...
20	
20	
30	
40	
10	
40	
20	
20	
20	
20	
20	
50	



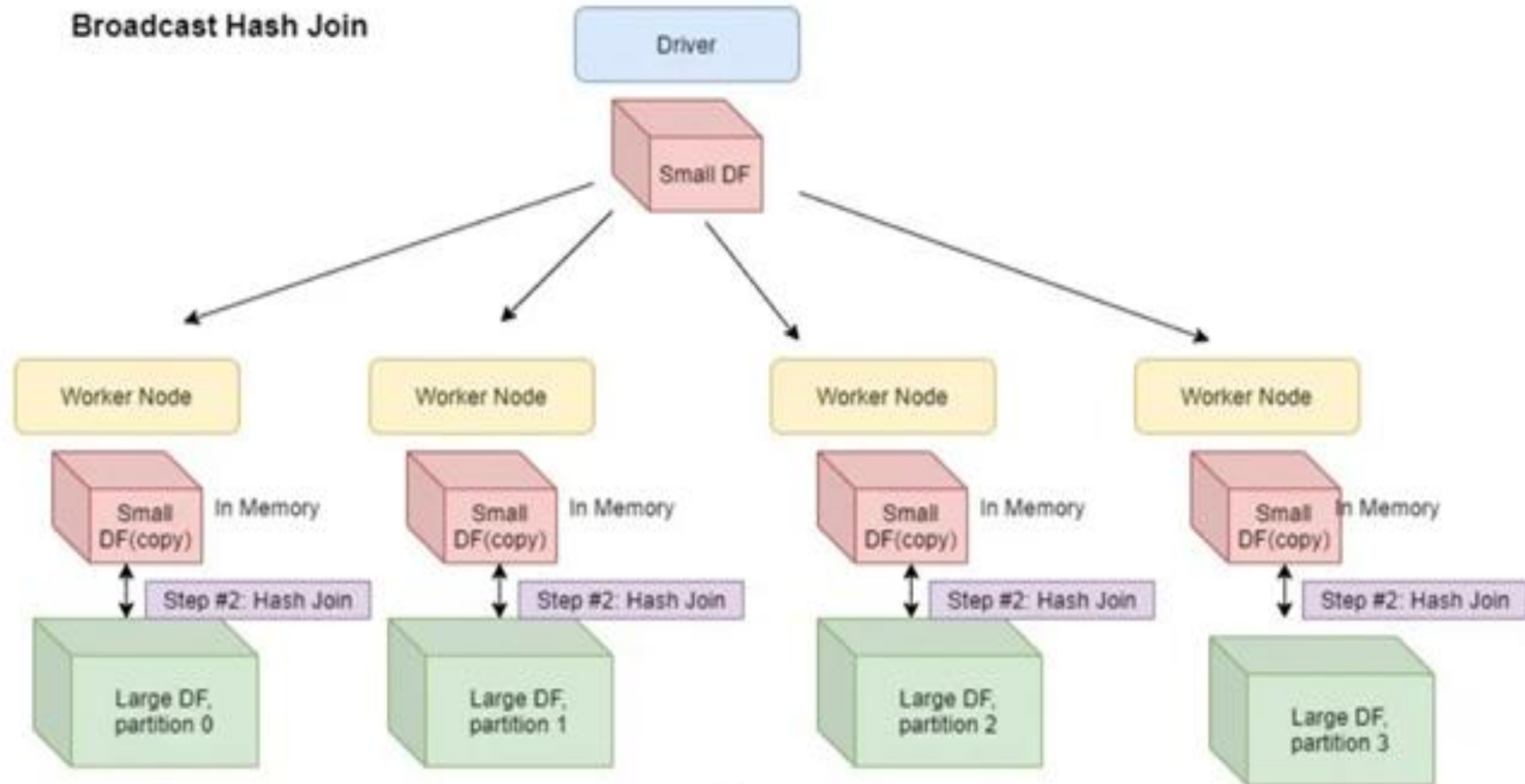
Id2	...
30	...
20	
40	
50	



Id1=Id2	...
50	
30	
20	
20	
20	
20	
20	
20	
40	
40	

In a broad join the drivers sends the a small dataframe to each node in the cluster

This avoids shuffling data between the worker nodes, but is only feasible for small datasets (10MB by default)



In a broad join the drivers sends the a small dataframe to each node in the cluster

Broadcast join can “hinted”

```
from pyspark.sql.functions import broadcast
joined_df = df1.join(broadcast(df2), df1.id1 == df2.id2)
joined_df.display()
```

▶ (3) Spark Jobs

▶ joined_df: pyspark.sql.dataframe.DataFrame = [id1: long, id2: long]

Table ▾ +

	id1	id2
4	40	40
5	40	40
6	20	20
7	20	20
8	20	20
9	20	20
10	50	50

↓ 10 rows | 0.49 seconds runtime

Source: <https://medium.com/@amarkrgupta96/join-strategies-in-apache-spark-a-hands-on-approach-d0696fc0a6c9>

In query plan we can see the broad cast join

```
execution_plan = joined_df._jdf.queryExecution().executedPlan()

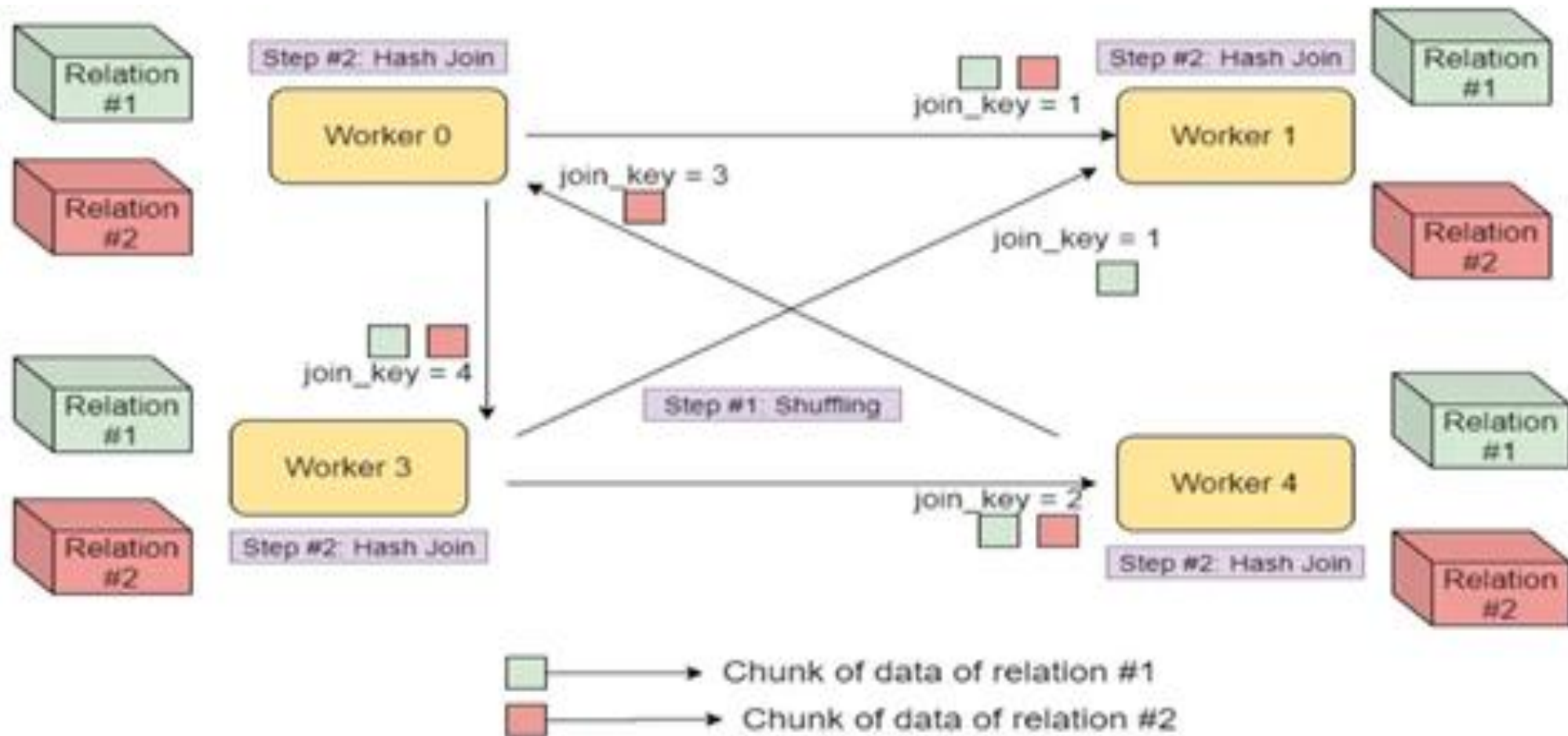
# Show the human-readable execution plan
print(execution_plan.toString())
```

_jdf is used to convert pyspark df to a java df in order to access the functions specific to java

```
AdaptiveSparkPlan isFinalPlan=false
+- BroadcastHashJoin [id1#12L], [id2#17L], Inner, BuildRight, false
   :- Filter isNotNull(id1#12L)
   : +- Scan ExistingRDD[id1#12L]
   +- Exchange SinglePartition, EXECUTOR_BROADCAST, [plan_id=701]
      +- Filter isNotNull(id2#17L)
         +- Scan ExistingRDD[id2#17L]
```


A shuffle hash join is a join strategy used in distributed systems like Apache Spark when both datasets are too large to fit into memory and require repartitioning to align the data.

...



Step 0: original tables

Two tables to be joined on customer_id. Orders is the larger (probe) side, Customers is the smaller (build) side.

ORDERS (PROBE SIDE)			CUSTOMERS (BUILD SIDE)	
order_id	cust_id	amount	cust_id	name
1	101	50	101	Alice
2	102	30	102	Bob
3	101	70	103	Carol
4	103	20	104	Diana
5	102	40		
6	104	60		

Step 1: shuffle both sides by join key

Both tables are repartitioned by $\text{hash}(\text{customer_id}) \% 2$. Odd IDs $\rightarrow$ partition 0, even IDs $\rightarrow$ partition 1. This is a full network shuffle — every executor sends rows to the right partition.

ORDERS AFTER SHUFFLE

Partition 0 (odd IDs)

order_id	cust_id	amt
1	101	50
3	101	70
4	103	20

Partition 1 (even IDs)

order_id	cust_id	amt
2	102	30
5	102	40
6	104	60

CUSTOMERS AFTER SHUFFLE

Partition 0 (odd IDs)

cust_id	name
101	Alice
103	Carol

Partition 1 (even IDs)

cust_id	name
102	Bob
104	Diana

Step 2: build hash table from small side

On each partition independently, the Customers (build) side is loaded into an in-memory hash table keyed by `customer_id`. This hash table must fit in executor memory — if it doesn't, you OOM

HASH TABLE — PARTITION 0

101 → {Alice}

103 → {Carol}

HASH TABLE — PARTITION 1

102 → {Bob}

104 → {Diana}

Step 3: probe — scan large side against hash table

Each Orders row is looked up in its partition's hash table. $O(1)$ per lookup. If the key exists, emit the joined row. If not, discard (inner join) or emit null (outer join).

PARTITION 0 PROBING

probe row	lookup?	result
{1, 101, 50}	HIT → Alice	{1, 101, 50, Alice}
{3, 101, 70}	HIT → Alice	{3, 101, 70, Alice}
{4, 103, 20}	HIT → Carol	{4, 103, 20, Carol}

PARTITION 1 PROBING

probe row	lookup?	result
{2, 102, 30}	HIT → Bob	{2, 102, 30, Bob}
{5, 102, 40}	HIT → Bob	{5, 102, 40, Bob}
{6, 104, 60}	HIT → Diana	{6, 104, 60, Diana}

Step 4: final joined output

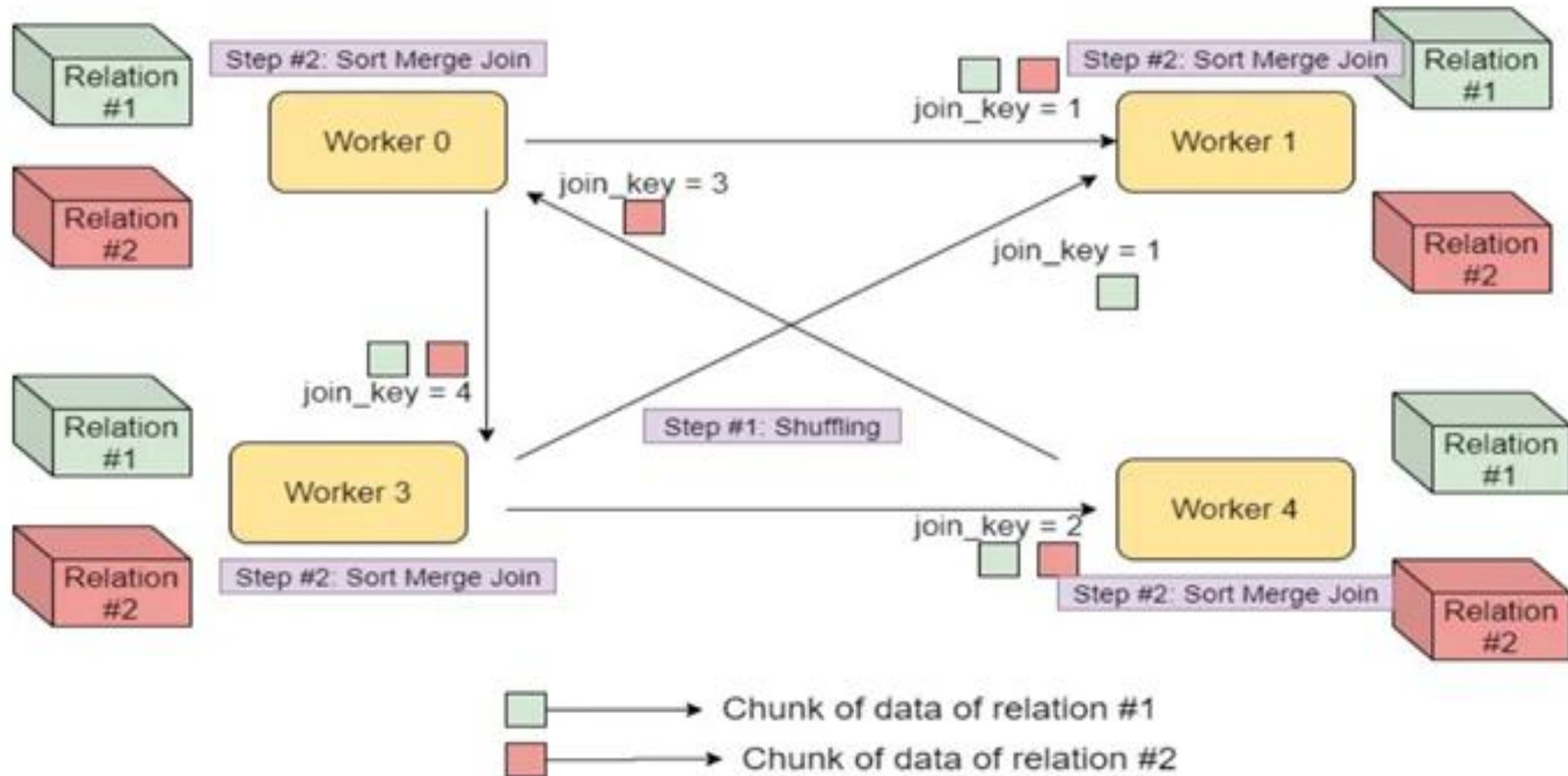
Union of all partition results. Total cost: one full shuffle of both tables + $O(n)$ hash build + $O(m)$ probe. No sorting anywhere.

JOINED RESULT

order_id	cust_id	amount	name
1	101	50	Alice
3	101	70	Alice
4	103	20	Carol
2	102	30	Bob
5	102	40	Bob
6	104	60	Diana

Sort-Merge Join is a distributed join strategy in Spark used for large datasets. It requires both datasets to be sorted on the join key and then merges the sorted datasets.

...



Step 0: original tables

Two tables to be joined on `customer_id`. Orders is the larger (probe) side, Customers is the smaller (build) side. We have `n` rows in the probe side and `m` rows in the build side.

ORDERS (PROBE SIDE)			CUSTOMERS (BUILD SIDE)	
order_id	cust_id	amount	cust_id	name
1	101	50	101	Alice
2	102	30	102	Bob
3	101	70	103	Carol
4	103	20	104	Diana
5	102	40		
6	104	60		

Step 1: shuffle (identical to hash join)

Same shuffle as before — both sides repartitioned by $\text{hash}(\text{customer_id}) \% 2$. The network cost is identical. The difference starts after the shuffle.

ORDERS AFTER SHUFFLE

Partition 0 (odd IDs)

order_id	cust_id	amt
1	101	50
3	101	70
4	103	20

Partition 1 (even IDs)

order_id	cust_id	amt
2	102	30
5	102	40
6	104	60

CUSTOMERS AFTER SHUFFLE

Partition 0 (odd IDs)

cust_id	name
101	Alice
103	Carol

Partition 1 (even IDs)

cust_id	name
102	Bob
104	Diana

Step 2: sort both sides within each partition

Each partition is sorted by `customer_id` independently. This is the extra cost sort merge pays — $O(n \log n)$ per partition. Uses external sort, so it can spill to disk if data exceeds memory. This is why sort merge handles arbitrarily large datasets.

ORDERS — PARTITION 0 SORTED

order_id	cust_id ↑	amt
1	101	50
3	101	70
4	103	20

ORDERS — PARTITION 1 SORTED

order_id	cust_id ↑	amt
2	102	30
5	102	40
6	104	60

CUSTOMERS — PARTITION 0 SORTED

cust_id ↑	name
101	Alice
103	Carol

CUSTOMERS — PARTITION 1 SORTED

cust_id ↑	name
102	Bob
104	Diana

Step 3: merge — walk two cursors (partition 0)

Two pointers advance through each sorted stream. Because both are sorted, you never backtrack. When keys match, emit all combinations. When they don't, advance the smaller key. Linear scan — $O(n+m)$.

L→{1,101,50} vs R→{101,Alice} — $101 == 101$ — **MATCH**. Emit {1,101,50,Alice}. Check next L row too: {3,101,70} same key — emit {3,101,70,Alice}. Advance L past all 101s.

L→{4,103,20} vs R→{101,Alice} — $103 > 101$ — advance R.

L→{4,103,20} vs R→{103,Carol} — $103 == 103$ — **MATCH**. Emit {4,103,20,Carol}. Advance L.

L exhausted. Partition 0 done. Output: 3 rows.

PARTITION 0 OUTPUT

order_id	cust_id	amt	name
1	101	50	Alice
3	101	70	Alice
4	103	20	Carol

Step 3b: merge — walk two cursors (partition 1)

Same algorithm, applied independently on partition 1.

L→{2,102,30} vs R→{102,Bob} — $102 == 102$ — **MATCH**. Emit {2,102,30,Bob}. Next L: {5,102,40} same key — emit {5,102,40,Bob}. Advance L past all 102s.

L→{6,104,60} vs R→{102,Bob} — $104 > 102$ — advance R.

L→{6,104,60} vs R→{104,Diana} — $104 == 104$ — **MATCH**. Emit {6,104,60,Diana}. Advance L.

L exhausted. Partition 1 done. Output: 3 rows.

PARTITION 1 OUTPUT

order_id	cust_id	amt	name
2	102	30	Bob
5	102	40	Bob
6	104	60	Diana

Step 4: final result (identical output)

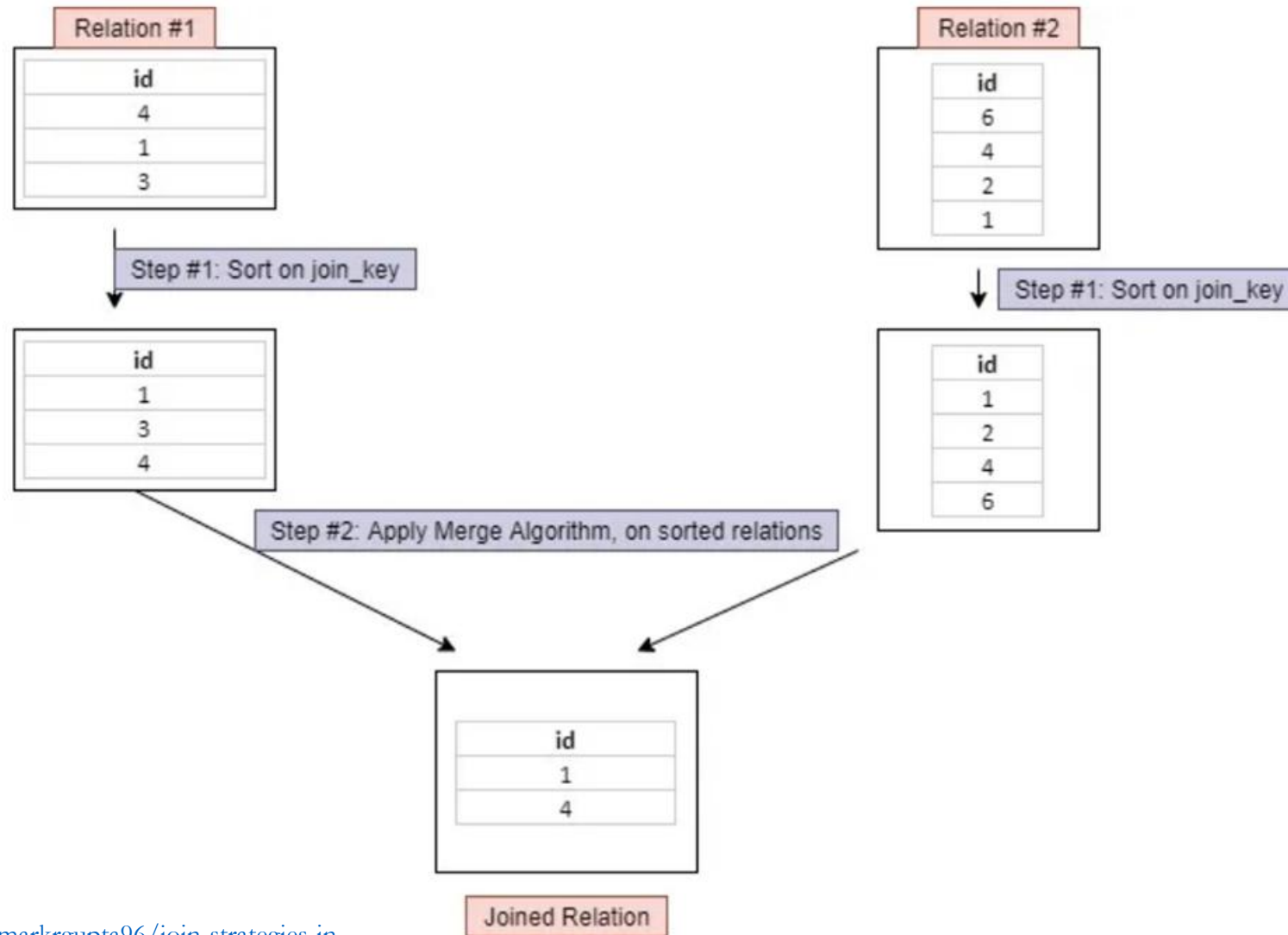
Same 6 rows as shuffle hash join. The result is identical — only the internal mechanics differ

JOINED RESULT

order_id	cust_id	amount	name
1	101	50	Alice
3	101	70	Alice
4	103	20	Carol
2	102	30	Bob
5	102	40	Bob
6	104	60	Diana

Sort-Merge Join is a distributed join strategy in Spark used for large datasets. It requires both datasets to be sorted on the join key and then merges the sorted datasets.

...



Sampling, bucketing and retention periods can help to manage massive amounts of data

Sometimes it does depend on the context on what you want to model. Keep the user in mind. The line is blurry!

Dimensions

- › **Sampling**: for some ML/AI applications we can randomly sample big tables and get representative data for model training. Not valid for all cases though
- › **Retention periods**: Oftentimes we don't need all historic facts. Delete old ones or move them to a history table.
- › **Bucketing**: is a strategy where big fact tables and corresponding dimension tables are portioned/bucketed on the same key. This can drastically improve performance as it avoid costly shuffles.

Example using bucketing

```
s3://my-data/users/  
├── bucket_0/  
│   ├── part-00000.parquet  
│   └── part-00001.parquet  
├── bucket_1/  
│   └── part-00000.parquet  
├── bucket_2/  
│   └── part-00000.parquet  
└── bucket_3/  
    └── part-00000.parquet
```

```
s3://my-data/transactions/  
├── bucket_0/  
│   └── part-00000.parquet  
├── bucket_1/  
│   ├── part-00000.parquet  
│   └── part-00001.parquet
```

In a bucket join, data of the same bucket is loaded by the same worker, eliminating the need for costly data shuffle. Of course, this only works if tables are partitioned properly.

Most of the functionality is provided by Spark/Hive/Trino under the hood without the need for user intervention. But partition your tables properly!

Data is processed by the same worker: No shuffle!

- › Each bucket is processed by one worker:
 - › Bucket 0 → Worker 1
 - › Bucket 1 → Worker 2
 - › Bucket 2 → Worker 3
 - › Bucket 3 → Worker 4
- › All data of one bucket is on the same worker:
 - › `users/bucket_0` is joined with `transactions/bucket_0`.
 - › `users/bucket_1` is joined with `transactions/bucket_1`.

```
spark.sql("""
    SELECT u.user_id, u.name, t.amount
    FROM users u
    JOIN transactions t ON u.user_id = t.user_id
    """)
```

Hive/Spark/... Will automatically perform bucket joins

- › **Logical Plan:** Spark analyzes the query and detects that users and transactions are bucketed by `user_id` with the same bucket count. It decides to **use a bucket join** (a.k.a. bucketed sort-merge join).
- › **Physical Plan:** Spark assigns **a task for each bucket** (e.g., one task per bucket pair: `bucket_0_users` + `bucket_0_transactions`). Tasks are distributed across executors (worker nodes).
- › **Data Loading:** Executors read only the bucket files assigned to them (e.g., `bucket_0/` for both users and transactions).
- › **Join Execution:** Each executor joins its bucket pair independently. Results are combined into the final output without requiring shuffle.

Use notebooks for exploratory, ad-hoc, or collaborative tasks, but avoid them for production, complex pipelines, or automated deployments – unless you are on Databricks

Don't use notebooks in production!

- › Lack of Version Control: Jupyter Notebooks mix code, documentation, and output, making it harder to manage changes systematically compared to traditional scripts in version control systems like Git.
- › Reproducibility Issues: The execution order of cells can lead to inconsistent states, making it difficult to ensure reproducibility in a production environment.
- › Limited Logging and Monitoring: Production systems require robust logging and monitoring, which Jupyter Notebooks lack natively, making debugging and tracking issues challenging.

Demo Notebook

The screenshot displays the Azure Databricks Notebook interface. The main area shows four code cells (Cmd 1 to Cmd 4) with their respective outputs. Cmd 1 and 2 are empty. Cmd 3 shows the output of a pandas import. Cmd 4 shows the output of a Spark SQL query, which is displayed as a table.

Revision history sidebar:

- Git: Not linked
- Pending revision (Save now)
- Jul 24 2023, 16:17 PM BST (Jessica Hill)
- Jul 24 2023, 16:16 PM BST (Jessica Hill)
- Jul 13 2023, 13:50 PM BST (Jessica Hill)
- Jul 10 2023, 18:01 PM BST (Jessica Hill)
- Jul 10 2023, 17:59 PM BST (Jessica Hill)
- Mar 29 2023, 18:38 PM BST (Jessica Hill)
- Mar 29 2023, 18:34 PM BST (Jessica Hill)
- Mar 29 2023, 18:32 PM BST (Jessica Hill)
- Mar 29 2023, 18:27 PM BST (Jessica Hill)
- Mar 29 2023, 18:24 PM BST (Jessica Hill)
- Mar 29 2023, 18:16 PM BST (Jessica Hill)
- Mar 29 2023, 18:08 PM BST (Jessica Hill)
- Mar 29 2023, 17:58 PM BST (Jessica Hill)
- Mar 29 2023, 17:52 PM BST (Jessica Hill)

Table output from Cmd 4:

	Product	Sales
1	Bread	1000
2	Milk	500
3	Cheese	300
4	Eggs	25

4 rows | 21.68 seconds runtime | Refreshed 1 minute ago

Exam demo question:

Which of the following keywords is a spark action?

- › `collect()`
- › `groupByKey()`
- › `join()`
- › `first()`
- › `count()`
- › `filter()`
- › `distinct()`
- › `sortByKey()`

Testing in Spark

Verifying your spark code

To test your (py)spark applications we can reuse what we learned about unit testing in python. The spark session can be initialized ones and reused!

Demo function to test

```
def to_uppercase(
    df: DataFrame,
    columns_to_transform: list) -> DataFrame:
    """Uppercase the columns provided in the dataframe
    Args:
        df (DataFrame): Input Dataframe
        columns_to_transform (List): List of columns to uppercase
    Returns:
        DataFrame: The transformed DataFrame
    """

    # Loop through columns to transform and convert to uppercase
    for column in columns_to_transform:
        if column in df.columns:
            df = df.withColumn(column, F.upper(F.col(column)))

    return df
```

Setting up spark is slow, so we use a scoped fixture

```
import pyspark.sql.functions as F
from pyspark.sql import SparkSession, DataFrame

@pytest.fixture(scope="session")
def spark():
    return (
        SparkSession.builder.master("local[*]")
        .appName("PySpark Installation Test")
        .config("spark.driver.host", "127.0.0.1")
        .config("spark.driver.bindAddress", "127.0.0.1")
        .config("spark.executor.heartbeatInterval", "60s")
        .config("spark.driver.port", "7077")
        .getOrCreate()
    )

def test_uppercase(spark) -> None:

    # Set up test data
    columns = ["Brand", "Model", "Age"]
    data = [["Ford", "Focus", 12.5], ["Volkswagen", "Golf", 25.4]]
    ...
```

Set up spark session for all tests

To properly test our spark code, we should follow the design principles (e.g., dependency injection) when setting it up.

Decouple processing from retrieving loading

```
from pyspark.pandas import DataFrame
from pyspark.sql import SparkSession

def transform_data() -> DataFrame:
    spark = SparkSession.builder.appName("HardToTest").getOrCreate()

    df = spark.sql("""SELECT * FROM schema.table""")

    transformed_df = df.filter(df["age"] > 18).select("name", "age")

    return transformed_df
```

Difficult to test since
we need access to the
db

```
def transform_data_dependency_injection(input_df: DataFrame):
    # Apply transformation
    transformed_df = input_df.filter(input_df["age"] > 18)
    .select("name", "age")
    return transformed_df
```

If not possible, use monkey patching

```
from unittest.mock import patch
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, IntegerType, StringType

def test_transform_data():
    # Mock data for testing
    schema = StructType([
        StructField("name", StringType(), True),
        StructField("age", IntegerType(), True)
    ])
    mock_data = [("Alice", 25), ("Bob", 17), ("Charlie", 30)]
    mock_df = spark_session.createDataFrame(mock_data, schema)

    # Patch the `spark.sql` method to return the mock DataFrame
    with patch.object(SparkSession, "sql", return_value=mock_df):
        result_df = transform_data()

        result = result_df.collect()
        expected = [("Alice", 25), ("Charlie", 30)]

        assert sorted(result) == sorted(expected)
```

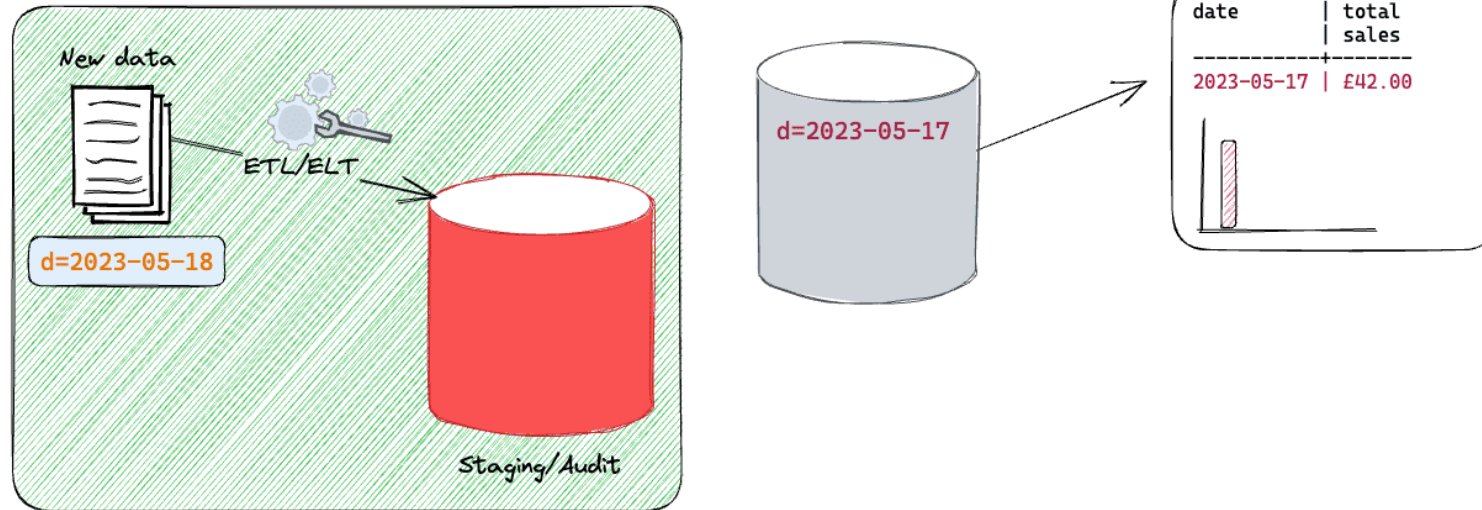
If you cannot use
dependency injection,
you can monkey patch
the response

Write-Audit-Publish (WAP) is a pattern in data engineering that gives teams greater control over data quality



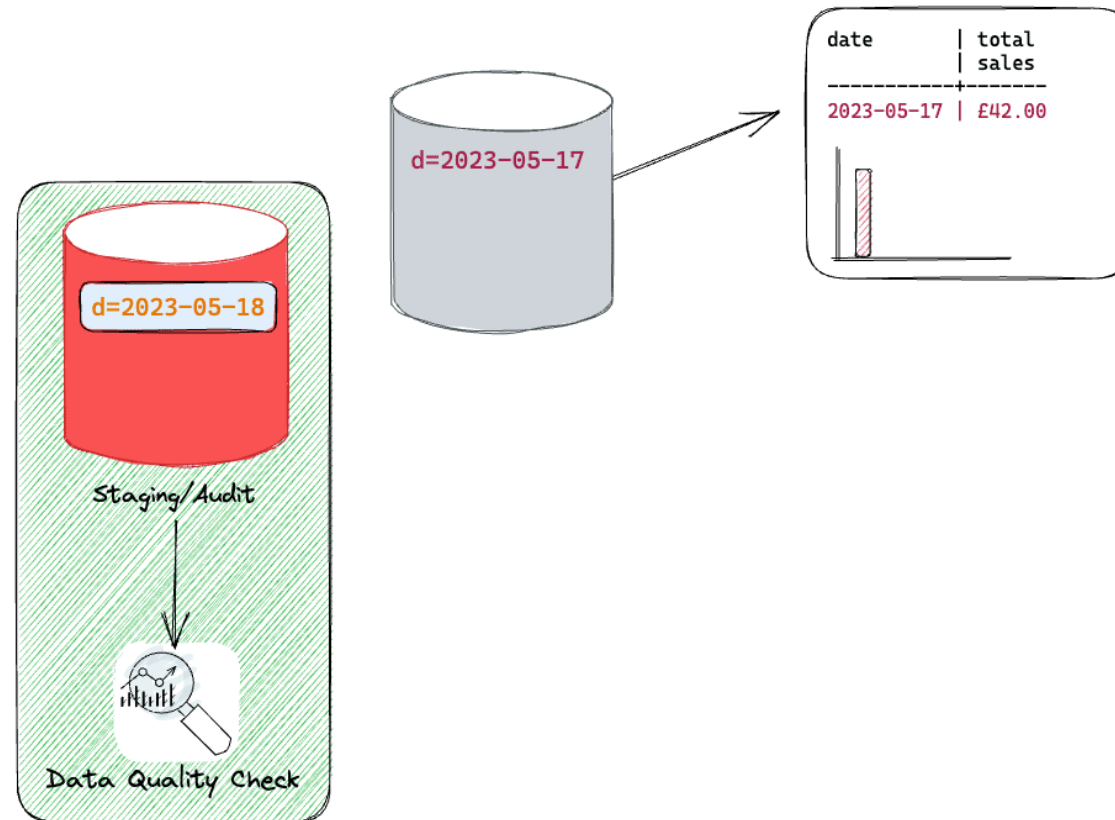
Write the data that you are processing to somewhere that is not read by consumers downstream. This could be a staging or temporary area, a branch, etc.

Write

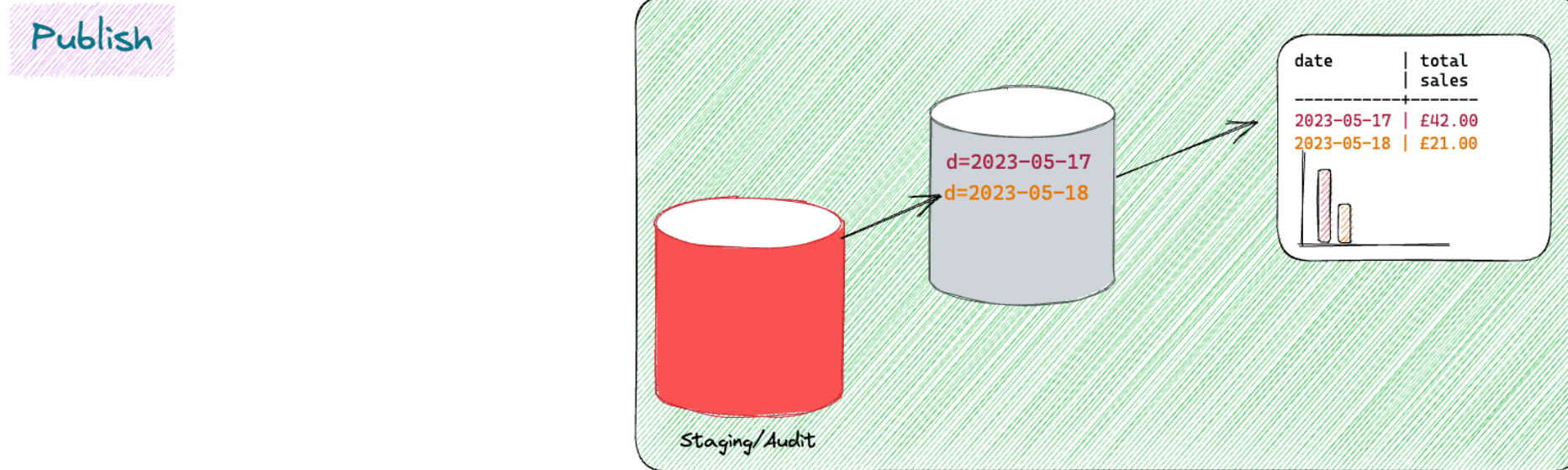


Audit the data to make sure that it meets the data quality specifications

Audit



Publish the data by writing it to the place from which consumers downstream read it



Data engineering is not as mature in terms of testing / development maturity compared to classical roles in software engineering – and this will change!

Data engineering, once seen as lower risk with less rigorous development practices, must evolve to match the rising stakes and trust placed in data-driven systems and AI.

- ▶ **Data Engineering is Currently Perceived as Lower Risk.** In traditional software engineering, system failures often have immediate and far-reaching consequences. For example, if a bug causes a major platform like Facebook to go down, the impact is widespread, affecting millions of users in real time. By contrast, data engineering issues—such as an internal analytical dashboard being unavailable—are often viewed as less critical because their immediate impact is confined to internal operations rather than customer-facing services.
- ▶ **Cultural and Educational Differences shape the Field:** The evolution of data engineering has been shaped by diverse backgrounds and methodologies. While software engineers often come from a computer science foundation with rigorous training in software development best practices, many data engineers enter the field from varied disciplines like data science, analytics, or even business domains. This diversity has contributed to different cultural approaches to testing, tooling, and development rigor.
- ▶ **The Rising Stakes of Data Engineering:** As organizations increasingly rely on data to drive automated decision-making and AI systems, the stakes are becoming higher. Trust in data systems is growing, but with this trust comes greater risk. A failure in data pipelines or inaccurate insights can lead to flawed AI decisions, financial losses, or reputational damage. This shift will demand more rigorous testing, development maturity, and operational resilience within data engineering.

Food for thought: Encapsulation is a programming principle. Do you know a popular framework in Data Engineering that is at odds with this?

Encapsulation is a programming principle that involves bundling data and the methods that operate on it within a single unit, such as a class, while restricting direct access to the internal details

SQL comes with “weaknesses”....

... when it comes to SWE principles

- › **Global Scope for Tables and Columns:** SQL operates on globally accessible tables and columns, meaning queries can directly interact with underlying data structures. There is no natural way to hide or abstract the internal workings of a table from SQL users, exposing the schema and implementation details to anyone with access.
- › **Limited Procedural Encapsulation:** While SQL supports views, stored procedures, and functions, these tools are often less flexible compared to encapsulation mechanisms in programming languages. For instance, views in SQL do not truly encapsulate because users can often bypass them by querying the underlying tables directly.
- › **Schema Changes Impact Consumers:** Changes to the underlying schema (e.g., renaming a column or restructuring a table) often break dependent queries and views, demonstrating a lack of isolation. Encapsulation in other paradigms would typically abstract these changes from the end user.
- › **Direct Data Access Encouraged:** SQL encourages direct querying of data, which can lead to tightly coupled logic spread across different queries and applications. This tight coupling makes it difficult to isolate and reuse logic effectively.

AWS Glue

A managed spark solution

Running Apache Spark in the cloud enables scalable, distributed data processing by leveraging managed services for simplified deployment and resource management.

Cloud platforms provide pre-configured, elastic infrastructure to handle varying workloads, allowing Spark jobs to scale efficiently without manual intervention. Additionally, integration with cloud-native storage, monitoring, and security tools ensures seamless data pipelines and robust operational support for analytics and machine learning tasks.

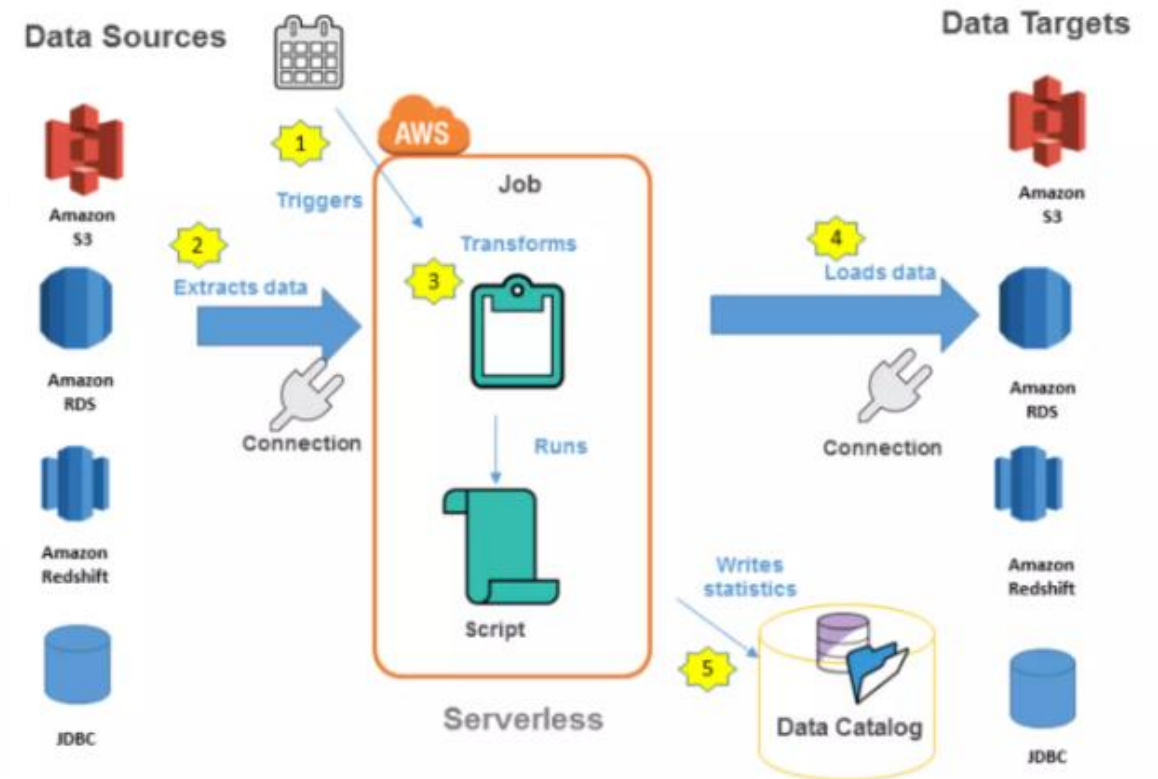
- ▶ **Databricks:** A managed Spark platform for collaborative data engineering, analytics, and machine learning, offering auto-scaling, optimized performance, and built-in workspace tools.
- ▶ **Amazon EMR:** AWS's managed cluster service for Spark and Hadoop, providing cost-effective, scalable big data processing integrated with AWS tools like S3 and CloudWatch.
- ▶ **AWS Glue:** A serverless ETL service with built-in Spark support, automating job orchestration and offering schema discovery for data transformation and integration.
- ▶ **Google Dataproc:** Google Cloud's managed Spark and Hadoop service, enabling fast cluster setup, preemptible instance support, and seamless Google Cloud integration.
- ▶ **Kubernetes with Spark Operator:** An open-source solution for running Spark jobs in containerized environments, offering fine-grained resource control and compatibility with cloud-native ecosystems.

Glue automates the undifferentiated heavy-lifting of ETL

Job composition and triggers

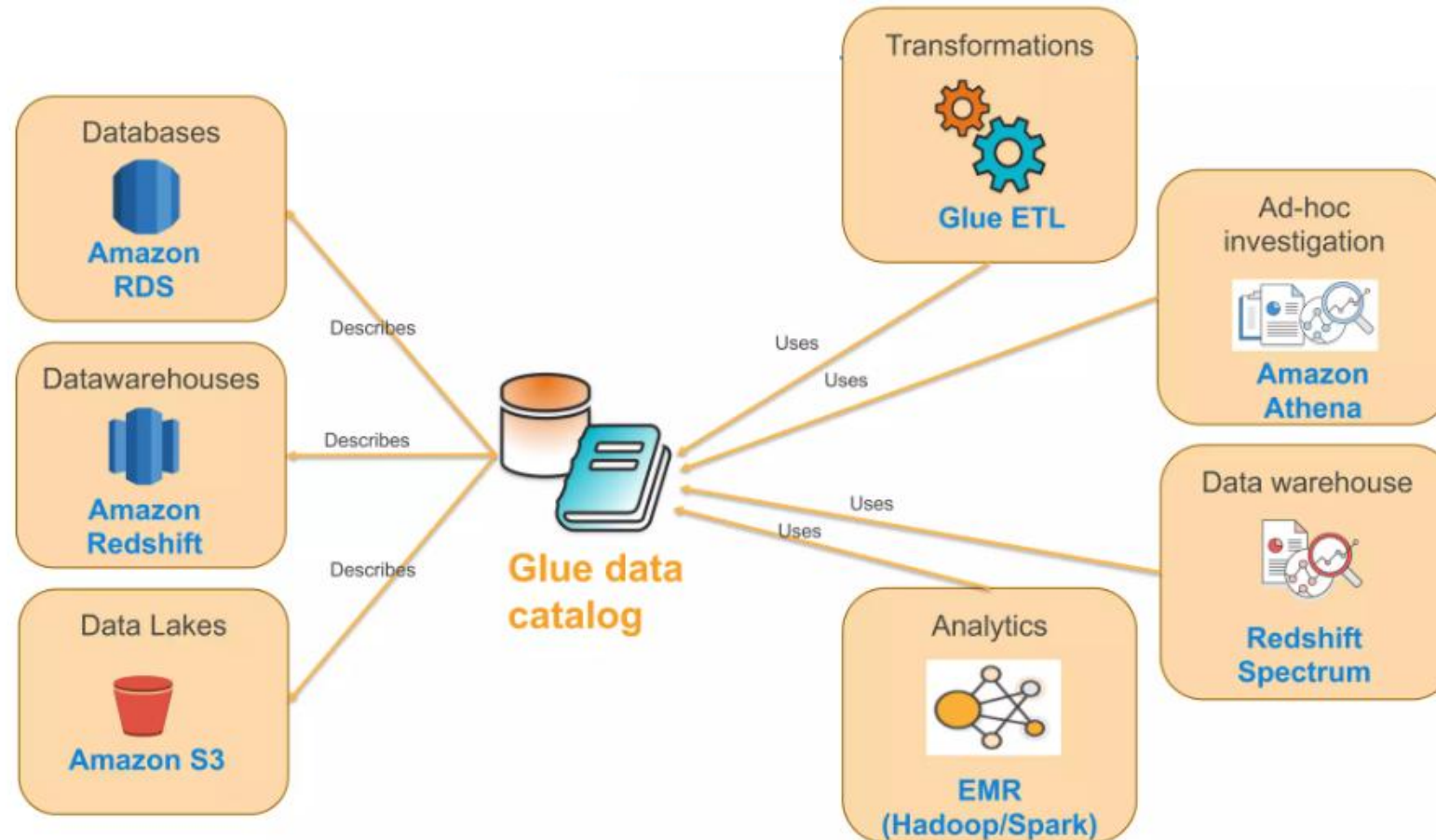
- › Discover and organize data, regardless of where it lives
- › Focus on writing transformations, not handling undifferentiating heavy lifting
- › ETL jobs run under a serverless execution model (not manual provisioning + pay only what you use)

AWS Glue: The Big Picture



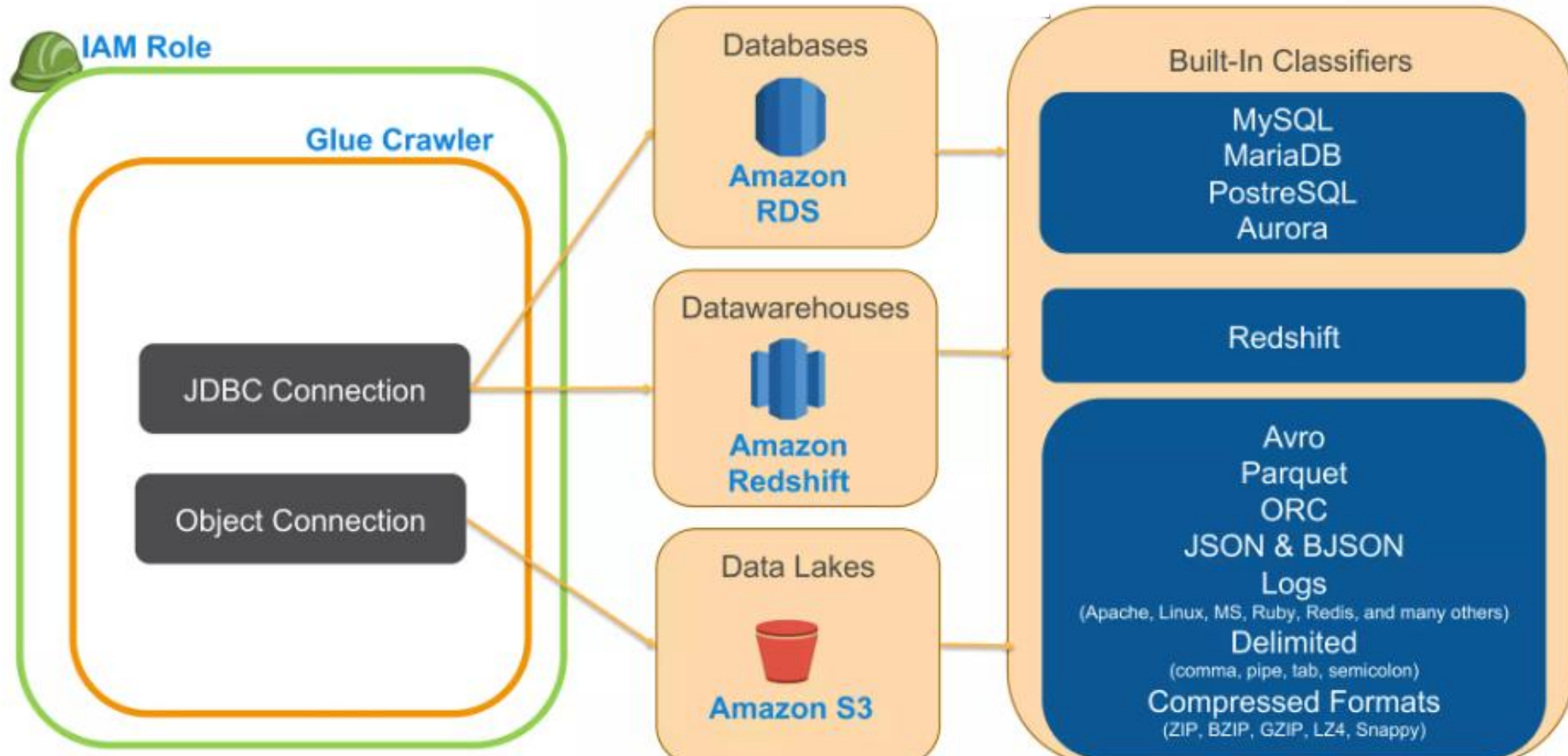
AWS Glue Data Catalog serves as a centralized repository that stores metadata about your data assets, including their schemas, locations, and formats across various data sources.

It enables efficient data discovery, management, and governance, allowing ETL jobs and analytics tools to seamlessly access and utilize consistent metadata for processing and analysis.



AWS Glue Crawlers automatically scan your data sources, infer their schemas, and populate the AWS Glue Data Catalog with up-to-date metadata.

By continuously monitoring data changes, they ensure that your metadata remains current, facilitating efficient data discovery, organization, and preparation for ETL processes and analytics tasks.

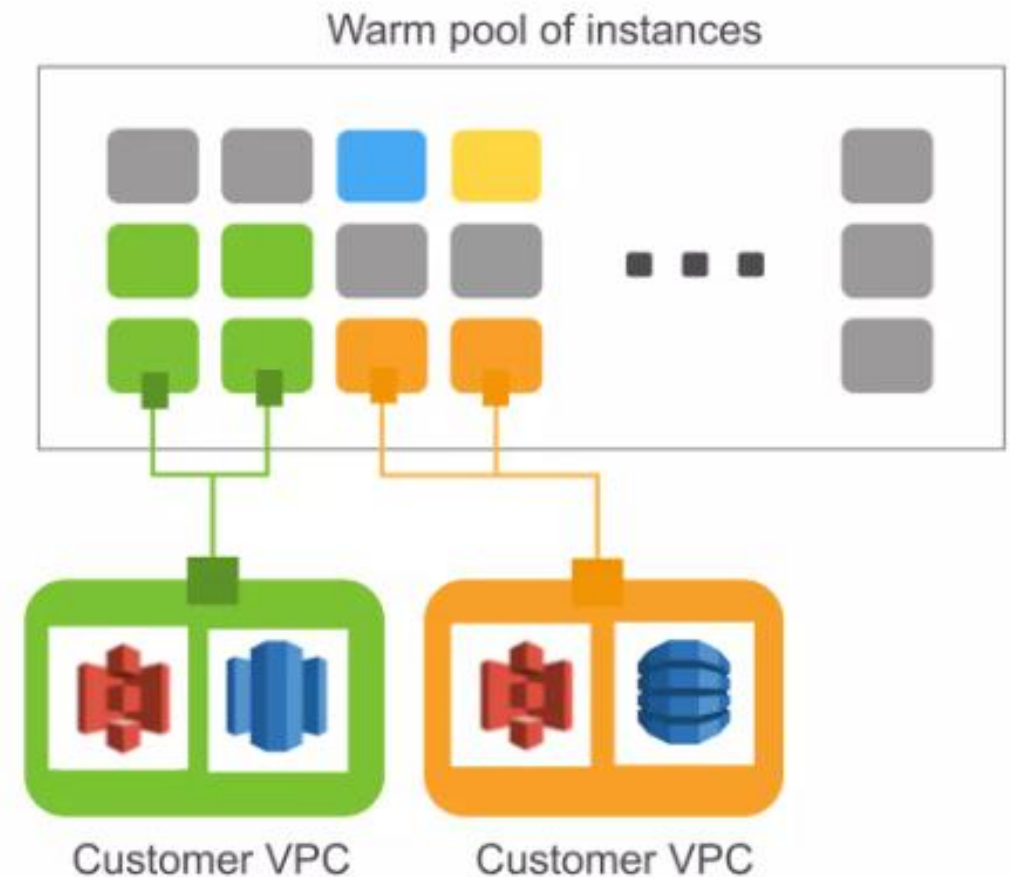


Glue offers serverless execution, so there is no need to provision, configure or manage servers & you pay only what you consume

Serverless eases setup costs

- › **Warm Pools:** Pre-configured fleets of instance to reduce job start up time
- › Auto configure VPC and role based access
- › Automatically scale resources to meet SLA and cost objectives

... when it comes to SWE principles

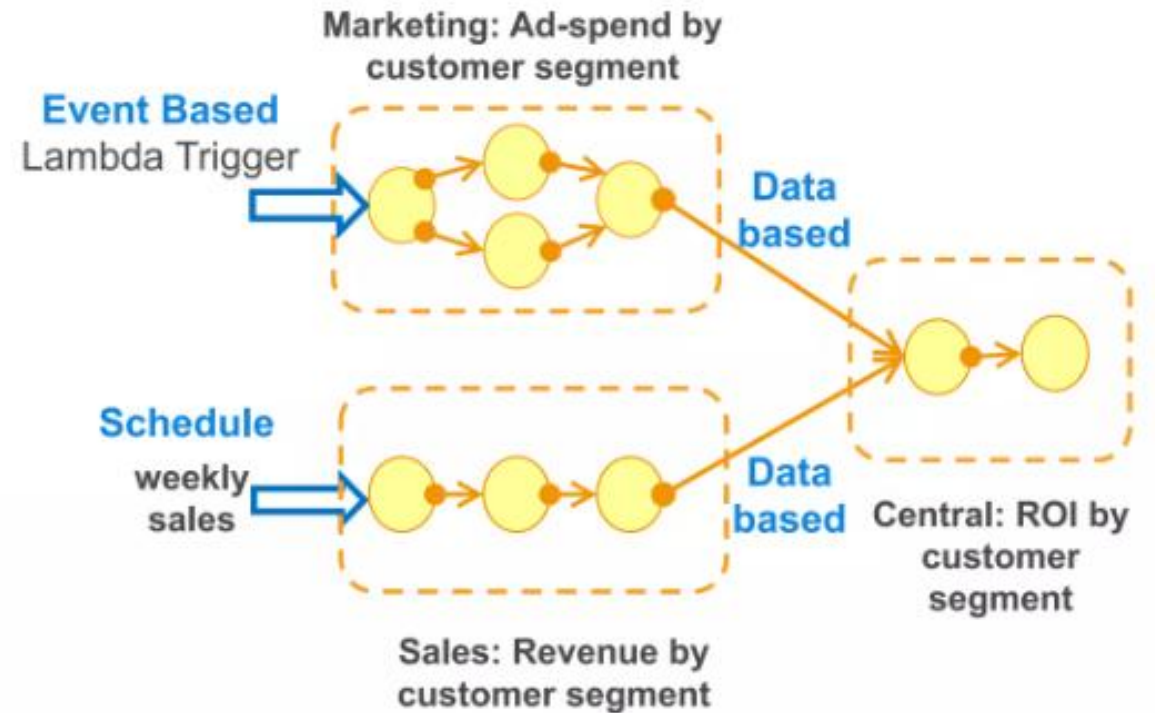


You can orchestrate glue jobs with tools like airflows or fully AWS native through step functions, schedules and event-based triggers

Job composition and triggers

- › Compose jobs globally with event-driven dependencies
 - › Easy to reuse and leverage work across organization
- › Multiple triggering mechanisms:
 - › Scheduled based: e.g., time of day, CRON
 - › Event-based: e.g., data availability, job competition
 - › External resources: e.g., AWS Lambda boto3 call
- › ETL jobs can be launched in a VPC
 - › Jobs requiring restricted access to data Glue utilizes a VPC endpoint launching dedicated ENIs to the customers VPC which are assigned private IP address from the customer's subnet.

Demo job



Data Lakehouse

A dream coming true...

Data Lakehouse

Data lakehouses are an emerging architecture that blends the best features of data lakes and data warehouses to address the limitations of each. Traditional data lakes offer vast storage capabilities at low cost, accommodating a wide variety of data types. However, they often lack the robust management and performance optimization necessary for advanced analytics and BI tasks, leading to issues like data swamps, where unorganized and ungoverned data becomes unusable. Data warehouses, on the other hand, provide structured data management, high performance, and strong governance but can be expensive and inflexible when dealing with large volumes of diverse data types. Data lakehouses aim to solve these problems by integrating the scalability and flexibility of data lakes with the reliability and performance of data warehouses.

1

Data Warehouse → Data Lakehouse

Learn about data warehouse deficiencies and how data lakehouses attempt to overcome them

2

Iceberg Architecture

Learn about icebergs architecture, in particular its meta data handling scheme

3

Iceberg transactions

Learn about iceberg transactions, create table statements, upserts, selects and time travel

4

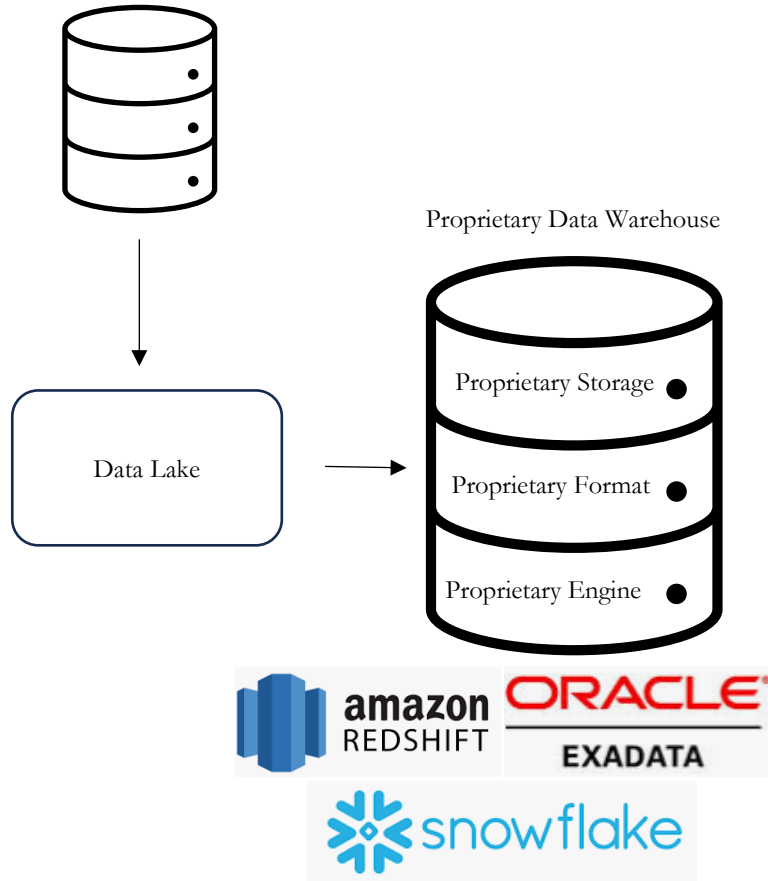
Iceberg schema & partition evolution

Learn why Iceberg performs superior when considering changing data models are a part of your problem

Data Warehouse → Data Lakehouse

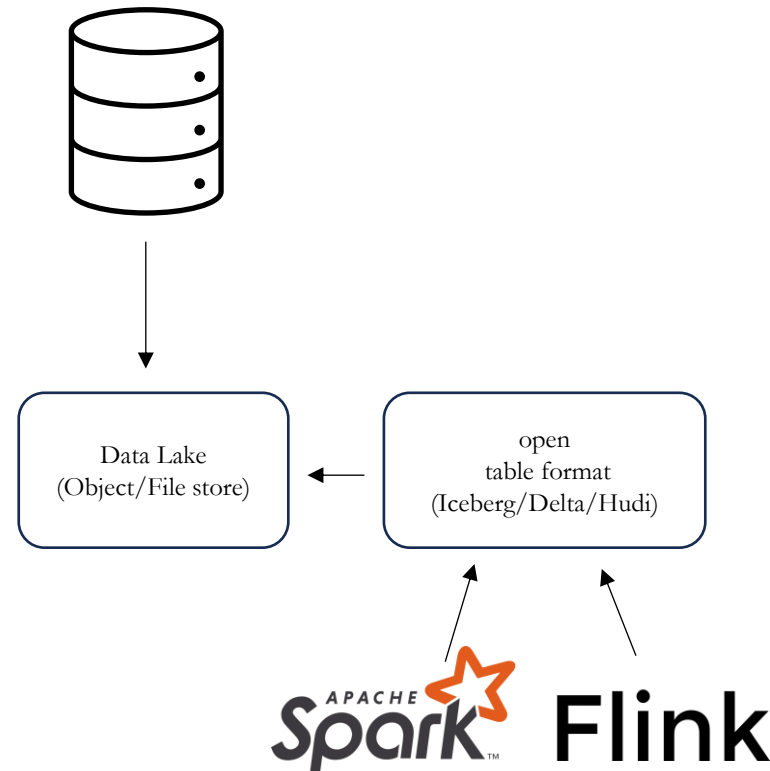
Status Quo

Transactional source database



Data Lakehouse

Transactional source database



Components

- › Storage Layer
 - › HDFS, S3,...
- › File Format
 - › parquet, ORC, AVRO, ...
- › **Table Format**
 - › Iceberg, Delta, Hudi ...
- › Catalog
 - › AWS Glue Catalog, Hive, Nessie
- › Lakehouse Engine
 - › spark, dremio, flink,...

Data Warehouse → Data Lakehouse

Data Warehouse vs. Data Lakehouse

Problems with Data Warehouse:

- › Expensive
- › Proprietary components
- › Vendor lock-in
- › Data drift

Benefits of Data Lakehouse:

- › Cheap
- › Performant
- › Open source

Data Lakehouse Features/Promises

- › **Data Consistency:** Through ACID transactions.
- › **Schema Evolution:** Allowing schema changes without breaking existing queries.
- › **Data Versioning:** Time travel for accessing historical data.
- › **Efficient Data Storage:** Using columnar formats like Parquet and ORC.
- › **Metadata Management:** Efficient handling of metadata for faster query performance.

Traditional data warehouse systems offered no insight into how data is stored and processed; open table formats filled that gap

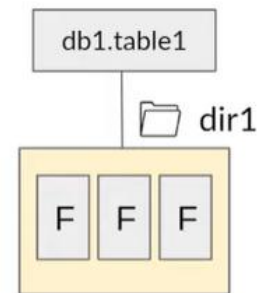
Traditional Database/data warehouse systems

- › Little / no insights into how data is physically persisted
- › Proprietary formats / engines
- › Scaling up is expensive / licensing
- › Vendor lock-in / “cool new tool” lock-out



What is table format?

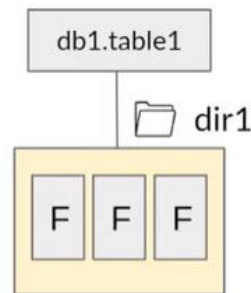
- › A way organize files of a dataset to present them as a single table
- › A way to answer the question: “what data is in this table”?
- › Why all that? MapReduce jobs would read and write files.
- › Annoying to figure out which files to read and write + complex Java coding
- › Facebook creates Hive to write SQL (which under the hood creates MapReduce code to do data transformations)
- › All files in folder are the tables data
- › Allows partitioning using sub folders
- › Very popular since easy to work with



Hive is the most popular and widely adopted table format, but some of its limitations created the need for modern table formats which are build for change

Pros

- › Defacto standard, works with most engines (e.g. spark, impala, dremio,...)
- › Efficient access patterns using partitions (no full table scan required)
- › File format agnostic (.csv, .tsv, .parquet,...)
- › Atomically update one partition (ACID compliant *within* a partition)



Cons

- › Small updates are costly (To update one record, in one partition, in one file,... rewrite the entire partition)
- › No way to change data in multiple partitions safely / Multiple jobs cannot modify the same data safely
- › List all directories / files in all relevant partitions, read data to filter → expensive
- › Users have to know the physical layout of the table
- › Hive statistics are often stale (need to rerun analyse all the time)
- › Two solutions
 - › Throw more compute at it, which is expensive
 - › Find a smarter way (→ Iceberg)

Iceberg/Delta/Hudi tables are three popular contenders to solve the issues Hive has. All promise improved performance and ease of use using a set of meta data layers

Iceberg/Delta/Hudi is...

- › A standard for how to organize data around your table
- › A table format specification
- › A set of APIs for interaction with that specification (spark, flink,...)
- › Not a storage engine (Hadoop or S3)
- › Not a file format (like parquet or csv)
- › Not an execution engine (like spark or flink)

Benefits

- › Efficient small updates / build for change
- › Abstract the physical, expose the logic view
- › Better performance due to richer meta data for query optimization
- › Lower cost as costly rewrites and list operations are avoided

The three main data lakehouse contenders are Iceberg, DeltaTables and Hudi

Apache Iceberg



- › Developed by Netflix in 2018; later donated to the Apache Software Foundation.
- › Distinct Features: Supports hidden partitioning, which avoids issues related to partition management; provides full schema evolution without the need for data rewrites.
- › Widely adopted by companies like Netflix, Apple, and LinkedIn; known for large-scale analytics and its integration with engines like Apache Spark, Flink, and Trino.

Delta Lake



- › Developed by Databricks in 2017 and open-sourced in 2019.
- › Distinct Features: Seamlessly with Databricks' ecosystem and Apache Spark.
- › Highly popular within the Databricks ecosystem; used by companies like Microsoft, Shell, and Comcast for data engineering and machine learning workflows.

Apache Hudi



- › Developed by Uber in 2016; became an Apache Top-Level Project in 2020.
- › Distinct Features: Offers near-real-time data ingestion with its "merge on read" feature; supports upserts and deletes efficiently, which is particularly beneficial for stream processing.
- › Gaining traction for use cases involving real-time analytics and CDC; used by companies like Uber, Amazon, and Alibaba.

Iceberg, DeltaTables and Hudi differ in terms of meta data management, but the core idea is similar: Use metadata for more efficient data processing

Solution

Differences in terms of

Schematic

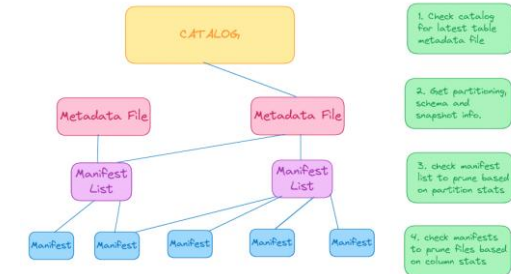
1

Apache
Iceberg

Meta data approach: 3-tier metadata tree

Data Skipping: Based on Manifest Files

File Types: Metadata.json , Manifest Lists, Manifest



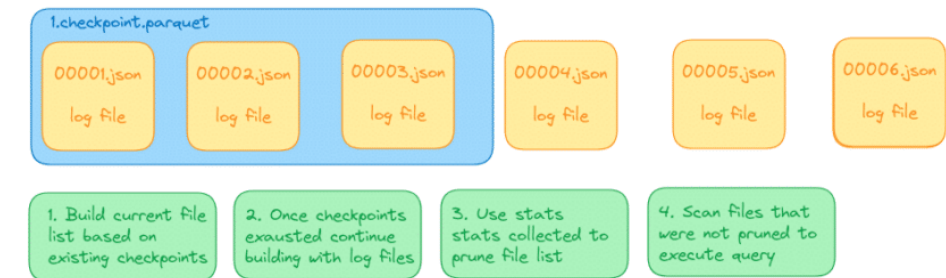
2

Databricks
Delta Tables

Meta data approach: Logs & Checkpoints

Data Skipping: File Stats from Log files

File Types: Log Files and Checkpoints



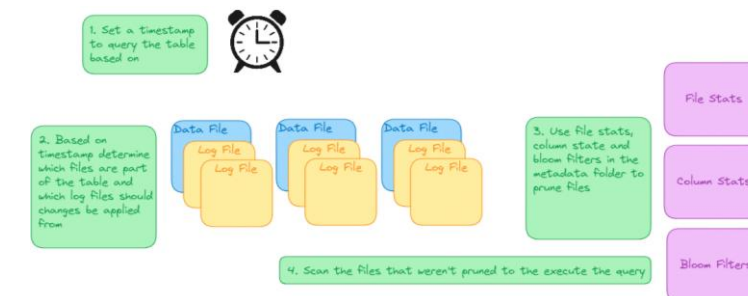
3

Apache Hudi

Meta data approach: Timeline

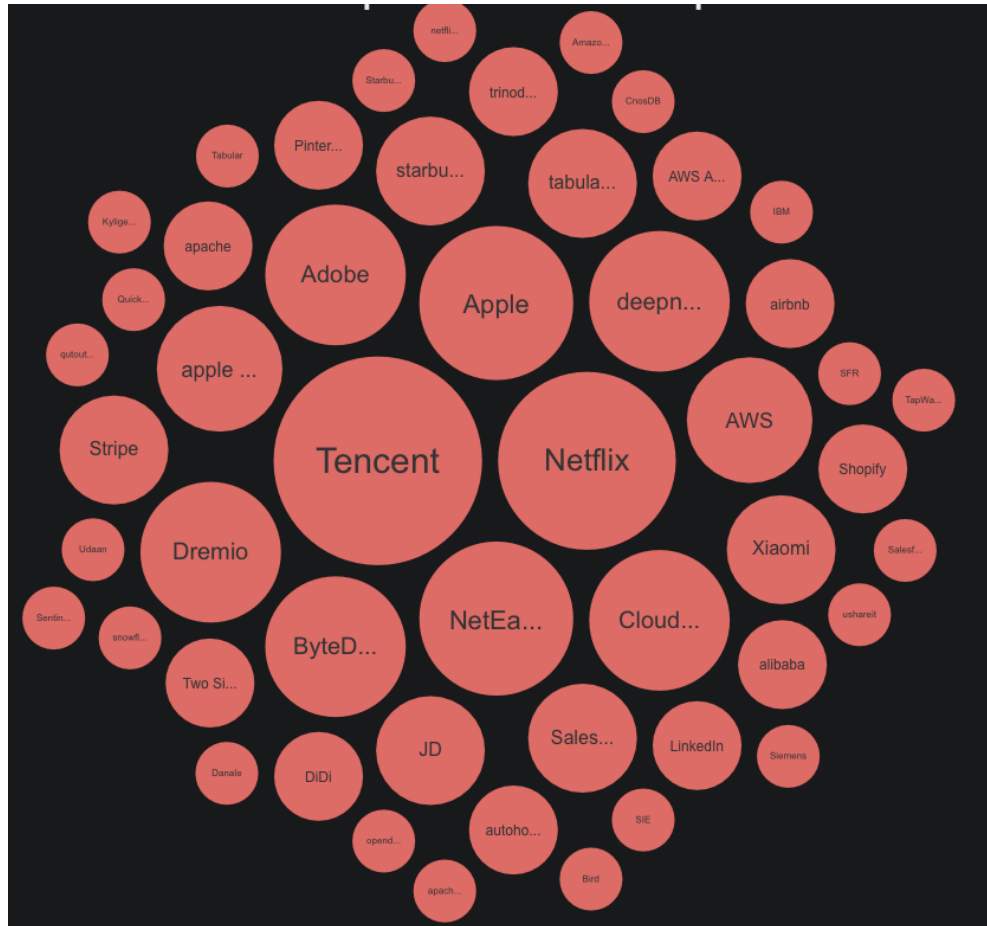
Data Skipping: Based on column statistics indexes in Metadata table

File Types: Log Files, Metadata Table, Requested Inflight Commit Files



1000000

Pull requests creators companies

Source: [OSSInsight](#)

Even Zach is convinced...

Zach Wilson (Annoying, Unhinged) • [in](#) • F...

Join my Analytics Engineering Boot camp starting October 21st!

Visit my website

23h • 🌐

There will be companies in 2050 who are still using Apache Iceberg and getting tons of value.

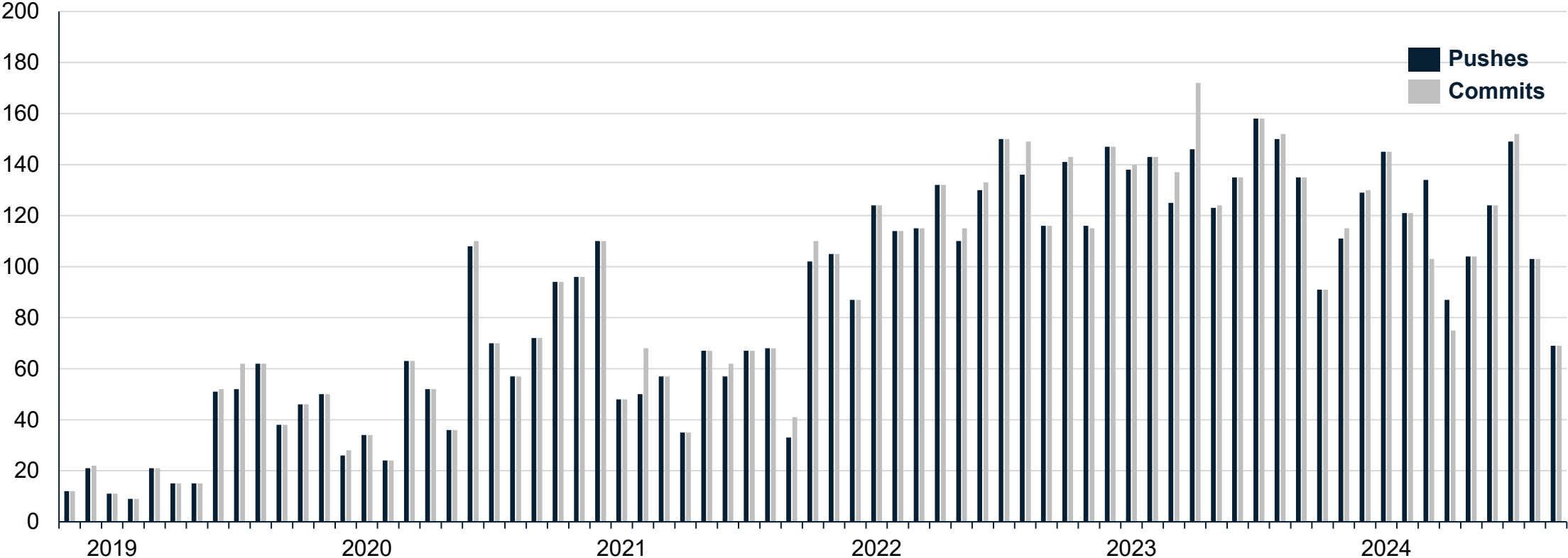
Platform agnostic storage is unbelievably powerful. This is the open source future we've been waiting for. Hive almost delivered it which is why it had over a decade of longevity.

Iceberg is designed the right way to solve this problem
universally for decades to come!

#dataengineering

Iceberg is a popular open-source project

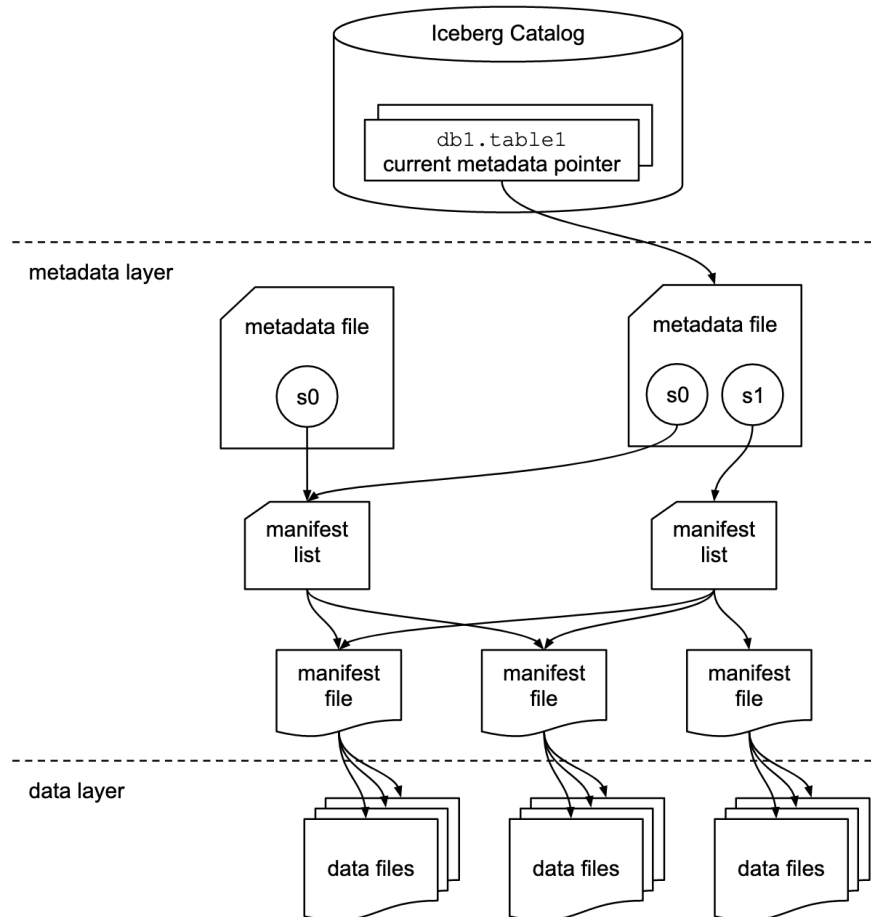
Commits & Pushes in Iceberg, count



Source: <https://ossinsight.io/analyze/apache/iceberg#commits>

Apache Iceberg's architecture is based on a set of metadata files, manifest list and manifest files which points to data files

Structure

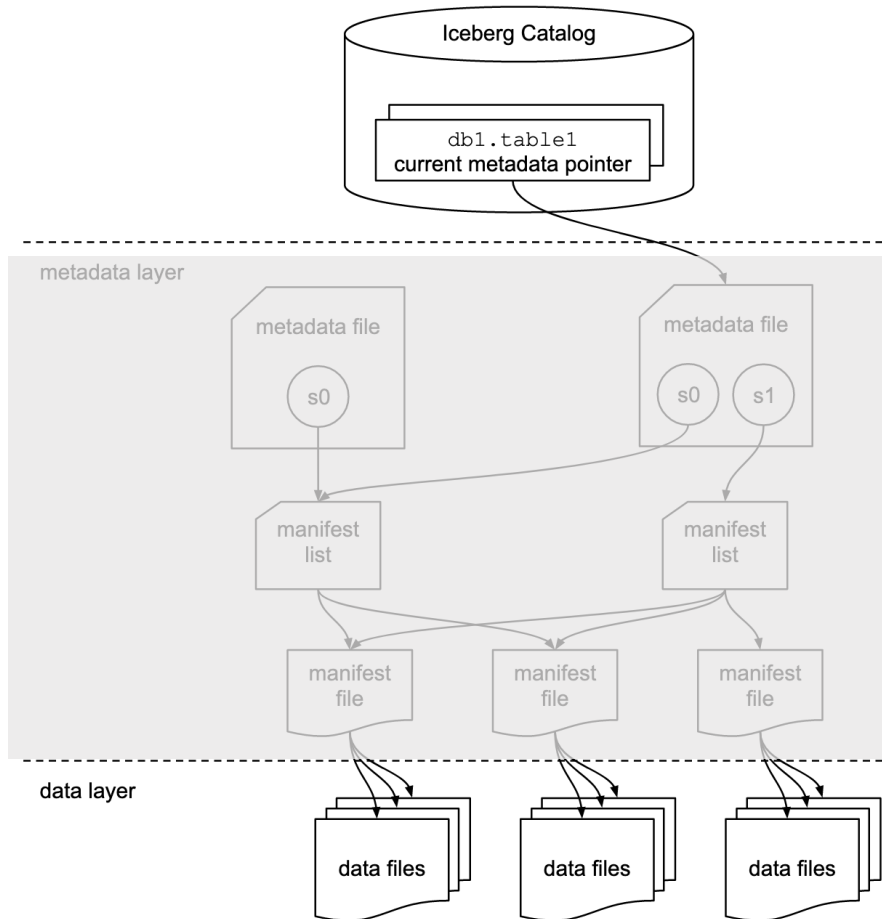


Description

- › Organized in three layers: catalog, metadata, data
- › Iceberg Catalog
 - › Which tables exist
- › Metadata Layer
 - › Metadata file: defines the table
 - › Manifest list: defines a snapshot of the table with a list of manifest files that make up snapshot and metadata about their data
 - › Manifest file: list of data files (e.g. s3 keys)
- › Data layer
 - › Files on S3/HDFS/...

Compared to pure Hive, which only has information on table and folder level

Structure

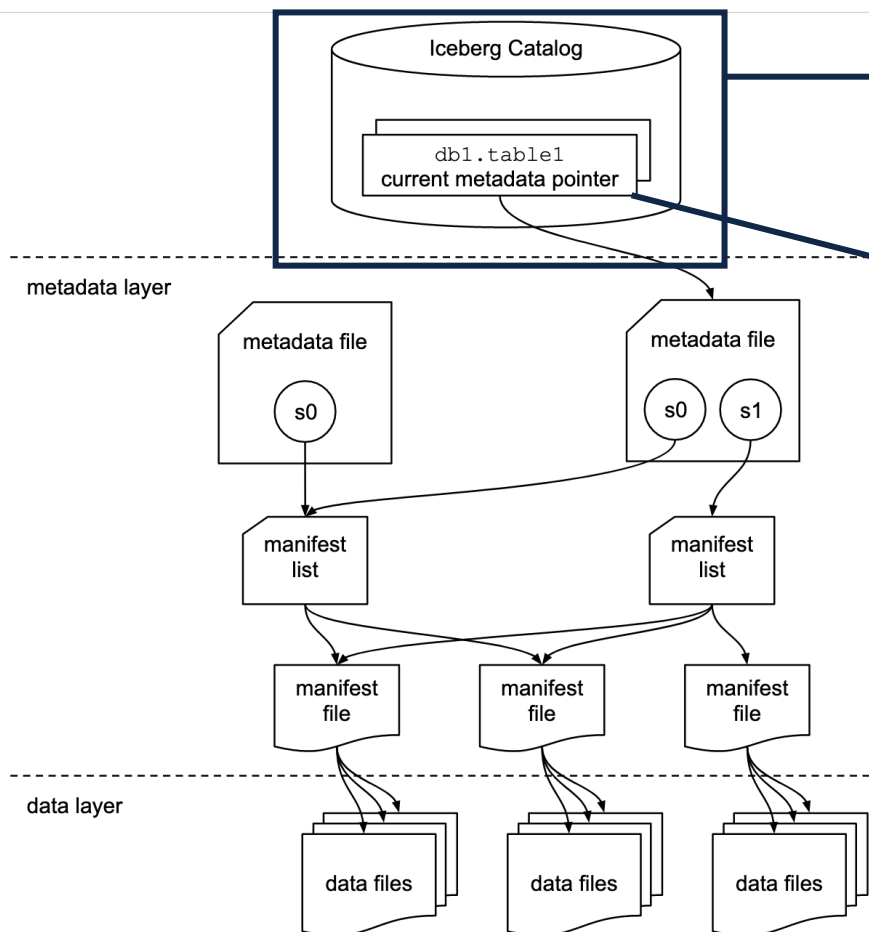


Description

- › Static information of a table
- › Little/No meta information

Apache Iceberg Architecture

Structure



Description

› Iceberg Catalog

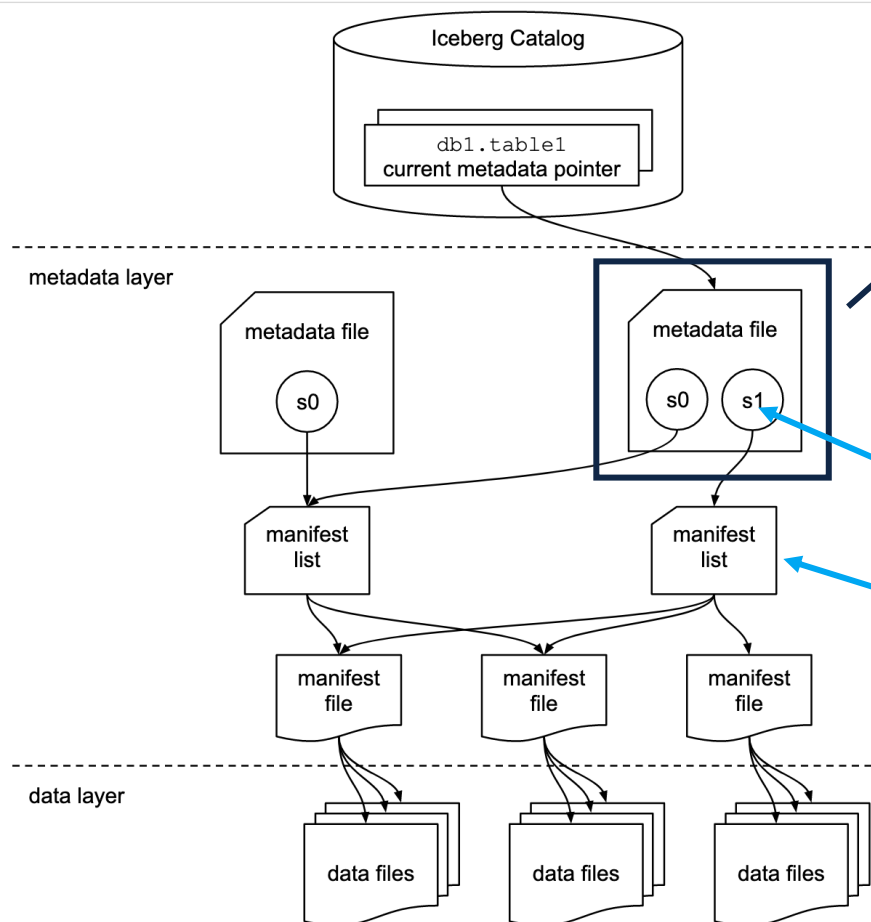
- › A store that houses the current metadata pointer for Iceberg tables
- › Must support atomic operations

› table1's current metadata pointer

- › Mapping of table name to metadata file

Apache Iceberg Architecture

Structure



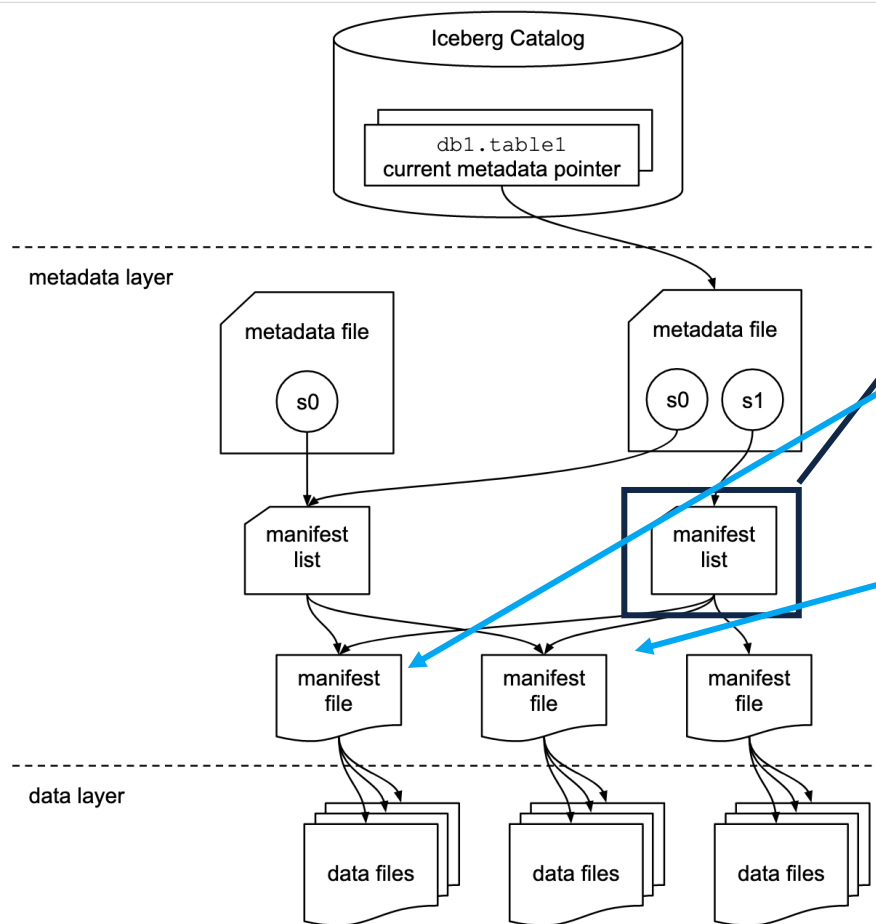
Description

Metadata file: stores metadata about a table at a certain point in time

```
{  
  "table--uuid": "<uuid>",  
  "location": "/path/to/table/dir"  
  "schema": {...},  
  "partition_spec": [{<partition-details>},...],  
  "current-snapshot-id": "s1",  
  "snapshots": [  
    {"snapshot_id": "s0",  
     "manifest-list": "path/manifests0.avro"},  
    {"snapshot_id": "s1",  
     "manifest-list": "path/manifests1.avro"},  
  ]  
}
```

Apache Iceberg Architecture

Structure



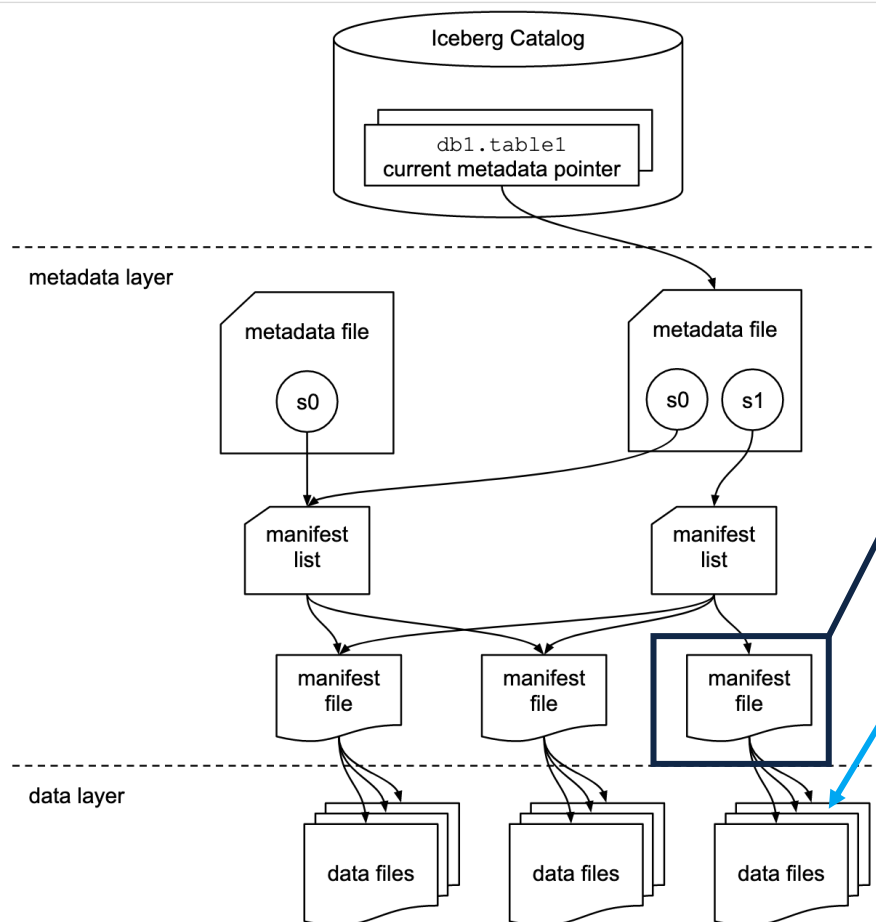
Description

› **Manifest list file:** a list of manifest files

```
{  
  "manifest-path": "/path/manifest/file0.avro",  
  "added-snapshot-id": "s0",  
  "partition-spec-id": <partitions-spec-s0>,  
  "partitions": [{partition_info},...],  
  ...  
}, {  
  "manifest-path": "/path/manifest/file1.avro",  
  "added-snapshot-id": "s1",  
  "partition-spec-id": <partitions-spec-s1>,  
  "partitions": [{partition_info},...],  
  ...  
},  
  "manifest-path": "/path/manifest/file2.avro",  
  "added-snapshot-id": "s1",  
  "partition-spec-id": <partitions-spec-s1>,  
  "partitions": [{partition_info},...],  
  ...  
},  
  ...  
}
```


Apache Iceberg Architecture

Structure



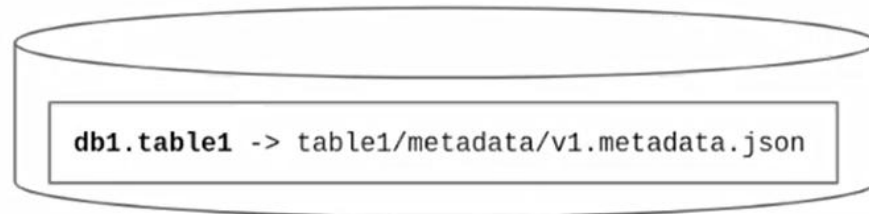
Description

Manifest file: a list of data files, along with details and stats about each data file

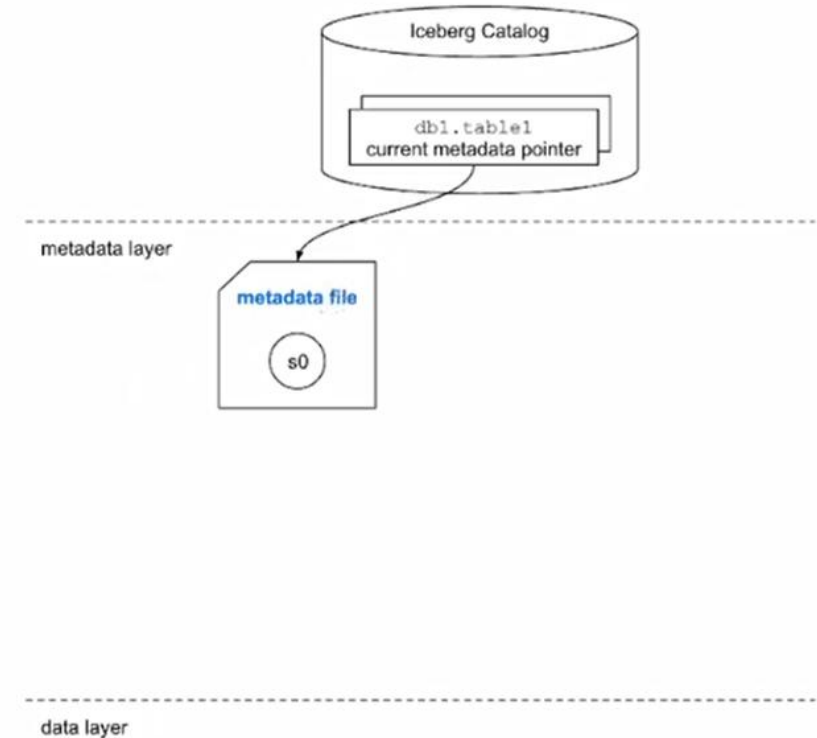
```
{  
  "file_path": "/path/to/data/file1.parquet",  
  "file-format": "parquet",  
  "partition": {"part_field": {"type": <value>}},  
  "record-count": <num-records>,  
  "null-value-counts": [  
    {"column-index": "1", "value": 4}, ...],  
  "lower-bounds": [{...}],  
  "upper_bounds": [{...}],  
}, {  
  "file_path": "/path/to/data/file2.parquet",  
  "file-format": "parquet",  
  ...  
}, ...
```

In a Create table statement, only a metadata file is created, as no data is written yet

```
CREATE TABLE db1.table1 (  
  order_id bigint,  
  customer_id bigint,  
  order_amount DECIMAL(10, 2),  
  order_ts TIMESTAMP  
)  
USING iceberg  
PARTITIONED BY (hour(order_ts));
```



```
table1/  
|- metadata/  
|   |- v1.metadata.json
```



The metadata file contains the current schema and an id

 File Browser

[Back](#)

[Home](#)

[Edit file](#)

[Refresh](#)

[View as binary](#)

[Download](#)

Last modified

08/13/2024 4:03 PM +02:00

User

Group

Size

1.61 KB

Mode

100666

Page 1 to 1 of 1

[⏪](#)[⏴](#)[⏵](#)[⏩](#)

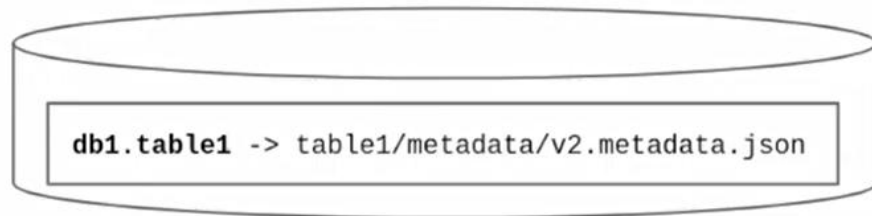
s3a://

:/ db/ table1/ metadata/ 00000-3009c284-3151-45c6-8432-c96b92583358.metadata.json

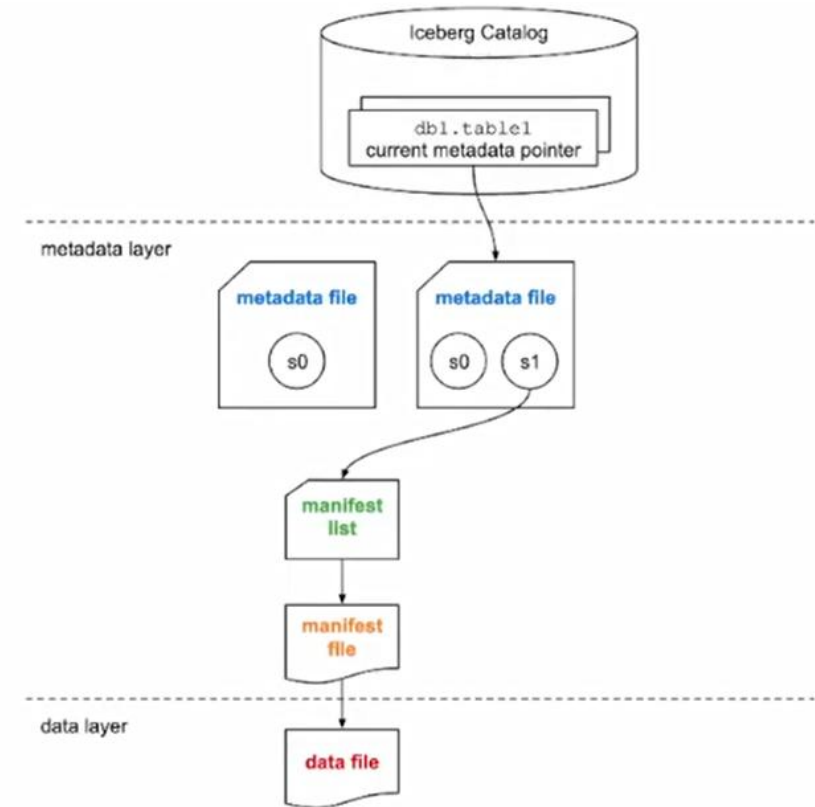
```
{
  "format-version" : 2,
  "table-uuid" : "5f39727e-31e4-45bd-ada6-7014617cd55d",
  "location" : "s3a:///db/table1",
  "last-sequence-number" : 0,
  "last-updated-ms" : 1723557800126,
  "last-column-id" : 4,
  "current-schema-id" : 0,
  "schemas" : [ {
    "type" : "struct",
    "schema-id" : 0,
    "fields" : [ {
      "id" : 1,
      "name" : "order_id",
      "required" : false,
      "type" : "long"
    }, {
      "id" : 2,
      "name" : "customer_id",
      "required" : false,
      "type" : "long"
    }, {
      "id" : 3,
      "name" : "order_amount",
      "required" : false,
      "type" : "decimal(10, 2)"
    }, {
      "id" : 4,
      "name" : "order_ts",
      "required" : false,
      "type" : "timestamp"
    } ]
  } ]
} ]
```

In an INSERT operation, a data file is created along its manifest file and the manifest list

```
INSERT INTO db1.table1 VALUES (  
  123,  
  456,  
  36.17,  
  '2021-01-26 08:10:23'  
);
```



```
table1/  
|- metadata/  
|  |- v1.metadata.json  
|  |- v2.metadata.json  
|  |- snap29c8-1-b103.avro  
|  |- d8f9-ad19-4e.avro  
|- data/  
|  |- order_ts_hour=2021-01-26-08/  
|    |- 0000-5-cae2d.parquet
```



UPSERT Case 1: Previous data changes

db1.table1

order_id	customer_id	order_amount	order_ts
123	456	36.17	'2021-01-26 08:10:23'

table1_stage

order_id	customer_id	order_amount	order_ts
123	456	38.17	'2021-01-26 08:10:23'
456	789	20.00	'2021-01-27 09:00:00'

After update:

order_id	customer_id	order_amount	order_ts
123	456	38.17	'2021-01-26 08:10:23'
456	789	20.00	'2021-01-27 09:00:00'

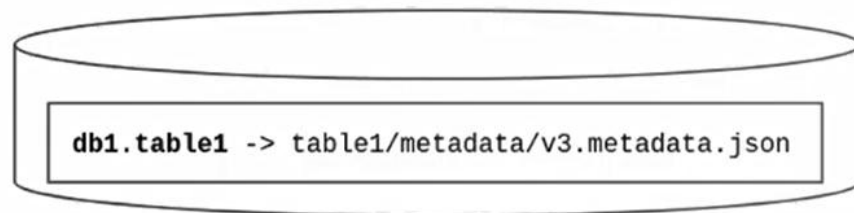


UPSERT query

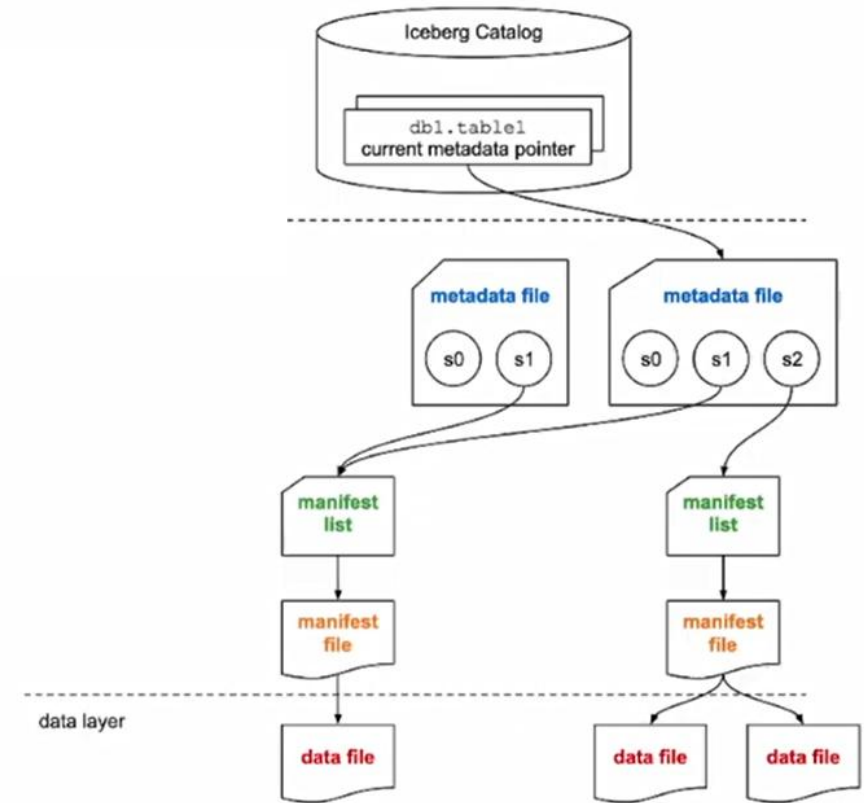
```
MERGE INTO db1.table1
USING (SELECT * FROM table1_stage) s
ON table1.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET
    order_amount=s.order_amount
WHEN NOT MATCHED THEN INSERT *
```

UPSERT Case 1: Previous data changes

```
MERGE INTO db1.table1
USING (SELECT * FROM table1_stage) s
ON table1.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET order_amount=s.order_amount
WHEN NOT MATCHED THEN INSERT *
```



```
table1/
|- metadata/
|   |- v1.metadata.json
|   |- v2.metadata.json
|   |- v3.metadata.json
|   |- snap29c8-1-b103.avro
|   |- snap9fa1-3-16c3.avro
|   |- d8f9-ad19-4e.avro
|   |- 0d9a-98fa-77.avro
|- data/
|   |- order_ts_hour=2021-01-26-08/
|       |- 0000-5-cae2d.parquet
|       |- 0000-1-aef71.parquet
|   |- order_ts_hour=2021-01-27-10/
|       |- 0000-3-0fa3ad.parquet
```



UPSERT Case 2: previous data & partition not affected

db1.table1

order_id	customer_id	order_amount	order_ts
123	456	36.17	'2021-01-26 08:10:23'

table1_stage

order_id	customer_id	order_amount	order_ts
186	548	10.00	'2021-01-27 10:15:00'
456	789	20.00	'2021-01-27 10:05:00'

After update:

order_id	customer_id	order_amount	order_ts
123	456	36.17	'2021-01-26 08:10:23'
186	548	10.00	'2021-01-27 10:15:00'
456	789	20.00	'2021-01-27 10:00:00'

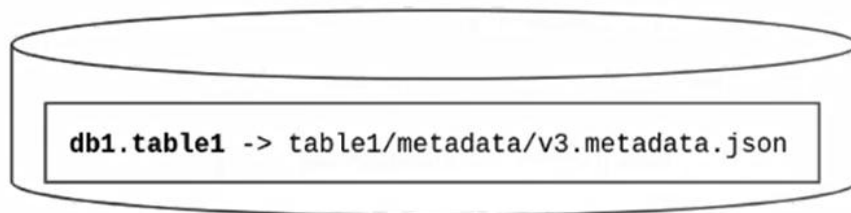


UPSERT query

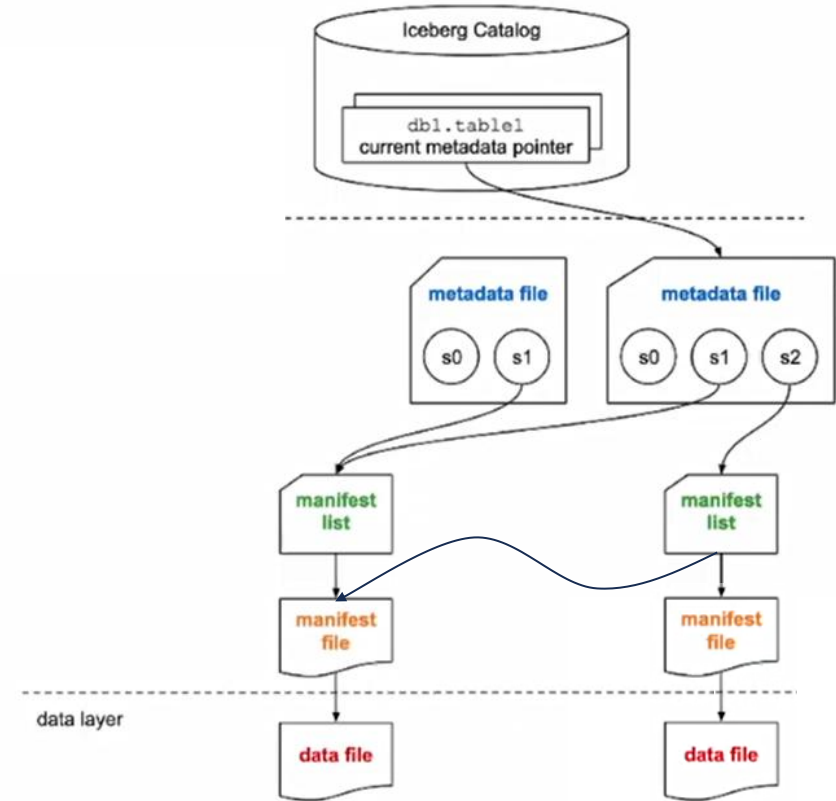
```
MERGE INTO db1.table1
USING (SELECT * FROM table1_stage) s
ON table1.order_id = s.order_id
WHEN MATCHED THEN UPDATED
    table1.order_amount=s.order_amount
WHEN NOT MATCHED THEN INSERT
```

UPSERT Case 2: previous data & partition not affected

```
MERGE INTO db1.table1
USING (SELECT * FROM table1_stage) s
ON table1.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET order_amount=s.order_amount
WHEN NOT MATCHED THEN INSERT *
```



```
table1/
|- metadata/
|   |- v1.metadata.json
|   |- v2.metadata.json
|   |- v3.metadata.json
|   |- snap29c8-1-b103.avro
|   |- snap9fa1-3-16c3.avro
|   |- d8f9-ad19-4e.avro
|   |- 0d9a-98fa-77.avro
|- data/
|   |- order_ts_hour=2021-01-26-08/
|       |- 0000-5-cae2d.parquet
|   |- order_ts_hour=2021-01-27-10/
|       |- 0000-3-0fa3ad.parquet
```



UPSERT Case 3: previous data unaffected, but partition affected

db1.table1

order_id	customer_id	order_amount	order_ts
123	456	36.17	'2021-01-26 08:10:23'

table1_stage

order_id	customer_id	order_amount	order_ts
186	548	10.00	'2021-01-26 08:15:00'
456	789	20.00	'2021-01-27 09:00:00'

After update:

order_id	customer_id	order_amount	order_ts
123	456	36.17	'2021-01-26 08:10:23'
186	548	10.00	'2021-01-26 08:15:00'
456	789	20.00	'2021-01-27 09:00:00'

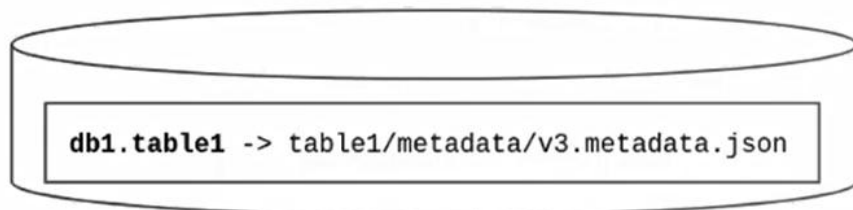


UPSERT query

```
MERGE INTO db1.table1
USING (SELECT * FROM table1_stage) s
ON table1.order_id = s.order_id
WHEN MATCHED THEN UPDATED
    table1.order_amount=s.order_amount
WHEN NOT MATCHED THEN INSERT
```

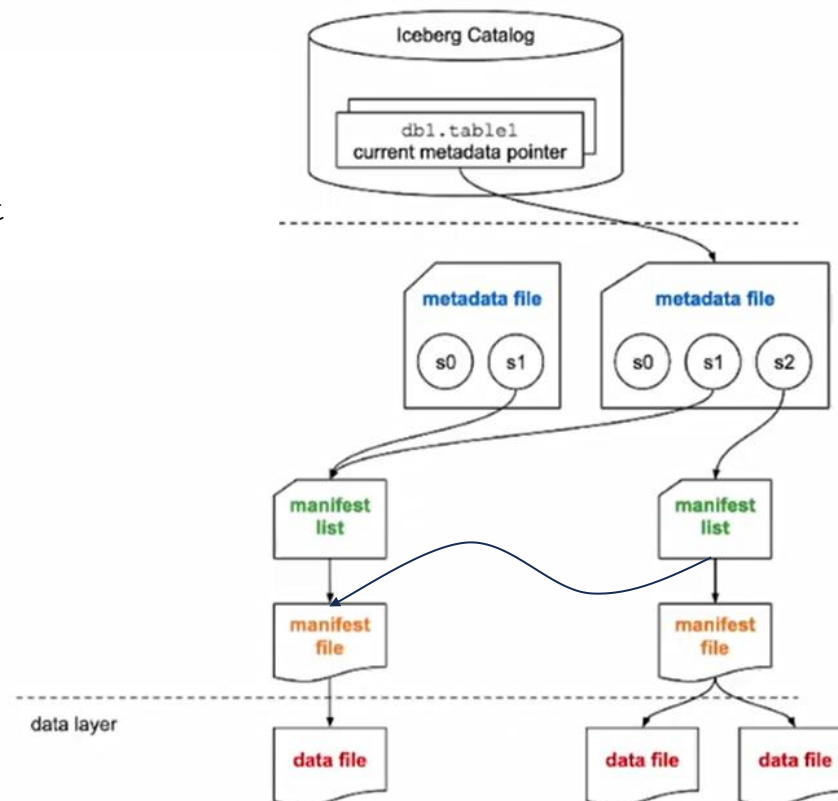
UPSERT Case 3: previous data unaffected, but partition affected

```
MERGE INTO db1.table1
USING (SELECT * FROM table1_stage) s
ON table1.order_id = s.order_id
WHEN MATCHED THEN UPDATED SET order_amount=s.order_amount
WHEN NOT MATCHED THEN INSERT *
```



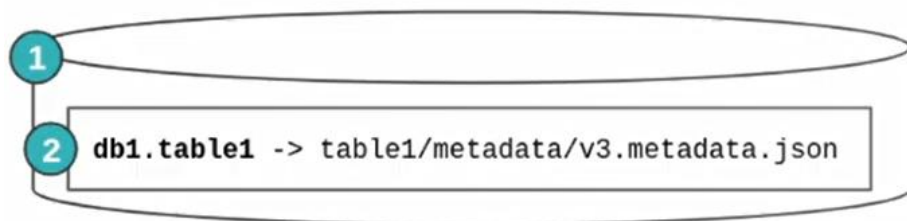
```
table1/
|- metadata/
|   |- v1.metadata.json
|   |- v2.metadata.json
|   |- v3.metadata.json
|   |- snap29c8-1-b103.avro
|   |- snap9fa1-3-16c3.avro
|   |- d8f9-ad19-4e.avro
|   |- 0d9a-98fa-77.avro
|- data/
|   |- order_ts_hour=2021-01-26-08/
|       |- 0000-5-cae2d.parquet
|       |- 0000-1-aef71.parquet
|   |- order_ts_hour=2021-01-27-10/
|       |- 0000-3-0fa3ad.parquet
```

Nice! This would not be possible in Hive since you would have to update the entire partition!

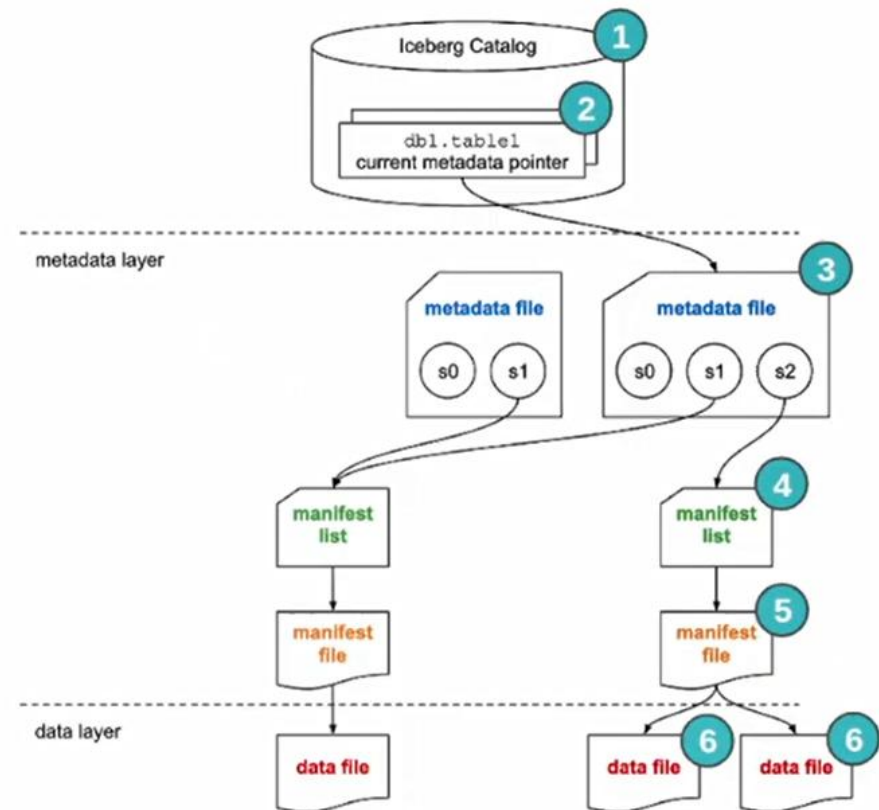


SELECT

```
SELECT *  
FROM db1.table1
```

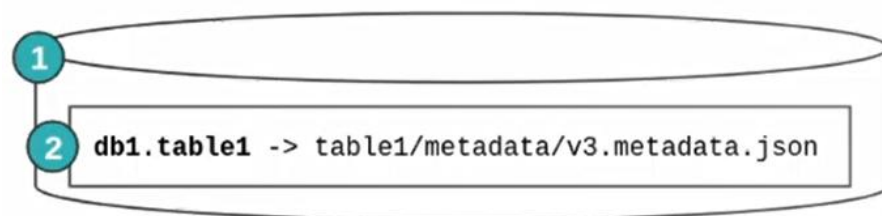


```
table1/  
- metadata/  
  - v1.metadata.json  
  - v2.metadata.json  
  3 - v3.metadata.json  
  - snap-29c8-1-b103.avro  
  4 - snap-9fa1-3-16c3.avro  
  - d8f9-ad19-4e.avro  
  5 - 0d9a-98fa-77.avro  
- data/  
  - order_ts_hour=2021-01-26-08/  
    - 00000-5-cae2d.parquet  
    6 - 00000-1-aef71.parquet  
  - order_ts_hour=2021-01-27-10/  
    6 - 00000-3-0fa3a.parquet
```

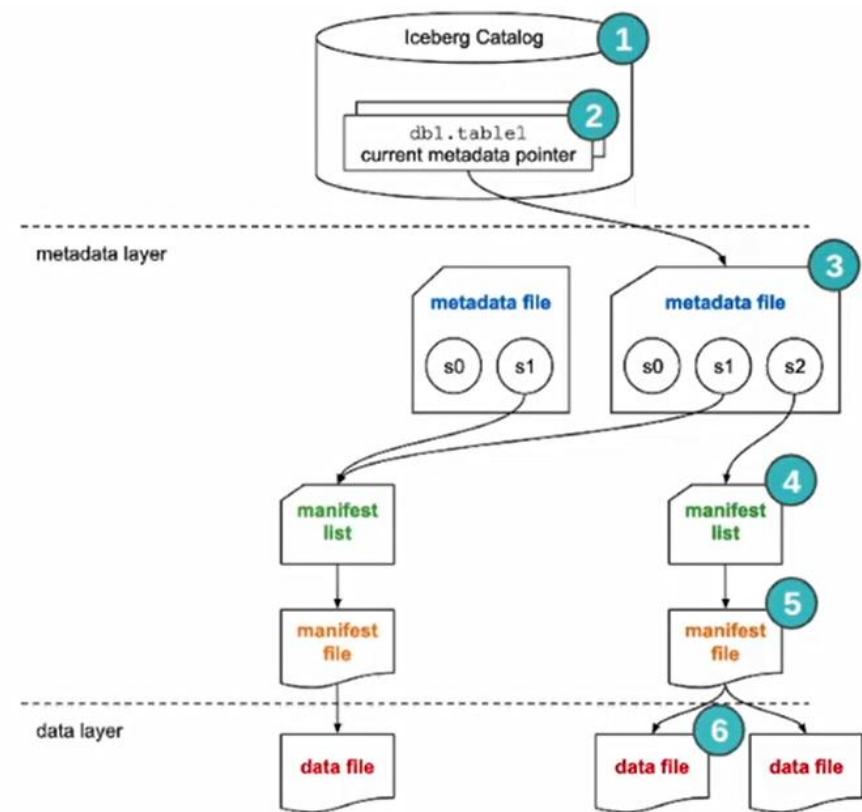


WHERE

```
SELECT *  
FROM db1.table1  
WHERE order_ts = DATE '2021-01-26'
```

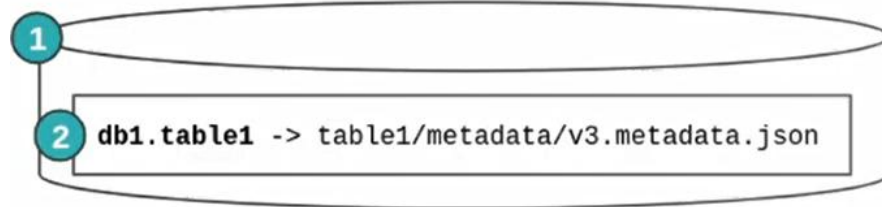


```
table1/  
- metadata/  
  - v1.metadata.json  
  - v2.metadata.json  
  3 v3.metadata.json  
  - snap-29c8-1-b103.avro  
  4 snap-9fa1-3-16c3.avro  
  - d8f9-ad19-4e.avro  
  5 0d9a-98fa-77.avro  
- data/  
  - order_ts_hour=2021-01-26-08/  
    - 00000-5-cae2d.parquet  
    6 00000-1-aef71.parquet  
  - order_ts_hour=2021-01-27-10/  
    - 00000-3-0fa3a.parquet
```

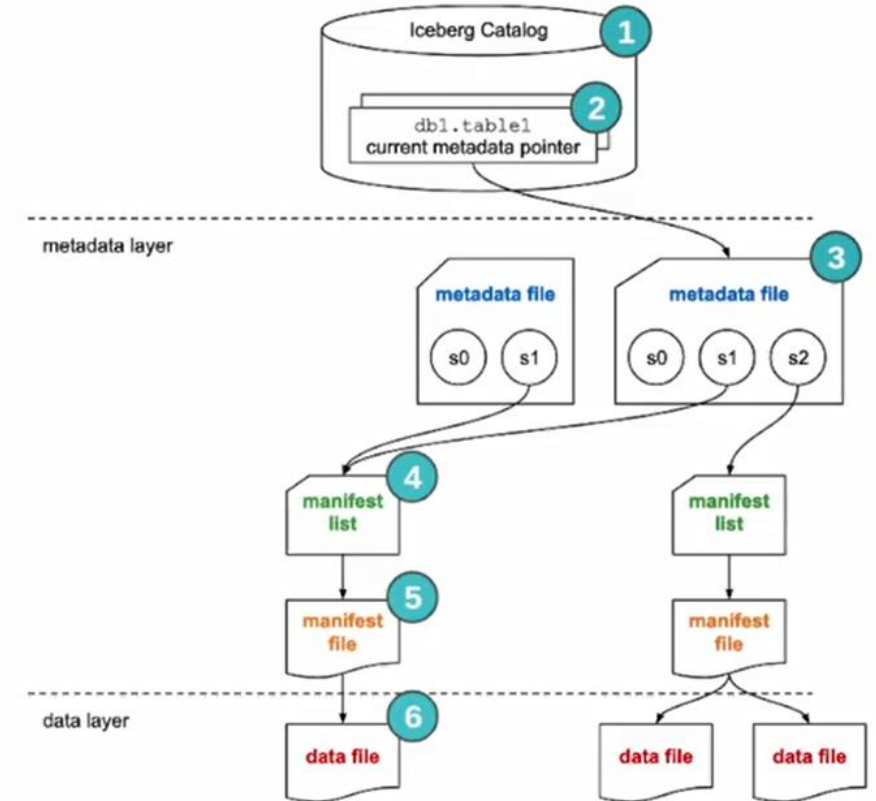


Iceberg allows you to query the data as of a specific point in time (time travel)

```
SELECT *  
FROM db1.table1 AS OF '2021-05-26 09:30:00'  
-- (timestamp is from before MERGE INTO operation)
```



```
table1/  
|- metadata/  
|  |- v1.metadata.json  
|  |- v2.metadata.json  
|  3 - v3.metadata.json  
|  4 - snap-29c8-1-b103.avro  
|  - snap-9fa1-3-16c3.avro  
|  5 - d8f9-ad19-4e.avro  
|  - 0d9a-98fa-77.avro  
|- data/  
|  - order_ts_hour=2021-01-26-08/  
|    6 - 00000-5-cae2d.parquet  
|    - 00000-1-aef71.parquet  
|  - order_ts_hour=2021-01-27-10/  
|    - 00000-3-0fa3a.parquet
```



UPSERTS

Copy-on-write

- › Whenever there is an update or delete, data files that are affected by the change will be rewritten entirely



Merge-on-Read

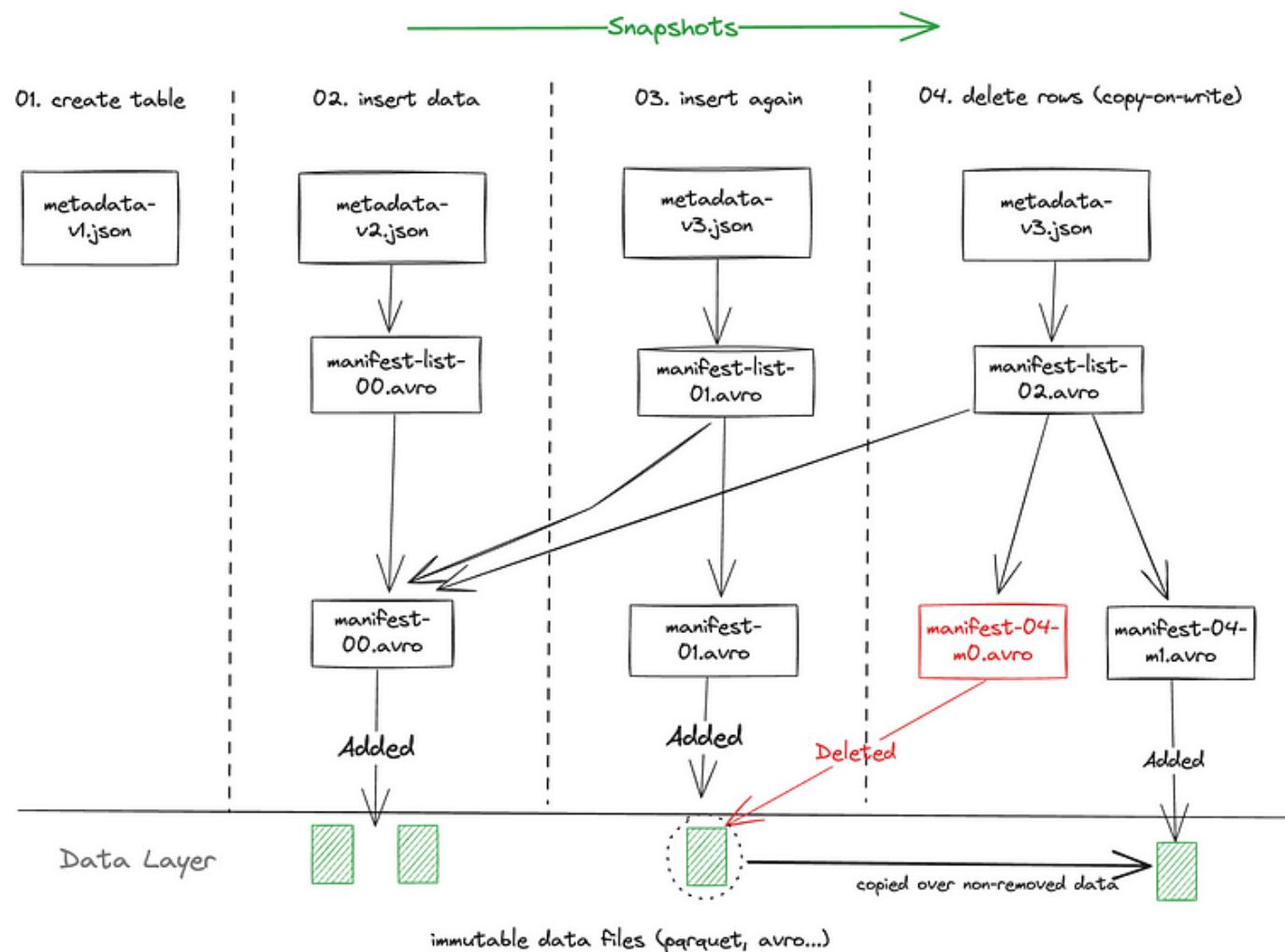
- › Whenever there is an update or delete, the rows that are updated or deleted are tracked via delete files, so they are ignored in subsequent reads. New files only written with updated rows



Example: COPY ON WRITE

Demo example

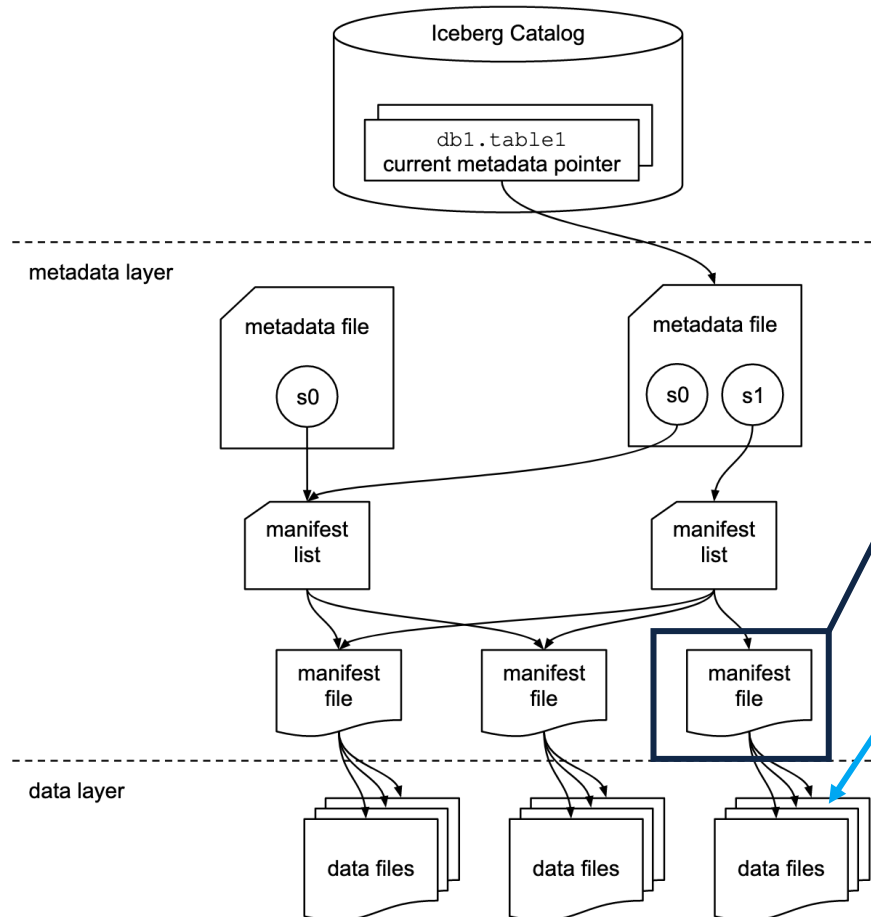
1. Create table creates the first meta data json
2. Insert data creates first files
3. Next insert create more files
4. Delete rows
 1. adds a manifest which points to a delete of a file
 2. copies the non deleted data to a new file



Source; <https://medium.com/@udayaw/demystifying-a-table-format-apache-iceberg-2e1bdceb84ab>

Metadata helps to identify which files to update: In a **MERGE INTO** statement, we only need to check files which we know contain IDs in the relevant ranger

Structure



Description

Manifest file: a list of data files, along with details and stats about each data file. Can be used to infer relevant files for a merge/join operation

```
{  
  "file_path": "/path/to/data/file1.parquet",  
  "file-format": "parquet",  
  "partition": {"part_field": {"type": <value>}},  
  "record-count": <num-records>,  
  "null-value-counts": [  
    {"column-index": "1", "value": 4}, ...],  
  "lower_bounds": [ {... } ],  
  "upper_bounds": [ {... } ],  
}, {  
  "file_path": "/path/to/data/file2.parquet",  
  "file-format": "parquet",  
  ...  
}, ...
```


Metadata helps to identify which files to update: In a `MERGE INTO` statement, we only need to check files which we know contain IDs in the relevant ranger

Existing Data

Data files	Manifest file (min/max)
data_file_1.parquet	order_id: 1 20
data_file_2.parquet	order_id: 21 40
data_file_3.parquet	order_id: 41 60
data_file_4.parquet	order_id: 61 80
data_file_5.parquet	order_id: 81 100

Staging Data

order_id	...
45	...
85	...

The data to merge contains only
ids 45 and 85. Hence, we only
need to check files which contain
those value

```
MERGE INTO db1.table1
USING (SELECT * FROM table1_stage) s
ON table1.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET
order_amount=s.order_amount
WHEN NOT MATCHED THEN INSERT *
```

Hive does not allow for column renaming without data rewriting

```
SELECT * FROM table1_hive
```

Query History

Saved Queries

Results (1)

table1_hive.order_id

table1_hive.customer_id

table1_hive.order_amount

table1_hive.order_ts

1 123

456

36.17

2021-01-26 08:10:23

```
ALTER TABLE db.table1_hive order_id order_id_new BIGINT
```

```
SELECT * FROM table1_hive
```

Query History

Saved Queries

Results (1)

table1_hive.order_id_new

table1_hive.customer_id

table1_hive.order_amount

table1_hive.order_ts

1 NULL

456

36.17

2021-01-26 08:10:23

Would need to
change name in
column of underlying
parquet file

Iceberg on the other hand allows to rename column without data rewriting

```
SELECT * FROM table1
```

Query History

Saved Queries

Results (1)



	table1.order_id	table1.customer_id	table1.order_amount	table1.order_ts
1	123	456	36.17	2021-01-26 08:10:23

```
ALTER TABLE db.table1 order_id order_id_new BIGINT  
SELECT * FROM table1
```

Query History

Saved Queries

Results (1)



	table1.order_id_new	table1.customer_id	table1.order_amount	table1.order_ts
1	123	456	36.17	2021-01-26 08:10:23

Iceberg uses a pointer
under the hood to
resolve it

This is achieved by tracking metadata of column positions rather than using names in the parquet file

Metadata v1 (schema-id 0)

```
{
  ...
  "current-schema-id" : 0,
  "schemas" : [ {
    "type" : "struct",
    "schema-id" : 0,
    "fields" : [ {
      "id" : 1,
      "name" : "order_id",
      "required" : false,
      "type" : "long"
    }, {
      "id" : 2,
      "name" : "customer_id",
      "required" : false,
      "type" : "long"
    }, ...
  ]
}
```

Number of current schema

List of all schemas

Field 1 is named "order_id"

Metadata v2 (schema-id 1)

```
{
  ...
  "current-schema-id" : 1,
  "schemas" : [ {
    "type" : "struct",
    "schema-id" : 0,
    "fields" : [ {
      "id" : 1,
      "name" : "order_id",
      "required" : false,
      "type" : "long"
    }, {
      "id" : 2,
      "type" : "timestamp",
      ...
    }
  ]
}, {
  "type" : "struct",
  "schema-id" : 1,
  "fields" : [ {
    "id" : 1,
    "name" : "order_id_new",
    "required" : false,
    "type" : "long",
    ...
  ]
}
```

Field 1 was renamed "order_id_new"

Manifest file

› No column names in the underlying manifest files, only IDs

```
...
'null_value_counts': [
  {'key': 1, 'value': 0},
  {'key': 2, 'value': 0},
  {'key': 3, 'value': 0},
  {'key': 4, 'value': 0}],
...
```

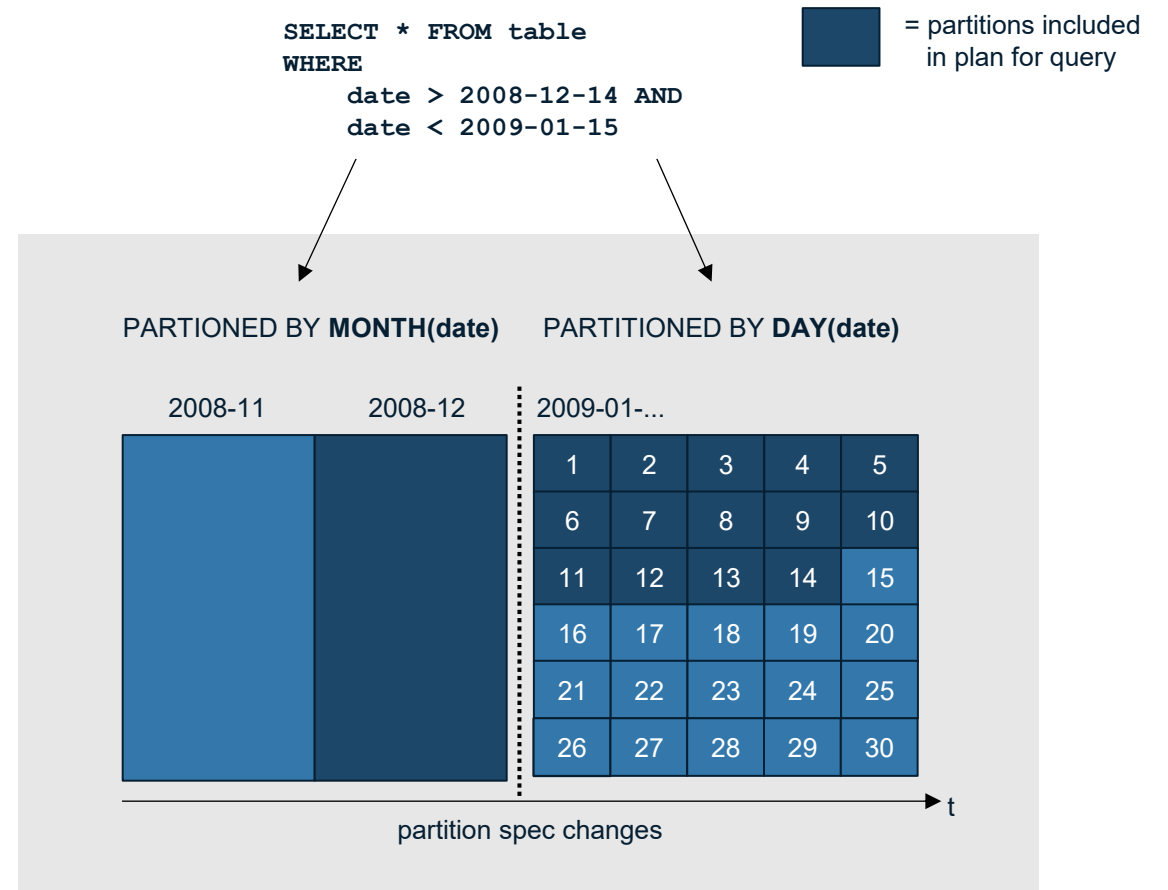
Datafile metadata has no knowledge on column names, just the order

Iceberg allows for partition evolution without rewriting data

Iceberg easily handles partition evolution

- › Partitioning strategy can be changed at any time, without having to rewrite old data
- › Old files can stay as they are; new files will be created using a different partitioning scheme.
- › Queries use meta data to understand which files to read
- › Great for datasets where velocity grows
 - › Partition big early, smaller later
 - › No need to rewrite old data
 - › Keeps file size & #files/partition similar

Illustration



Source: <https://iceberg.apache.org/docs/1.5.1/evolution/>

Iceberg exposes a logical representation of the table to the user, not a physical one which means that users can benefit from meta data without knowing so!

Iceberg is able to do logical partitioning

Partitioning a table is easy in Hive:

```
CREATE TABLE db.table1_hive (  
  order_id BIGINT,  
  customer_id BIGINT,  
  order_amount DECIMAL(10, 2),  
  order_timestamp TIMESTAMP  
)  
  
USING ICEBERG  
  
PARTITIONED BY (date(order_timestamp))  
  
STORED AS PARQUET;
```

(1) Time stamp column as part of the source data

(2) Using Iceberg we can create a partition based on the date of timestamp). The data will be partitioned by day (i.e. stored in daily folders)

```
EXPLAIN EXTENDED SELECT * FROM db.table1_hive  
WHERE order_timestamp < "2021-01-18 15:00:00"
```

(3) But the user does not have to know that! They can query by timestamp and iceberg will still only read the required files!

In Iceberg we can simply query by the column we want

In Hive, the user would have to know that there is a derived column “order_date” and filter using that

4	STAGE DEPENDENCIES:
5	Stage-0 is a root stage
6	
7	STAGE PLANS:
8	Stage: Stage-0
9	Fetch Operator
10	limit: -1
11	Partition Description:
12	Partition
13	input format: org.apache.hadoop.hive ql.io.parquet.MapredParquetInputFormat
14	output format: org.apache.hadoop.hive ql.io.parquet.MapredParquetOutputFormat
15	partition values:
16	date(order_timestamp) < "2021-01-18 15:00:00"

Iceberg is made for WAP

Apache Iceberg Integration

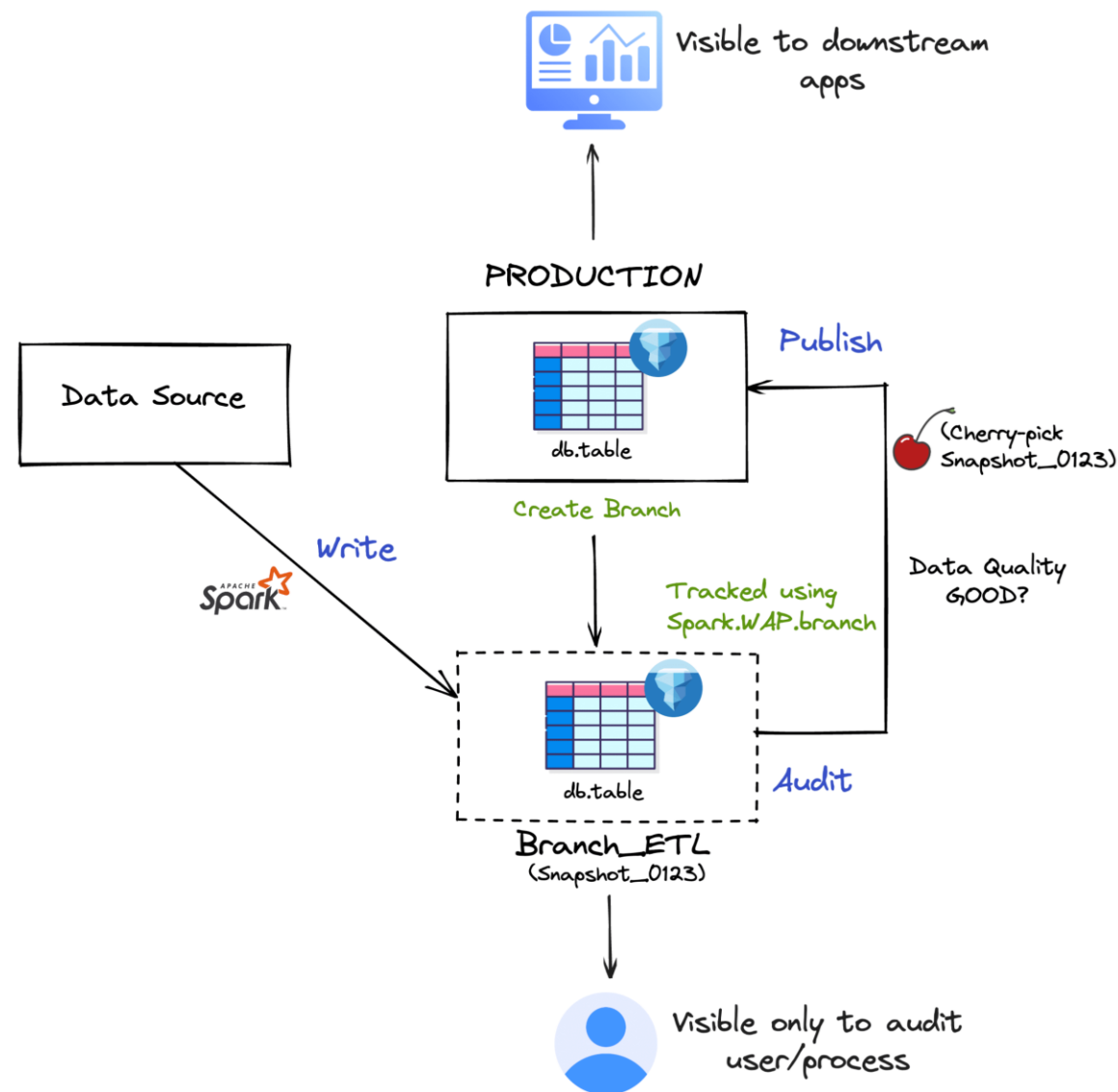
ACID Transaction Support: Iceberg's transaction model ensures atomicity of INSERT and DELETE operations, preventing partial commits during the publish phase.

Snapshot Isolation: Concurrent readers continue accessing consistent snapshots of production data while WAP processing occurs, ensuring zero downtime.

Metadata Management: Iceberg tracks all table operations through metadata files, providing complete lineage and enabling precise rollback capabilities.

Schema Evolution: Native support for schema changes allows WAP pipeline to adapt to evolving data structures without breaking existing processes.

Table Branching: Advanced Iceberg implementations can leverage branch isolation for even stronger separation between staging and production environments.



Source: <https://www.dremio.com/blog/streamlining-data-quality-in-apache-iceberg-with-write-audit-publish-branching/>

Announcing Amazon S3 Tables – Fully managed Apache Iceberg tables optimized for analytics workloads

Store tabular data at scale in S3

Amazon S3 Tables deliver the first cloud object store with built-in Apache Iceberg support and streamline storing tabular data at scale. S3 Tables deliver up to 3x faster query performance and up to 10x higher transactions per second compared to self-managed Iceberg tables stored in general purpose S3 buckets, making them specifically optimized for analytics workloads. With S3 Tables support for the Apache Iceberg standard, your tabular data can be easily queried with popular AWS and third-party query engines including Amazon Athena, Redshift, EMR, and Apache Spark. Use S3 Tables to store tabular data such as daily purchase transactions, streaming sensor data, or ad impressions as an Iceberg table in S3, and optimize performance and cost as your data evolves using automatic table maintenance. Read more on the [blog](#)

How it works



Why you should use Iceberg over Hive

Outdated Architecture

- › Both collect meta data, but Iceberg collects more and different kinds
- › Iceberg provides a layer of abstraction above the raw cloud object storage (s3,...) abstracting away the need to think of files
- › Hive tracks data at the folder level (good for HDFS, bad for object storage). Iceberg tracks data at file level (eradicating the need for costly list operations)

Performance

- › Hive is old and was never meant for the data sizes available today and frequent updates
- › Frequent list operations for updating/deleting due to the folder-based directory structure
- › Iceberg is built with change in mind. Snapshot files allow for much faster updates and deletes
- › Iceberg is faster for partitioning since it avoids irrelevant partitions

Cost

- › Schema changes are costly in Hive, so a common approach is to rewrite entire table
- › Iceberg supports in place schema evolution
- › Each select requires a list operation in each affected partition in Hive, driving up costs, Iceberg meta files avoid this

Source: <https://www.youtube.com/watch?v=qenkgl6Dni0>

Why you should use Iceberg over Hive

Easier data manipulation

- › Iceberg is ACID compliant (hive is only ACID compliant *within* a partition)
- › Iceberg can handle schema changes on the fly without the need to do expansive rewrites

Ease of use

- › Hive requires extensive rewrites, folder level updates and does not adapt to change well
- › Iceberg is built for modern data and build for change making it much easier to use

Logical data representation

- › Iceberg provides a logical representation of the data to users, while Hive provides a physical one
- › E.g if you want to use WHERE efficiently, the user must know the physical partitioning in hive even if they care about a different column (e.g. filtering on timestamp, but table is partitioned in day(timestamp))

Source: <https://www.youtube.com/watch?v=qenkgl6Dni0>